

Master Degree in Cybersecurity  
2025-2026

*Master Thesis*

# "Uptime, Churn, and Concentration: A Month-Long Empirical Study of Tor V3 Hidden Service Directories"

---

Alejandro García Berrocal

Marcelo Gabriel Bagnulo Braun  
Madrid, 14 Sep 2026

## AVOID PLAGIARISM

The University uses the **Turnitin Feedback Studio** for the delivery of student work. This program compares the originality of the work delivered by each student with millions of electronic resources and detects those parts of the text that are copied and pasted. Plagiarizing in a TFM is considered a **Serious Misconduct**, and may result in permanent expulsion from the University.



This work is licensed under Creative Commons **Attribution – Non Commercial – Non Derivatives**

# Uptime, Churn, and Concentration: A Month-Long Empirical Study of Tor V3 Hidden Service Directories

Alejandro García Berrocal  
Student, Carlos III University, Madrid, Spain,  
100451059@alumnos.uc3m.es

**Abstract**—The hidden service directories (HSDirs) of Tor’s third-generation (V3) onion service protocol store and serve the descriptors that make onion services reachable, yet their reliability and operator structure in deployment are more often specified than measured. This work presents an empirical, one-month measurement of modern V3 HSDirs, observing the network-wide directory consensus together with a selected and controlled population of static, ephemeral and third-party onion services. The method combines passive hourly auditing of the directory consensus with active descriptor probing, the latter exposing silent failures in which a directory listed as healthy cannot serve the descriptor it should hold. Across uptime, churn, operator identity, hash-ring position and descriptor hosting, the directory population is found to be highly stable in aggregate despite substantial daily churn, and uniformly distributed across the hash ring even where Sybil concentration is strong. A single declared family dominates every measured population without translating that concentration into disproportionate ring-position control, and its failures appear as ongoing partial churn rather than a collapse of the entire operator. On this basis the study shows that a single-node failure can be distinguished from an operator-level failure through family and network-prefix clustering, and that combining consensus auditing with active probing uncovers operator structure and silent hosting failures that a consensus-only view would miss.

**Index Terms**—Tor, onion services, hidden service directories, V3 protocol, hash ring, uptime, churn, Sybil concentration, operator attribution, descriptor hosting, network measurement

## I. INTRODUCTION

Tor is among the most widely used overlay networks for anonymous communication. Beyond protecting the users who initiate connections, it also provides a mechanism for protecting the operators of the services being reached. Onion services, first introduced in the mid-2000s and deployed in what became the second-generation (V2) design, allow a service to be reached without revealing its network location [1]. Each onion service is identified by a self-generated address that clients use to locate it, with the connection carried over a multi-hop circuit so that neither end learns the other’s identity or position.

The V2 design was formally deprecated by the Tor Project, with the withdrawal completed in 2021 [2], in favour of a third-generation (V3) design that addresses vulnerabilities identified in the earlier protocol. In summary, V2 relied on cryptographic primitives now considered weak and exposed onion addresses to the directory relays that indexed them, which enabled address harvesting and enumeration by anyone operating or targeting those relays [3]. V3 replaces the cryptography with modern primitives, restructures publication so that a directory no longer learns the addresses it stores, and reduces the corresponding attack surface. The mechanics of

the V3 scheme are specified by the Tor Project [4], [5] and are summarised in the background section.

The directories at the centre of this design, the hidden service directories (HSDirs), are ordinary Tor relays that have earned the responsibility of storing and serving onion service descriptors. Their reliability and their operator structure therefore bear directly on the availability and security of every V3 service, yet they are more often described from the protocol specification than measured in deployment. This work examines the current state of V3 HSDirs through direct measurement. Over a continuous one-month window it observes both the network-wide directory consensus and a selected and controlled set of onion services of three kinds, persistent (static) services, short-lived (ephemeral) services, and third-party services operated by others, recording which directories host their descriptors and how the directory population behaves over time. Alongside passive consensus auditing, the collection infrastructure actively probes for descriptors, which exposes silent failures in which a directory listed as healthy cannot actually serve the descriptor it should hold. The mechanics of this probing are described in the methodology and in the engineered collection tool.

The Evaluation is organised around six analytical themes, the basis and scope of the measurement. Directory reliability in terms of uptime and stability, churn and flag dynamics, operator identity and Sybil concentration, position within the hash ring, and descriptor-hosting behaviour, including upload-versus-fetch persistence and silent hosting failures. Together these address a single central question, determining whether an observed directory failure can be attributed to the loss of a single node or to the loss of an entire operator, where an operator is identified by its Sybil footprint across family identifiers and network prefixes. Specific measurements and their interpretation are developed in the evaluation and drawn together in the conclusions.

## II. BACKGROUND

This section presents the Tor’s onion service V3 protocol with an overview of the essential parties of the mechanism, including the onion services address generation, publication and indexing, and connection establishment [4], [6].

### A. Mechanism and Protocol Overview

The architectural ecosystem of the Tor version 3 (V3) onion service protocol implements a decentralized framework designed to facilitate mutual anonymized application-layer communication for both the client and the service provider while masking the identities and locations of both endpoints.

Unlike from centralized domain registries, the V3 architecture operates entirely within the Tor relay network, managing connections through a specialized set of relays avoiding direct network visibility of both endpoints. The public service address is split from their current IP routing path with a transient overlay and a multi-hop circuit [1]. The framework relies essential in the interaction of six operational entities distributed across the network [4], [6]:

- 1) **Tor Client:** The end-user application seeking to establish an authenticated, secure connection to the onion service without disclosing its own identity. Commonly a local proxy daemon that encapsulates the traffic, handles keys exchanges and establishes multi-hop virtual paths to mask the origins requests.
- 2) **Onion Service:** The destination instance and isolated server hosting a service or application endpoints that listen uniquely to incoming traffic through a pre-arranged anonymous path while preserving the confidentiality of its public IP address and network topology.
- 3) **Directory Authorities:** A cluster of core trusted infrastructure server nodes that collaboratively manage and monitor the network consensus status, calculate the daily Shared Random Values (SRVs), and issue the network consensus directory each hour required for layout lookups [5], [7].
- 4) **Introduction Point - IPO:** A set of publicly accessible Tor relays arbitrarily selected by the onion service to maintain long-lived and persistent circuits. The operational duty of these nodes is to accept received incoming initialization requests from clients and proxying them to the service via pre-established circuits without knowing its actual location.
- 5) **Hidden Service Directory - HSDir:** A dynamic subset of Tor relays that meet strict stability and uptime criteria. These nodes are authorized to store and serve signed cryptographic service descriptors within a decentralized and time dependent Distributed Hash Table (DHT) structure known as Hash Ring [4], [5].
- 6) **Rendezvous Point - RPO:** Ephemeral relays dynamically chosen by the client on the spot. These nodes act as the neutral nexus where the circuit branches from the client and service safely converge.

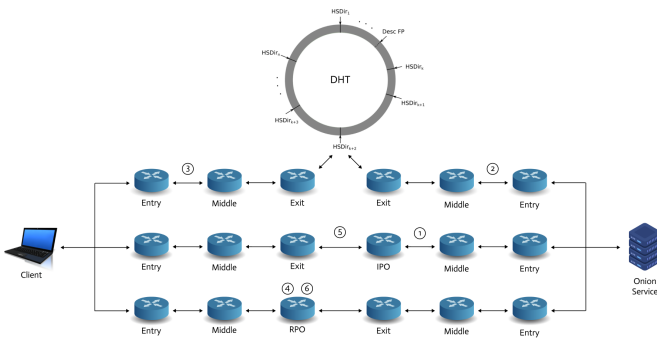


Fig. 1. Onion service structure

The chronological lifecycle required to establish an end-to-end connection between the client and a V3 onion service transitions through a six cryptographic and operational stages [4], [6]:

- 1) **IPO Setup:** Upon initialization, the V3 onion service generates its root identity key pairs, derives its cryptographic .onion address and establishes individual three-hop circuits to multiple chosen public relays to act as active IPOs.
- 2) **Descriptor Publication:** The service generates an onion service descriptor containing its public verification keys and the network routing parameters of its IPOs. This document is cryptographically signed using an Ed25519 key-blinding mechanism that daily rotates, and uploaded to a default of eight designated HSDir positions derived mathematically on the Hash Ring.
- 3) **Discovery and Descriptor Retrieval:** The client to reach the target onion address queries the active network consensus along the global SRV to pinpoint and compute the exact HSDir relays responsible of hosting the service's current descriptor and downloading the authenticated record over an anonymous circuit.
- 4) **Rendezvous Establishment:** The client designates an independent Tor relay from the active consensus pool to act as RPO, builds a separate three-hop circuit to it and establish a single-use cryptographic secret denominated as rendezvous cookie.
- 5) **Introduction Delivery:** The client dispatches an encrypted cell (INTRODUCE1), containing the RPO routing descriptors and the rendezvous cookie and encrypted using the service's public key, to one of the service's published IPOs, which blindly forwards it through its pre-established circuit to the hidden service.
- 6) **Circuit Splicing:** Upon decrypting the cell, the onion service parses the IPO cell, establishes a newly three-hop circuit to the specified RPO and hand overs a validation via a (RENDEZVOUS1) cell. The RPO then confirms the cookie matches and joins the two independent streams into a unified six-hop end-to-end circuit.

## B. Address Generation

The Tor version 3 (V3) onion routing introduces a more cryptographically and decentralized naming convention replacing the legacy 16-character version 2 (V2) address space which derivation introduced significant cryptographic vulnerabilities including partial-match dictionary collisions, namespace harvesting and lack of support for forward-secure operations [2], [3]. The V3 protocol remediates these vulnerabilities by transitioning to Ed25519 elliptic curve cryptography mapping the public key directly into the string encoding of the public identifier serving as self-authentication. This shift increases the .onion address length into a 56-character base32-encoded string, eliminating central name authorities dependencies and preventing unauthorized directory harvesting from mapping or enumerating hidden application endpoints while proving ownership [4].

The generation of a valid V3 onion address consists on the congregation of a 35-byte binary payload composed of three crucial fields, then serialized using Base32 encoding [4]:

- 1) **Master Public Key (32 bytes):** The Ed25519 master public key ( $P_k$ ) generated by the onion service during its initialization phase and root identity of the hidden service.
- 2) **Verification Checksum (2 bytes):** A truncated security chunk ( $C_s$ ) calculated derived from cryptographic hash

function to ensure string integrity and protect against typos or transmission distortions.

- 3) **Protocol Version (1 byte):** A fixed administrative byte ( $V$ ) indicating the technical generation of the protocol explicitly configured to  $0x03$  for Tor version 3 (V3).

To calculate the 2-byte verification checksum, the protocol combines a predefined static string `".onion checksum"` as a fixed salt, the 32-byte master public key, and the the protocol version byte into a unified byte stream. The stream is processed using the SHA3-256 hashing algorithm, which resulting digest is truncated to its first two bytes [4]:

$$C_s = \text{SHA3-256}(".onion\ checksum" \parallel P_k \parallel V)[ : 2] \quad (1)$$

Once the checksum ( $C_s$ ) is derived, it is appended to the master public key and the protocol version byte to construct the 35-byte vector. The resulting domain name is produced by converting the byte vector into a Base32 sequence with the domain suffix appended:

$$A_{\text{onion}} = \text{Base32}(P_k \parallel C_s \parallel V) \parallel ".onion" \quad (2)$$

This infrastructure produces direct implications for network telemetry and monitoring. Because the resulting onion address wraps the raw master key ( $P_k$ ), any external system that captures the service's address can extract the inherent cryptographic key. The master identity key acts as the fundamental parameter to calculate daily blinded keys, which govern the exact time dependent coordinates the service will occupy on the network's HRT over the given lifecycle.

### C. Publication and Indexing

To shield hidden services from targeted denial-of-service (DoS) vectors and metadata harvesting by adversarial directory servers [3], the Tor version 3 (V3) introduces a key-blinding subsystem along a consistent distributed hashing topology [4]. Rather than uploading static routing records to fixed servers, V3 onion services routinely rotates ephemeral blinded public keys ( $K_b$ ) to dynamically compute a set of time dependent storage nodes within the HRT. Services ensure confidentiality by serializing active routing vectors and cryptographic information in a structural document known as onion descriptor, protected with an architectural encryption to prevent intermediate nodes from harvesting onion addresses or identify the services being hosted by it unless they already possess the corresponding credentials.

The derivation of blinded credentials, spatial positioning and temporal rotation descriptors across the global HRT shifts are governed by two synchronized parameters daily updated within the network consensus, the global SRV and the time period, or temporal epoch [4]. The consensus cluster of directory authorities produce a newly generated SRV every consecutive interval (epoch) defined by the consensus parameter `hmdir-interval`, which defaults to an operational duration of 1440 minutes (24 hours), at 00:00 UTC over hourly voting intervals [7]:

- 1) **Commit Phase (00:00 - 12:00 UTC):** Authorities include cryptographic commitments of privately generated random numbers within their hourly consensus votes.
- 2) **Reveal Phase (12:00 - 00:00 UTC):** Authorities expose their random values, validating them against the previous

phase's commitments. At 00:00 UTC, the final unified SRV is compiled and appended to the live consensus.

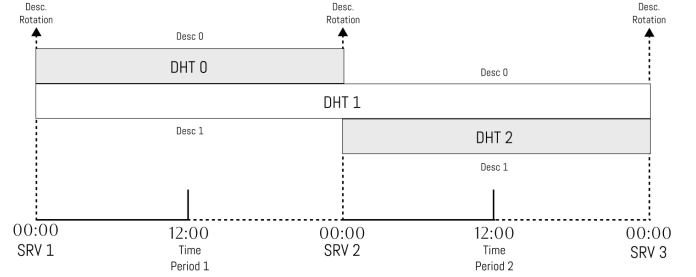


Fig. 2. DHT update

This shared secret acts a global seed that enforces pseudo-random reshuffling of the entire hash ring, preventing pre-calculation and maintenance of malicious nodes adjacent to a target's publication space [7].

To publish its onion descriptor, the hidden service applies a key-blinding function mixed with the two current synchronized temporal variables to produce the temporal blinded public key  $K_b$  [4]. For each independent publication replica the service derives a distinct 256-bit coordinate refer as the service index, where the number of replicas ( $N_r$ ) defaults to 2 replicas. The derivation of the service index for a given replica identifier ( $r$ ) is as follows:

$$HS_{\text{index}} = \text{SHA3-256}("store-at-idx" \parallel K_b \parallel r \parallel L_p \parallel N_p) \quad (3)$$

where  $K_{\text{blinded}}$  represents the blinded public key,  $L_p$  is the standardized length of the temporal epoch, and  $N_p$  is the sequential index of the current time period.

Concurrently, every active relay in the consensus that satisfies specific uptime, stability, and flag requirements (HSDir, Stable, Fast) to operate as HSDir computes its own relative position on the ring, recalculated for each temporal epoch using the global SRV [4], [5]:

$$R_{\text{index}} = \text{SHA3-256}("node-idx" \parallel ID \parallel SRV \parallel N_p \parallel L_p) \quad (4)$$

where  $ID$  denotes the cryptographic fingerprint of the relay. Once both indices are mapped onto the 256-bit circular index table of the HRT, the onion identifies the specific relays in charge of redundantly storing the current onion descriptor. The service selects the first distinct HSDir nodes ( $k$ ) whose positions on the HRT ( $R_{\text{index}}$ ) immediately succeed the hidden service positions ( $HS_{\text{index}}$ ), where the pool number of nodes ( $k$ ) is given by the consensus parameter `hmdir-spread-store` explicitly set to 4 [4]. Thus, the service initiates a sequence of encrypted descriptor uploads to these first 4 unique HSDir nodes identified for each service index computed, resulting in the descriptor upload to total of 8 distinct HSDir nodes to host and serve the record during the designated time period.

Because these positions shift daily along the global SRV, this parameter along  $N_p$ ,  $L_p$  are publicly observable through the public network consensus files which serve to track a targeted .onion address with an independent system to extract the onion's master key, compute  $K_b$ , and determine the exact 8 HSDir relays responsible of hosting the onion's descriptor any given day. This mapping provides the basis to track HSDir node uptime distributions, assess runtime stability or operator entity cluster profiling via IP range correlation [8].

#### D. Service Access

Unlike the classical Internet, where a client establishes a direct transport layer socket to an IP address, the Tor version 3 (V3) requires both the client and server to construct separate three-hop virtual circuits which congregation to a mutually agreed intermediate node is referred as rendezvous mechanism [4], [6]. This architecture ensures neither end can deduce the routing trajectory, geo-origin and underlying network topology of the opposite party.

The service access process begins with the client parsing the 56-character .onion address to locate the master public identity key and resolves the inherent Ed25519 public key, which combined with the temporal epoch and SRV retrieved from the network status consensus, the client cryptographically computes the target service  $K_b$  and derive the settled service indices mapped onto the HRT. The client then selects one of the four active HSDir relays appointed to the replica, constructs an anonymous three-hop circuit toward it, and transmits an authenticated HTTP GET request to download the signed onion descriptor records [4].

Upon receiving the payload, the client parses the descriptor utilizing the blinded key structures to verify the descriptor's cryptographic signature and decrypt its inner layer. This extracts the array of active IPOs routing vectors along with their respective public keys and link specifiers, and the service's temporal encryption keys. The connection sequence then diverges into two parallel paths [4]:

- 1) **Rendezvous Establishment:** The client selects an external relay from the consensus to act as an isolated cryptographic bridge referred as RPO. A three-hop circuit is constructed toward this node and delivered with an ESTABLISH\_RENDEZVOUS cell, which provisions the relay with a secret and an unique single-use 20-byte rendezvous cookie. Upon acknowledging the setup, the RPO registers the cookie in its active volatile session cache and awaits an inbound circuit connection that presents the corresponding token.
- 2) **Introduction Delivery:** Simultaneously, the client constructs a three-hop circuit to one of the public IPOs specified in the decrypted descriptor records. An INTRODUCE1 authenticated cell is then transmitted to it, which envelops an encrypted message containing the routing specifiers and the matching 20-byte rendezvous cookie encrypted and safeguarded under the service's onion key against intermediate inspection.

Upon receiving this cell, the IPO forwards the encrypted inner payload to the onion service via an INTRODUCE2 cell. The hidden service then decrypts this instruction block using its blinded private key, validates its integrity, draws out the RPO's routing parameters alongside the 20-byte rendezvous cookie and establishes a separate three-hop circuit to the specified RPO relay by submitting a RENDEZVOUS1 cell embedded with the matching rendezvous cookie. The RPO finally splices the incoming circuit into a unified six-hop transparent pipeline facilitating end-to-end forward-secure data streams protected with symmetric encryption keys safeguarding against node sniffing or network-wide traffic correlation attack [4].

### III. RELATED WORK

Prior research on the Tor version 3 (V3) hidden service directory divides into measurement studies characterise how

directory relays and onion descriptors behave in the live network, whereas attack studies exploit the directory to disrupt or deanonymise the services that relies on it. This section groups the closest prior works by their relationship to the measurement framework. It separates the studies that provide direct methodological basis from those that explain why particular measurements were designed the way they are.

The closest methodological predecessor is the sybilhunter system of Winter et al. [8], which mined roughly nine years of archived consensus and router descriptors to expose coordinated relay groups that are operated by a single entity. Three of its techniques map onto analyses reproduced here. Its churn technique quantifies the joining and leaving of relays between consecutive consensus using an adapted Godfrey formula split into separate join and leave series [8], the same concept computed underlying the per-flag churn distributions. Its uptime technique renders relay availability as binary online/offline sequences and sorts them with single-linkage clustering under a Pearson-correlation distance so that synchronised reboot relays become visually adjacent [8], which is the clustered uptime matrix that the present work recomputes for the tracked HSDir population. Its fingerprint technique tracks relays that repeatedly change their identity fingerprint from a fixed address, an indicative of attempts to occupy chosen positions in the descriptor hash ring [8], and this directly parallels the fingerprint-change tracking applied here to the same end. That study also records the flag thresholds, the HSDir flag requires more than ninety-six hours of uptime and that a large share of running relays act as directories [8]. Its evidence precedes the period in which V3 became common, so its data is not V3-specific, but the analytical methodology transfers unchanged to the V3 directory studied here.

The most empirical basis comes from Höller et al. [9], who operated a family of fifty relays admitted to the hidden service directory in order to observe V3 blinded-key uploads and downloads from a controlled position. The work extrapolates the number of active V3 services from the fraction of the directory it observes, and it reports a prominent asymmetry between publication and retrieval: a large majority of uploaded blinded public keys were never fetched from the authors' relays, and a majority of received download requests could not be answered because the corresponding descriptor had never been uploaded [9]. The upload-versus-fetch persistence histograms in that study are the model on which the descriptor upload and fetch persistence analysis of this thesis is based. Equally relevant is its privacy-preserving measurement design, in which timestamps are truncated and stored blinded keys are reordered before analysis to prevent the data from enabling correlation against individual users [9]. The same constraints guide the non-intrusive collection strategy here.

A second group of prior art does not provide methodology but motivates the specific stability measurements this thesis performs. Höller and Mayrhofer [10] document a distributed denial-of-service (DoS) incident against two relays belonging to the same fifty-relay directory project, in which memory exhaustion forced the targeted relays to reboot. Because the HSDir flag depends on sustained uptime, both relays lost that flag as a direct consequence of the reboot, and the disruption was not visible through the metrics that the relays report about themselves [10]. This is precisely the context that the

stripped-flag churn analysis of this thesis is designed to characterise, and the broader account of relays disappearing and returning mid-experiment, with uptime reset in the process, also supports the general churn and uptime-matrix analyses. From an unrelated research group, Zhang et al. [11] describe HSDirSniper, an attack that floods a responsible directory's descriptor cache within a single hour so that its aging mechanism purges every stored descriptor, portraying an arbitrary hidden service unreachable while the service itself observes nothing unusual [11]. That study matters as a complementary threat model. It establishes a concrete mechanism by which a directory relay listed as healthy may hold no valid descriptor for a service, which is one reason the silent-failure and upload-versus-fetch observations gathered by this thesis are treated as security-relevant signals and not merely descriptive statistics.

A more distant art, but relevant to the design of the probe population, is the provisioning-time study of Höller et al. [12]. It measures the latency of deploying an onion service and finds that the transition to V3 reduced deployment to a few seconds, with an overwhelming share of that time spent uploading the descriptor to the directory [12]. Among the configurations it measures is the ephemeral onion service, created through the control protocol and existing only while the controlling connection remains open [12]. This study motivates why cycled ephemeral onions are worth including in the study population of this thesis. It establishes that short-lived, dynamically generated services are a realistic use of the V3 directory, so probing the hash ring with daily-rotated ephemeral endpoints samples a genuine class of service rather than an artificial one. Nonetheless, it is not the source of any measurement method employed here.

Finally, the operator-diversity analysis of Sendner et al. [13] forms a distinct research thread. Working from consensus and descriptor data, that study groups relays by declared family and related attributes, and shows that a small number of operators concentrate a large share of network capacity, to the point that strategically deployed Sybil families can occupy both circuit endpoints for a quantifiable share of traffic [13]. Its family-based grouping corresponds to the declared-family and family-identifier clustering used in this thesis to detect coordinated directory operators. It also provides an important caution: evaluating sybilhunter [8], the same detector whose techniques this thesis adapts, that study reports that a prepared adversary can evade detection by diversifying nicknames, contact information, addresses, uptimes, and ports [13]. Family-based and configuration-based Sybil detection is therefore treated here as an indicator of coordination rather than as a complete defence.

In summary, this prior art measures the V3 directory from a fixed relay position, attacks it, or analyses relay-family concentration across the network as a whole. The present work instead tracks the directory from the perspective of monitored services and relays simultaneously, correlating node-level uptime and stability, churn and flag churn, hash-ring position, and family-based Sybil indicators over a continuous measurement window.

#### IV. METHODOLOGY

To investigate and analyze the architectural security, runtime, stability, behaviour and Sybil deployments within Tor version 3 (V3) HSDirs, a non-intrusive and reliable tracking

framework is designed. Rather than relying in on historical third-party directory archives which lack granularity, an automated and concurrent program was developed and executed during one-month time-period. The measurement infrastructure interacts directly with a local Tor instance via its asynchronous control port pipeline [14]. The core approach of this methodology is the real-time interception of routing descriptors and propagation, track individual relay flag churn, state fluctuations, availability on the HRT and isolate structural network identifiers from the live network.

##### A. Concurrent System Architecture and State Synchronization

The measurement framework operates continuously over a set of decoupled multi-threaded structure to isolate telemetry vectors and maintain a minimal overload and performance of Tor's control port socket. The high throughput and directory update rates across the network would bring forth thread blockages and risk frame drops on the asynchronous controller stream in a single-threaded architecture. Consequently, the architecture separates its duties across five autonomous and persistent threads under a strict lock-synchronized schema to prevent race conditions and transaction deadlocks during continuous disk write operations.

To prevent filesystem I/O bottlenecks when logging dense network events, a collection of transaction-safe locks are instantiated and uniquely mapped to individual JSON lines (.jsonl) logging files. At the same time, independent memory-state locks are allocated to manage runtime mutations to shared memory containers, preventing data corruption when multiple monitoring components concurrently access or modify relay data.

The framework restricts its data collection to all available relays on the network, a distinct target collection containing controlled static endpoints, a selected third-party endpoint probes, and a scalable pool of ephemeral addresses generated dynamically at runtime. By monitoring this mixed collection of hidden service categories, the system ensures comprehensive coverage across distinct regions of the HRT.

A five-thread collection pipeline splits with each thread targeting a specific operational routine:

- 1) **Stateful Consensus Auditing and Sybil Vector Extraction:** These threads are driven by two independent consensus tracking routines, network-wide consensus monitor and tracked HSDirs status monitor. Triggered hourly at XX:05 UTC, these modules ingest the global and tracked directory network consensus information to map the state of active global and tracked relays. The audit engine reads the microdescriptor headers of every operational relay flagged with HSDir, Stable and Fast properties [5], mapping their IP addresses, nickname, open Onion Routing (OR) ports, and cryptographic fingerprints. Implemented with a microdescriptor parser bypass, Sybil identity markers and flag clustered relays operated by coordinated entities are extracted with the capture of raw declared family lists and cryptographic famil\_ids vectors. Tracked active node that disappear from the consensus directory accumulates hourly absences considering an offline churn; a consecutive absence count of  $\geq 2$  sets off a state reassignment to a dead-offline churn, capturing long-lived relay drop rates.

- 2) **Active Outbound Prober:** A dedicated prober operating on a fixed periodic 20-minute loop. The mechanism randomizes the target request queue and dispatches raw asynchronous HSFETCH instruction lookups directly to the control port socket [14]. These queries bypass Tor’s internal circuit-level memory dependencies within the local Tor client, forcing the daemon to query the remote directory nodes directly. This method discloses silent failures where a node listed as online cannot serve requested descriptors. To achieve complete coverage a two-tier lookup is triggered, a standard dynamic hash ring resolution, and a direct poll targeted at individual fingerprints historically recorded in memory as active hosting relays. These lookup patterns compels the local Tor daemon to query the remote relays directly, causing an asynchronous event listener to log the resulting network events.
- 3) **Asynchronous Event Listener:** Thread instantiated to establish a persistent, isolated, direct subscription to the controller’s internal HS\_DESC stream event pipeline, which tracks real-time descriptor uploads, fetch requests, and operational failures [14].
- 4) **Scaled Ephemeral Manager:** To scale the observation scope and neutralize spatial placement bias inherent to a set of static domains, this runtime allocation script automatically invokes Tor control commands to dynamically deploy fifty detached, ephemeral hidden service endpoints and cycled at regular daily intervals [14]. These generated cryptographic addresses are automatically appended to the active tracking set memory, and spreaded across the HRT ensuring broader statistical sampling across the decentralized DHT ring space.

### B. Hash Ring Mapping and Low-Level Protocol Parsing

A significant engineering barrier arose upon the degradation of the primary Python control library Stem for several years since June 2023 [15], which lacks support of native parsing tailored to the modern version 3 onion service architecture. To bypass this limitation without sacrificing data granularity, custom protocol-level bypasses were introduced. Instead of using the broken library abstractions, the script builds isolated channels to prevent control port command congestion and interact directly with the raw socket stream.

While the targeted consensus auditing can directly query (`.get_microdescriptor()`) and partially parse relay consensus streams, the global consensus audit downloads unparsed consensus streams via raw `get_info("dir/status-vote/current/consensus-microdesc")` [14]. It then extracts critical metrics and processes the text files using custom-built parsing that manually extracts the global SRV, decodes base64-encoded relay components, and handles raw-byte structures to compute the 32-byte HRT coordinate positions [5].

These bypasses allows the framework to replicate the 32-byte DHT ring topology, which by capturing the global SRV and individual relay identity keys ( $K_{ID}$ ) from the consensus, and parsing the current time period ( $T_p$ ), calculates the precise position index of every directory node using native SHA3-256 hashing routine [4]:

$$R_{index} = \text{SHA3-256}(K_{ID} \parallel T_p \parallel \text{SRV}) \quad (5)$$

The tracked consensus contrary to the network-wide consensus exclusively extracts the unique 40-character hexadecimal hash ring coordinate (HSDIR\_INDEX), the base64-encoded blinded public descriptor identifier which requires knowing the target 35-byte onion service address, and the server-side validation error reasons from the asynchronous event listener, which populates the shared memory container for targeted directory nodes. Then evaluates the clockwise distance  $\Delta$  between a directory’s coordinate  $I_{HSDir}$  on the HRT and the target onion service descriptor index  $I_{onion}$  across the circular ring space  $\mathcal{M} = 2^{256}$ :

$$\Delta = (I_{HSDir} - I_{onion}) \pmod{2^{256}} \quad (6)$$

This distance is subsequently converted into a normalized position percentage relative to the total ring space to chart whether monitored directories are distributed uniformly or clustered around specific identities.

### C. Temporal Synchronization and Boundary Isolation

A foundational requisite of the measurement system is the management of daily SRV transaction and the subsequent reshuffling of the hash ring coordinates and active blinded keys, which Tor V3 protocol enforces the rotation exactly at 00:00 UTC [4], [7].

To prevent data contamination from overlapping consensus epochs, the framework enforces a strict exclusion boundary window spanning from 23:55 UTC to 00:07 UTC. During this interval, live network probes and live event captures are suspended or systematically filtered.

To provide directory authorities sufficient time to settle their consensus voting and distribute the updated SRV across the network, the framework uses staggered offsets to resume tracking:

Old ephemeral service nodes are terminated and newly deployed at 00:01 UTC. Precisely at 00:05 UTC, upon verifying a calendar day or month rollover, the active runtime directory nodes on shared memory, network-wide and targeted, are state verified on the new calendar rollover over a script routine, archived to a disk `.jsonl` log file, flushed from shared memory, and re-initialized to a blank vector slate. Active outbound network probes resume at 00:08 UTC, and tracked and network-wide consensus audits re-engaged at 00:10 UTC.

This synchronized time structure ensures that all local tracking states, rotated blinded keys, and newly designated remote directory authorities have fully stabilized across the network before data collection resumes, preserving high-fidelity of long-term churn measurements.

### D. Structural Sybil Footprint Identification

An analytical objective is differentiate random, isolated relay failures from systematic disruptions managed by large-scale coordinated node operators. When a monitored HSDir drops from the network or fails to serve valid descriptor, the framework collects structural identity markers to detect potential Sybil concentrations based on selected criteria:

- **Network Prefix Aggregation:** The framework maps IPv4 addresses of every directory node and into standard /24 subnets. This grouping isolates cases where a single operator deploys multiple discrete directory nodes into the consensus within identical data center allocations or

IP routing subnets to artificially influence a section of the HRT.

- **Declared and Effective Administrative Families:** The microdescriptor records are parsed hourly to extract both declared relay family identifiers and mutual recognized effective family lists [5]. This tracks evaluate compliance with Tor’s administrative policies and flags undeclared Sybil networks that exhibit identical uptime profiles or infrastructure features while omitting formal administrative declarations.

## V. EVALUATION

This section presents the observational results of the one-month measurement campaign and is organised as a single sequence from the measurement basis to increasingly specific findings. It commences by establishing what was observed and how the observations are scoped and filtered, so that every later result can be read against a common baseline. It then characterises the reliability of individual directory relays through their uptime and stability, followed by their churn and flag dynamics, which together describe how single nodes behave over time. Only afterwards does the analysis group relays into operators and quantify Sybil concentration, since deciding whether a failure affects one node or a whole operator requires the per-node picture to be in place first. The final subsections survey the position of directories on the hash ring and, at best granularity, to the upload and fetch behaviour of individual descriptors. The ordering is deliberate, each subsection introduces the concepts that the next relies on, and the unit of analysis widens step by step from the relay snapshot, to the operator, to the ring geometry, and finally to the descriptor.

### A. Measurement Basis and Scope

The analysis draws on five recorded data streams, each capturing a distinct view of the directory over the observation window. The *network-wide* consensus stream records hourly snapshots of every directory relay listed in the consensus, providing the population-level baseline. The *tracked* consensus stream records the same hourly snapshots but restricted to the directories responsible for the study’s own set of probed onion services, so that directory behaviour can be observed from the perspective of specific services. Two further streams, the *network-wide rotation-check* stream and the *tracked rotation-check* stream, hold the daily-rotated snapshots used to assure stability against the deliberately cycled node set, mirroring the two populations above. Finally, the *ring-event* probe log records descriptor-level upload and fetch events observed for the tracked onions, and is the only stream that operates at the granularity of individual blinded public keys rather than relays.

Two reporting conventions apply throughout the section. First, almost every analysis is computed for two *contexts*: the network-wide population drawn from the network-wide consensus stream, and the tracked population drawn from the tracked consensus stream. Second, within the tracked context the results are also broken down by *service type*, of which there are three: static controlled services (STATIC\_CONTROL) as SC constituting 50 services, ephemeral services (EPHEMERAL\_VARIABLE) as EV constituting 50 daily-rotated services, and third-party probed services

(THIRD\_PARTY\_PROBE) as TP constituting 13 services. Unless said otherwise, results are reported for both contexts, with the tracked context further split by service type.

The observation window spans one month of continuous hourly collection. To keep the temporal axis independent of the calendar dates, each distinct calendar date is normalised to a sequential label of the form “Day  $N$ ”, ordered chronologically, and all per-day results are called on these labels.

A single filtering convention dictates which observations count as a genuine presence of a relay in a given snapshot, as opposed to a transient gap in collection. Each observation carries a count of consecutive hourly absences, and the standard filter retains observations with at most one such absence, denoting a single missed snapshot as a true disappearance measurement rather than the consecutive serving as confirmation. A stricter variant of the filter admits only observations with zero consecutive absences, used by the analyses whose correctness depends on an uninterrupted presence signal, namely the churn analysis, the per-relay identity-change analyses, and the undeclared-family graph analysis.

One measurement limitation is established before any per-service-type result is reported, because it governs how those results must be read. In the tracked data a single row represents the group of a relays within the probed onion services, not one row per physical relay, so a relay that acts as directory for onions of more than one service type is counted once within each service-type subset it graps while contributing only once to the overall total. The per-service-type unique-relay counts therefore overlap and are not additive. A dedicated diagnostic quantifies this overlap directly, by collecting for each relay the set of distinct service types it was ever observed under and counting the exact combinations. The diagnostic is meaningful only where a service-type label exists, therefore it is to not run on the network-wide population, which carries no such label, but it is run on both the tracked consensus stream and the tracked rotation-check stream, since the per-service-type breakdowns of both populations require the same limitation to be read correctly.

For the tracked consensus stream, the diagnostic identifies 7,425 distinct relays carrying a usable service-type label, of which 6,519 (87.80%) were observed under two or more service types over the study period. The exact combination breakdown is given in Table I. For the tracked rotation-check stream, the same diagnostic identifies 6,943 distinct relays, of which 5,523 (79.55%) touched two or more service types, which breakdown is given in Table II. In both populations the large majority of relays serve onions across several service types, which is why the per-service-type counts reported by later subsections must be read as overlapping subsets rather than as a partition of the tracked population.

### B. HSDir Reliability: Uptime and Stability

This subsection reports the reliability of the observed directories over the observation window, understood as how long relays remain active and how stable the deliberately cycled population is from one day to the next. Proceeds from the most direct visual of presence over time, through the duration statistics that quantify it, to the rotation-check viewpoint on the cycled population.

| Service-type combination     | Relays       |
|------------------------------|--------------|
| EV + SC + TP                 | 2,049        |
| EV + SC                      | 4,365        |
| EV + TP                      | 68           |
| SC + TP                      | 37           |
| EV                           | 529          |
| SC                           | 360          |
| TP                           | 17           |
| <b>Total distinct relays</b> | <b>7,425</b> |

Table I. Service-type overlap tracked stream

| Service-type combination     | Relays       |
|------------------------------|--------------|
| EV + SC + TP                 | 1,130        |
| EV + SC                      | 4,209        |
| EV + TP                      | 134          |
| SC + TP                      | 50           |
| EV                           | 822          |
| SC                           | 573          |
| TP                           | 25           |
| <b>Total distinct relays</b> | <b>6,943</b> |

Table II. Service-type overlap rotation-check stream

The uptime matrix is the most direct rendering of which relay is present and when. Each column is a distinct relay and each row a consecutive snapshot, ordered with the earliest at the top. Every cell is coloured by one of three states, black marks an active relay in the ring at that snapshot, white marks a relay that went offline listed in the consensus, and purple marks an absent relay from the consensus.

Two representations of this same matrix are given for every context and service type, differing only in how the columns are ordered. In the plain representation the columns follow first-appearance order with each relay placed left to right by when it first entered the population, this ordering carries no behavioural meaning, and the resulting diagonal purple boundary is merely an artifact of sorting the relays by arrival time. In the clustered representation columns are instead reordered so that relays with similar uptime sequences sit adjacent. The correlation between every pair of relays' uptime sequences over the whole window is computed, converted to a distance of one minus the Pearson correlation, and passed through single-linkage hierarchical clustering, with the columns laid out in the resulting order. Two relays that rise and fall in unison are thereby placed side by side, whereas two behaving independently are separated. This reordering is what enables the one labeling the plain matrix cannot support, any run of five or more adjacent columns whose uptime sequences are exactly identical is recoloured red. Such sequences are only detectable once clustering has gathered identically behaving relays together; in first-appearance order they would be dispersed and invisible. The plain matrix therefore answers "who was up, when," while the clustered matrix regroups the same cells to answer "which relays move together." The network-wide pair is shown in Figure 3, the tracked pair in Figure 4, and the three per-service-type pairs

in Figures 18, 19 and 20.

Two features of the matrices must be read with care. The large purple share is a consequence of the matrix dimensions rather than a reliability result, with this many snapshots and this many distinct relays gathered into one figure, most relays are part of the population for only a fraction of the full window, so most columns are mostly purple by definition. The meaningful comparison is instead between the two states that describe a relay while it is genuinely part of the population, online against listed offline. Wherever a relay does appear in the consensus, black clearly and consistently outweighs white, which is barely present next to black across the figures. Relays that reach the consensus are therefore more often genuinely active in the ring than merely listed and idle, which is a positive reliability signal and is independent of how much of the matrix is purple. Separately, the clustered network-wide matrix shows several dispersed red blocks of identical uptime sequences. These are an early visual indication of synchronised behaviour across sets of relays and are noted here as a preliminary observation rather than a conclusion, the operator-grouping analysis that follows examines whether such groups correspond to common operators.

Among the service-specific matrices, that of the TP services is visually distinct from the other two. Its black activity does not form broad columns but appears in narrow, progressively later diagonal bands, and a large contiguous red block sits at the very left edge, marking a sizeable group of relays that share a near identical uptime sequence. This qualitative pattern lines up with the duration statistics reported below, where TP stands well apart from SC and EV, as with the network-wide matrix, it is the diagonal streaks and the red block that are the notable features, not the larger purple area, which is again a matter of scale.

Although the matrices indicate the presence qualitatively, the per-day duration statistics quantify its duration. For each day the analysis aggregates the session durations of the active relays that day and reports the daily mean, together with the 15th, 50th and 85th percentiles, in decimal hours. A single overall mean of the daily averages summarises each context and service type. For the tracked context a second, onion-averaged variant is also computed, rather than aggregating every relay session into one set, it first averages the session durations within each probed onion and then averages those per-onion values, so that a small number of heavily hosted onions cannot dominate the figure. Table III collects these averages, and the full per-day duration plots on which they are based are provided in the appendix. Both sets of plots extend consistently through the full 30-day observation window, with no anomalous behaviour on the final day.

The single most notable contrast in this result set is that of TP against the other two service types. Its average session duration, 8.14 hours (7.93 hours onion-averaged), is considerably shorter than those of SC and EV, whose plain averages of 14.18 and 14.16 hours sit close together, and this quantitative gap is exactly what the TP matrix's diagonal band pattern anticipates. This contrast should be interpreted with some limitation, only 13 probed onions, whereas the SC and EV populations each comprise 50. Its duration estimate is therefore derived from a substantially smaller sample and warrants more caution in any comparison that considers it separately from the SC and EV estimates. The smaller

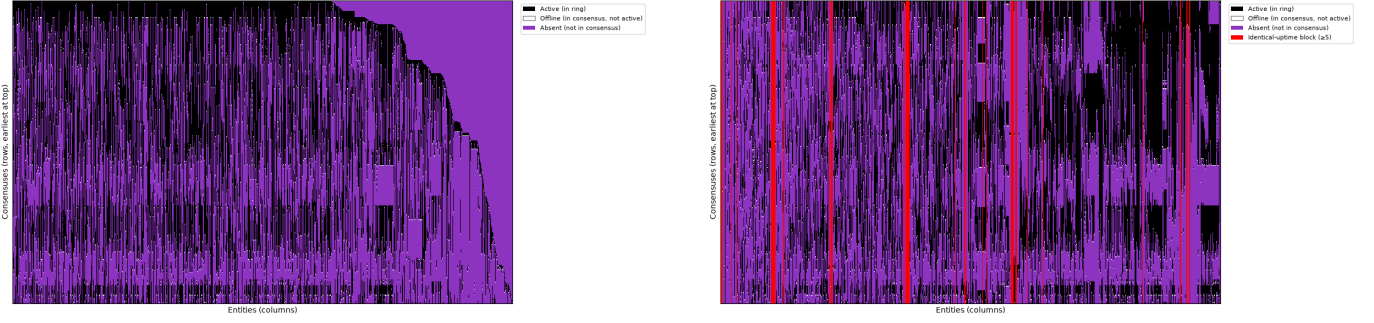


Fig. 3. Network-wide uptime matrix in first-appearance column order (left) and reordered by uptime similarity (right).

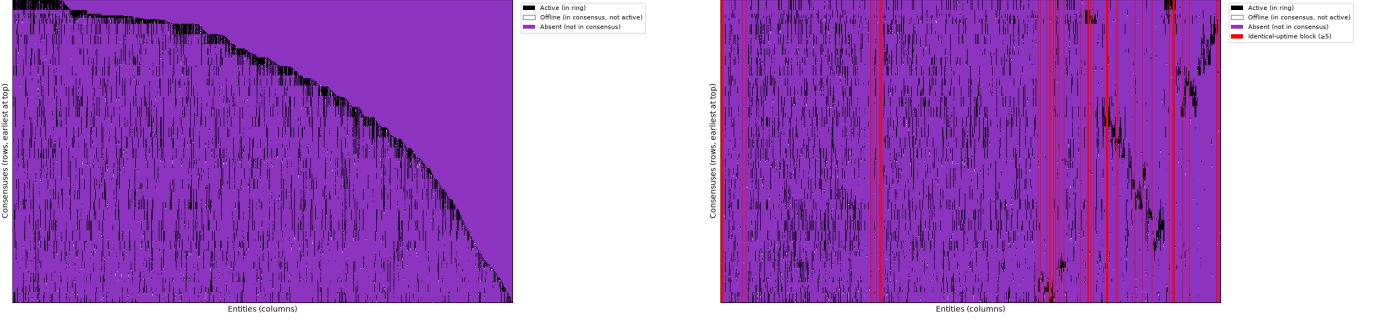


Fig. 4. Tracked uptime matrix in first-appearance column order (left) and reordered by uptime similarity (right).

sample does not, however, fully account for the pattern as demonstrated in the following paragraph.

A genuine tension in the reliability assessment concerns TP specifically and is worth stating explicitly rather than a simple side-by-side presentation of the two figures. Although TP has by far the shortest average session duration, its rotated population active share, 97.63%, is essentially identical to those of SC and EV, at 97.21% and 97.26%, all three within half a percentage point of one another, whereas the duration figures place TP well apart. These are two different quantities and need not be correlated. A population can show consistent high per-day presence, most probed relays being active on most days, while its individual sessions are considerably shorter and more fragmented, made up of frequent short disconnect and reconnect cycles rather than one long continuous span. This distinction is also why the small TP sample does not account the whole pattern, were the gap solely a small-sample artefact, both the duration and the active share would be expected to show more dispersion for TP. Only the duration measure exhibits this pattern, the active share remains within half a point of SC and EV, whereas the duration is roughly forty percent lower. The smaller sample constitutes a legitimate statistical limitation and the contrast between session duration and active fraction remains behaviourally meaningful, both coexistent.

The onion-averaged tracked figure, 12.76 hours, is slightly lower than the plain tracked average of 13.89 hours. The two differ because the onion-averaged version equalises the weight of each probed onion prior averaging, preventing a small number of onions hosted by many directories from inflate the average. The plain average, by contrast, weights every relay session equally and so reflects those heavily hosted onions more substantially. The same discrepancy holds within each service type, with the onion-averaged value slightly below the

plain throughout Table III.

| Population              | Avg. duration (h) |
|-------------------------|-------------------|
| Network-wide            | 16.68             |
| Tracked                 | 13.89             |
| Tracked, onion-averaged | 12.76             |
| EV                      | 14.16             |
| EV, onion-averaged      | 13.47             |
| SC                      | 14.18             |
| SC, onion-averaged      | 13.31             |
| TP                      | 8.14              |
| TP, onion-averaged      | 7.93              |

Table III. Average uptime duration (decimal hours)

The rotation-check stream provides a stability check on the deliberately cycled node set, complementing the consensus-based matrices and duration statistics. For each day it reports the fraction of the daily-rotated nodes that were active against the fraction that were inactive, with a service-specific breakdown for the tracked rotation-check stream. Table IV gives the overall averages. The tracked population is, on this metric, slightly more stable than the general consensus population, at 97.25% against 96.16% average active, a small but consistent difference that should not be exaggerated. Across the tracked service types the active shares are close, with TP slightly highest at 97.63% which, as already discussed, this daily stability coexists with TP's much shorter session durations rather than contradicting them.

### C. HSDir Churn and Flag Dynamics

Where the previous subsection measured how long directories remain present, this one measures how they transition between states. The churn engine classifies every entity at each

| Population           | Active (%) | Inactive (%) |
|----------------------|------------|--------------|
| Rotated network-wide | 96.16      | 3.84         |
| Rotated tracked      | 97.25      | 2.75         |
| EV                   | 97.26      | 2.74         |
| SC                   | 97.21      | 2.79         |
| TP                   | 97.63      | 2.37         |

Table IV. Daily rotated-node active and inactive fractions

hourly snapshot as one of four mutually informative events, *new*, *disappeared*, *reconnected* or *stripped* of its HSDir flag, with transitions never resetting at day boundaries, and reports each as both a count and a rate per snapshot. Two further analyses build upon it, the ring-event action rate distribution, which adds a chronological view of directory activity on the tracked population, and the stripped flag analysis, which examines what else is happening to a relay's flags at the moment its HSDir flag is lost. Following the convention of the previous subsection, the rate variant is the primary quantity of interest, the corresponding count variant exist as supplementary material but are not reproduced here.

The most apparent feature of every rate plot is a strong periodicity. The *new*, *disappeared* and *reconnected* lines spike sharply and regularly, and those spikes line up with day boundaries throughout the month rather than falling at random. This is structural rather than anomalous, and it has a concrete protocol explanation. As discussed in subsection II-C, the directory authority consensus derives a fresh shared random value once every 1440-minute epoch, compiled at 00:00 UTC, and this value drives a pseudo-random reshuffle of the entire hash ring [7]. Since each relay's ring position is recalculated relative to a value that changes daily, the refresh can cause many relays to appear to enter, leave, or re-enter a given region of the directory space, hence a substantial proportion of daily transitions naturally occurs at the daily refresh boundary. The spikes are thus best read as the visible signature of the daily ring reshuffle, not as a succession of independent disruptions. The network-wide rate plot is shown in Figure 5, the tracked plot in Figure 6, and the three per-service-type plots in Figures 7, 8 and 9.

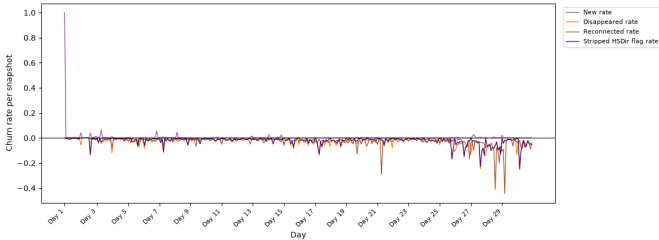


Fig. 5. Network-wide per-snapshot churn rates. *New* plotted positive; *disappeared*, *reconnected* and *stripped* plotted negative.

The rate figures for the network, tracked, EV and SC populations share another pattern. The *disappeared* and *reconnected* rates consistently and substantially exceed the *new* rate. In the tracked population, for instance, the daily rates are roughly 1.98% new against 11.19% disappeared and 10.12% reconnected, and Tables V and VI show the same ordering

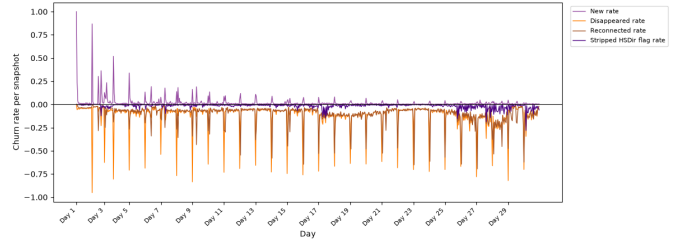


Fig. 6. Tracked per-snapshot churn rates.

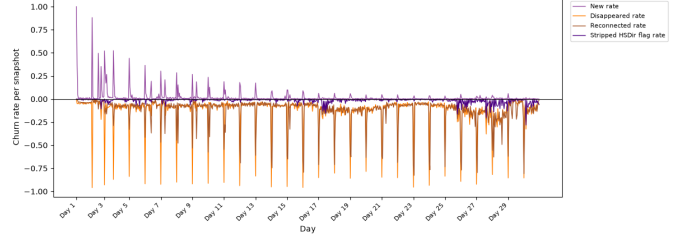


Fig. 7. EV per-snapshot churn rates.

across the other three of these four populations, following directly from the same daily-reshuffle mechanism. A reshuffle primarily moves already-known relays around the ring rather than introducing previously unseen ones, so it generates many more departure-and-return events than genuine first-time arrivals. The disappeared and reconnected rates therefore track each other closely and both sit well above the new rate, which is what the figures and tables show.

The TP population diverges from in magnitude since its churn rates are proportionally far higher than EV's or SC's, at 8.51% new and 13.13% disappeared on an hourly basis, despite much smaller absolute counts, consistent with the sample-size limitation established in the previous subsection. TP covers only 13 probed onions against 50 each for EV and SC, so each individual join or leave event is a proportionally larger fluctuation, raising the rates without necessarily indicating intrinsically more variable underlying behaviour. The elevated TP rates should therefore be read with the same

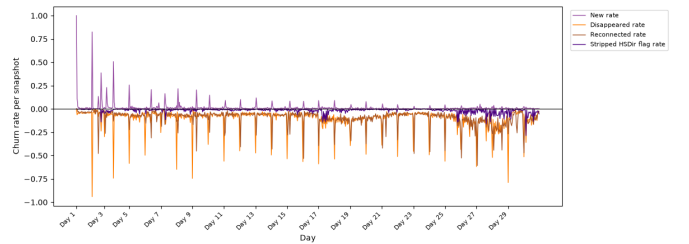


Fig. 8. SC per-snapshot churn rates.

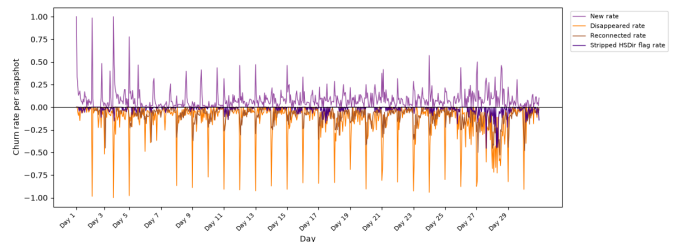


Fig. 9. TP per-snapshot churn rates.

caution applied to its shorter session durations earlier.

One churn rate, however, does not exhibit the otherwise elevated volatility observed in TP. The *stripped* rate, the rate of HSDir-flag loss, is the lowest of the four in every population, and TP is no exception. Its stripped rate of 1.77% hourly sits in the same range as EV’s and SC’s, both near 1.6%, unlike its other three churn rates which stand well above theirs. This recalls the interpretive pattern already used for the session duration and active fraction contrast in the previous subsection, where metrics that measure different things need not move together. TP is the most volatile population by arrival and departure churn yet an ordinary one by flag-loss churn, and the recurrence of this pattern across a second result set is a point of consistency rather than a coincidence.

| Population   | New  | Disap. | Recon. | Strip. |
|--------------|------|--------|--------|--------|
| Network-wide | 0.53 | 3.00   | 2.53   | 2.09   |
| Tracked      | 1.87 | 11.20  | 10.23  | 1.58   |
| EV           | 2.07 | 12.05  | 11.85  | 1.59   |
| SC           | 1.46 | 10.24  | 9.21   | 1.59   |
| TP           | 8.51 | 13.13  | 7.93   | 1.77   |

Table V. Churn hourly rates (%)

| Population   | New  | Disap. | Recon. | Strip. |
|--------------|------|--------|--------|--------|
| Network-wide | 0.52 | 3.00   | 2.53   | 2.09   |
| Tracked      | 1.98 | 11.19  | 10.12  | 1.59   |
| EV           | 2.27 | 12.07  | 11.73  | 1.59   |
| SC           | 1.53 | 10.20  | 9.12   | 1.60   |
| TP           | 8.48 | 13.06  | 7.85   | 1.76   |

Table VI. Churn daily rates (%)

The ring-event action rate distribution adds a temporal view events on top of the consensus-snapshot churn. It reports, for the tracked population and each service type, the rate at which the three ring-event actions occur, FAILED, RECEIVED and UPLOADED, both as a HSDir-level aggregate rate over all events and as an onion-averaged rate that weights each probed onion equally so that a heavily hosted onion cannot dominate. Table VII and VIII collects these figures.

The most notable feature of this distribution is that the TP breakdown contains no UPLOADED category, whereas EV and SC report all three actions. This is a definitional consequence of what third-party probing means since third-party probed onions are services the research infrastructure does not operate, and uploading a descriptor is an action only a service’s own operator performs. Against a third party’s service, only fetch attempts are possible. The missing cells are marked accordingly in Table VII and VIII.

A secondary observation concerning the TP breakdown is its FAILED and RECEIVED hour-rates sum to well above 100%, at 51.28% and 71.49% respectively, whereas EV’s and SC’s three-category breakdowns land close to 100%. A plausible interpretation is that FAILED and RECEIVED are not mutually exclusive within a given period, a HSDir may record both outcomes across different requests, so their rates need not sum to 100%.

The final analysis characterises, for every moment an HSDir flag is stripped, it records which of the six other flags,

| Population       | FAIL. | RECV. | UPL.  |
|------------------|-------|-------|-------|
| Tracked          | 24.25 | 18.69 | 57.06 |
| Tracked, o.-avg. | 21.38 | 18.56 | 60.05 |
| EV               | 23.38 | 18.57 | 58.15 |
| EV, o.-avg.      | 20.78 | 17.46 | 61.76 |
| SC               | 24.75 | 17.79 | 57.49 |
| SC, o.-avg.      | 22.04 | 16.02 | 61.93 |
| TP               | 51.28 | 71.49 | —     |
| TP, o.-avg.      | 27.05 | 72.95 | —     |

Table VII. Ring-event action hour-rate (%)

| Population       | FAIL. | RECV. | UPL.  |
|------------------|-------|-------|-------|
| Tracked          | 26.04 | 15.65 | 58.30 |
| Tracked, o.-avg. | 25.63 | 21.33 | 53.05 |
| EV               | 24.71 | 15.10 | 60.19 |
| EV, o.-avg.      | 24.31 | 15.00 | 60.69 |
| SC               | 26.84 | 14.69 | 58.47 |
| SC, o.-avg.      | 26.49 | 14.49 | 59.02 |
| TP               | 37.30 | 62.70 | —     |
| TP, o.-avg.      | 27.30 | 72.70 | —     |

Table VIII. Ring-event action day-rate (%)

Fast, Guard, Running, Stable, V2Dir and Valid remain, distinguishing HSDir-specific loss from broader relay failure. Table IX gives, for each population, the number of stripped events and the presence rate of each flag at the moment of stripping.

The clearest observation is that Guard presence at strip time is significantly lower than every other flag, in every population. Roughly 74–78% against 96–100% for the other flags. This is a notable finding of the flag-churn analysis and its explanation is that the HSDir flag is gated on a sustained uptime threshold in excess of ninety-six hours [8], and the Guard flag is governed by a comparable sustained-uptime and stability requirement [5]. Disruptions severe enough to strip a relay’s HSDir flag, a reboot or a prolonged resource-exhaustion outage that resets uptime [10], frequently disqualifies its Guard eligibility, whereas the other five flags carry less stringent or differently measured requirements and more often persist. The co-occurrence of HSDir and Guard loss is thus expected rather than incidental.

TP records both the lowest Guard presence, at 73.61%, and the lowest Stable presence, at 96.13%, of any population at strip time. Consistent with TP being the most volatile and shortest-session population overall, ties this subsection’s flag-churn results back to the uptime and stability findings.

#### D. Operator Identity and Sybil Concentration

The prior two subsections characterised directories individually. This one groups them into operators, which is what the main objective requires in order to decide, when a directory fails, whether a single node or a whole operator has gone down. Operators are approximated here by four structural indicators, the declared family identifier, IP identity (both exact address and /24 subnet), per-nickname identity mutation, and the mutual-recognition graph of undeclared families.

| Pop.    | N     | Fast  | Guard | Stab. | V2Dir  |
|---------|-------|-------|-------|-------|--------|
| Network | 19347 | 99.21 | 77.73 | 96.78 | 99.95  |
| Tracked | 13641 | 99.08 | 76.72 | 97.57 | 99.91  |
| EV      | 6277  | 99.00 | 76.49 | 97.58 | 99.94  |
| SC      | 6951  | 99.14 | 77.11 | 97.64 | 99.88  |
| TP      | 413   | 99.52 | 73.61 | 96.13 | 100.00 |

Table IX. Flag presence at HSDir-strip (%) with event count, Running and Valid assert 100(%) presence

All counts use the stricter presence filter established earlier, admitting only observations with zero consecutive absences. The full top-five rankings for every indicator and population are tabulated in the appendix. The prose below reports only the findings that hold across contexts.

One declared family dominates every population. The same family identifier, beginning `ri96phwtVtkwug/...` and hereafter the leading family, is the top group by unique-node count in every single context and service type: it holds 775 nodes network-wide (9.82% share), and its share does not diminish as the population narrows but climbs, reaching 10.44% in the tracked context and 13.54% within TP specifically. No other family approaches it, and the second-ranked group in any population sits near or below 4%. This concentration, one family accounting for roughly a tenth of the entire directory population and more of the probed subset, headline operator-identity result of this subsection, and it is visible directly in Figure 10, where the leading family’s per-day count and share track well above every other group across the whole window.

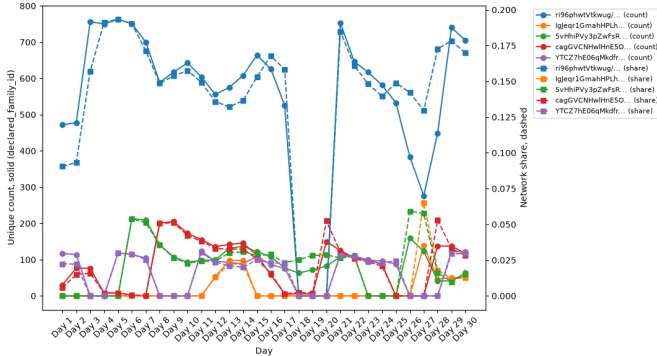


Fig. 10. Per-day unique-node count and network share for the top declared-family-identifier groups, network-wide.

The leading family dominates partial drops but never a total drop. In the operator-level drop analysis, which classifies each family’s per-snapshot behaviour as a *partial* drop (some member nodes fell) or a *total* drop (the entire family went dark at once), the leading family tops the partial-drop ranking in every population, yet it appears in no total-drop top-five. This is a statistical consequence of its size rather than a sign of instability. A family of more than 700 nodes will almost always have *some* member offline at any given snapshot, so partial drops are near-constant for it, whereas a total drop would require all 700-plus nodes to vanish simultaneously, a far rarer event whose rankings are instead dominated by small families for which bringing down every member requires only

a relatively minor failure. Partial and total drop behaviour therefore measure different things for a group this large, and the leading family’s absence from the total-drop rankings does not imply greater stability than smaller groups.

IP concentration is negligible at the exact address but substantial at the /24 subnet, with the same subnets recur throughout. Grouping directories by exact IP address reveals almost no concentration, with the busiest single address holding only 8–16 nodes in any population, never more than 0.3% share. Grouping the same directories by /24 subnet shows a contrast, with the top subnets reaching 150–320 nodes and 2–4.5% share, and the same handful of subnets, `204.8.96.0/24`, `64.65.62.0/24`, `96.9.98.0/24`, `64.65.1.0/24` and `192.42.116.0/24`, recurring across the network-wide context, the tracked context and all three service types. Figures 11 and 12 place the two groupings side by side for the network-wide context, the address plot is limited to single-digit node counts, while the subnet plot for the same population rises two orders of magnitude higher. This pattern, strong subnet-level concentration with no individual address concentration, is exactly what an operator spreading nodes across many distinct addresses within a single allocation, and it fills the gap left by the /24-based structural grouping described in the Methodology section IV-D, therefore grouping isolates operators who reuse a subnet, but nothing forces an operator to reuse an exact address, and prior work notes that a prepared adversary can deliberately diversify addresses to evade configuration-based detection [13].

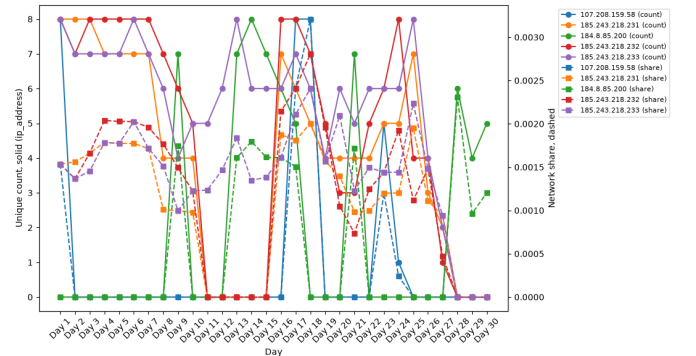


Fig. 11. Per-day unique-node count and network share for the top exact-IP-address groups, network-wide. Concentration is negligible: the busiest address never exceeds eight nodes.

Identity mutation is limited to few nicknames. The per-nickname mutation analysis counts how many distinct fingerprints, IP addresses and /24 subnets each nickname cycled through over the window. A small set of nicknames, led by `Quetzalcoat1` and `prsv`, cycled through several hundred distinct fingerprints and dozens of distinct subnets each, a known indicator of Sybil behavior since fingerprint churn shifts a relay’s hash-ring position. The full rankings are tabulated in the appendix. By contrast, the distinct-family-identifier count per nickname is uninformative, since almost every nickname carries exactly one family identifier across the whole window, and only a single nickname `drake`, at two, departs from that anywhere in the data, no ranking of that metric is reported.

The undeclared-family pattern is almost entirely between pairs of ID-less nodes. The mutual-recognition graph classifies

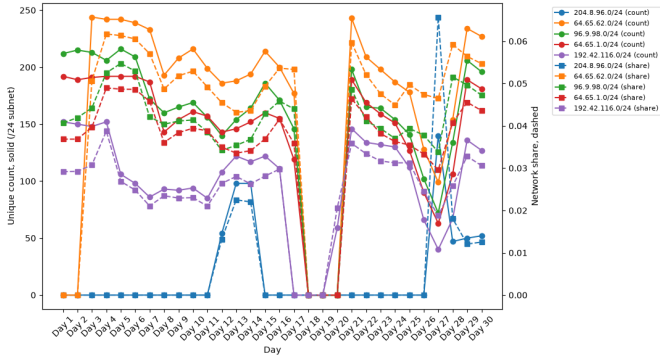


Fig. 12. Per-day unique-node count and network share for the top /24-subnet groups, network-wide, for the same population as Figure 11. The top subnets reach two orders of magnitude more nodes than any single address.

each focus node by whether its confirmed mutual neighbours carry a declared family identifier. In every population the “only-without-ID” category, mutual connections between two nodes that both lack a family identifier, exceeds the “mixed” and “only-with-ID” categories by roughly two orders of magnitude, with network-wide, averages about 812 nodes per day against 13.3 and 1.5 respectively. The effective-family pattern in this dataset is therefore primarily a matter of unlabelled nodes recognising one another, not of nodes bridging across declared identifiers. The per-day and month-wide averages for all three categories are given in the appendix.

The TP results are consistently lower in count than those for the other two service types across the drop analyses, with several partial-drop rows in the single digits where SC and EV reach dozens, consistent with the smaller probed TP population noted in the earlier subsections.

#### E. Hash-Ring Position

The structural question asked is where directories sit on the hash ring and how much of it they occupy, two analyses cover it. A ring-position uniformity analysis, which measures whether observed ring indices are spread evenly or clustered, and a coverage analysis, which measures what fraction of the ring is covered at all. Both are computed for the two absolute ring-index metrics, the HSDir index (the ring position of the responsible relays) and the onion index (the ring position of the onions’ own descriptor identifiers). The relative onion-to-HSDir distance metric is included in the uniformity analysis but not the coverage one, for which a relative distance has no meaning.

Ring-position placement is uniform everywhere, as the protocol predicts. A relay’s ring index is a SHA3-256 hash of its identity, the shared random value and the epoch, and an onion’s index is derived the same way from its blinded key [4], [5]. A well-distributed hash spreads its outputs evenly across the ring, so uniform placement is the expected result and confirming it empirically is the point. For every population examined, the measured cumulative distribution function of the normalized ring index closely tracks the ideal uniform diagonal. Figure 13 shows the network-wide HSDir-index case, whose empirical curve is visually indistinguishable from the ideal line. The one curve that departs visibly from a smooth diagonal is TP (Figure 14), which is noticeably steppier, but it still follows the diagonal, and the steppedness is

a sample-size artifact of drawing a curve from far fewer points rather than any genuine departure from uniformity. Uniformity therefore holds even for the smallest population.

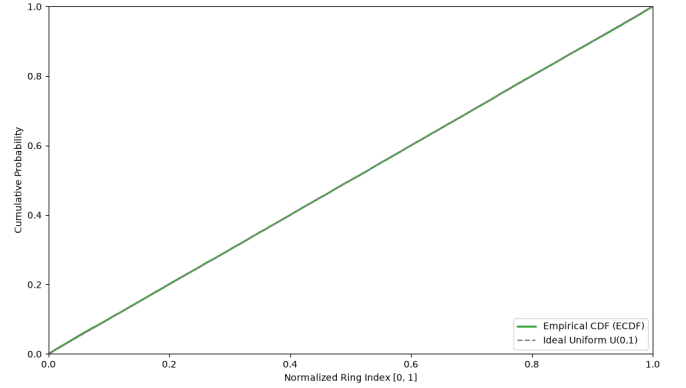


Fig. 13. Empirical distribution function of the normalized HSDir-index ring position, network-wide. The empirical curve sits on the ideal uniform diagonal, confirming even placement across the ring.

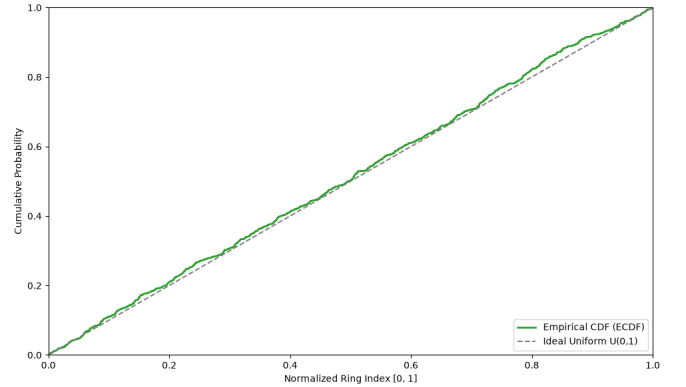


Fig. 14. The same distribution for THIRD\_PARTY\_PROBE. The curve still tracks the diagonal but is visibly steppier; this reflects the far smaller number of observed points, not a departure from uniform placement.

Daily churn and ring-position dispersion are unrelated. The uniformity analysis also pairs each day’s total HSDir churn against how dispersed that day’s ring positions were. Across every population the fitted trend is flat or near-flat, with the network-wide slope at roughly 0.003 (Figure 15). How many directories joined or left on a given day carries essentially no relationship to how spread out that day’s positions were. This is a null result with no further elaboration.

Ring coverage revisits a pattern seen twice already. The month-wide aggregate figure and the daily-average figure diverge sharply. Coverage is measured at a fixed resolution, discretizing the normalized ring into ten thousand equal-width buckets and reporting the fraction of buckets touched, both across the whole month and averaged over per-day values. For the HSDir index, the month-wide coverage saturates near completeness for the network, tracked, EV and SC populations, between 93.42% and 100.00%, while the corresponding daily-average coverage sits far lower, around 30–37% for the network and tracked contexts and 9–18% for EV and SC. As in the uptime and churn subsections, this divergence is expected rather than contradictory since only a fraction

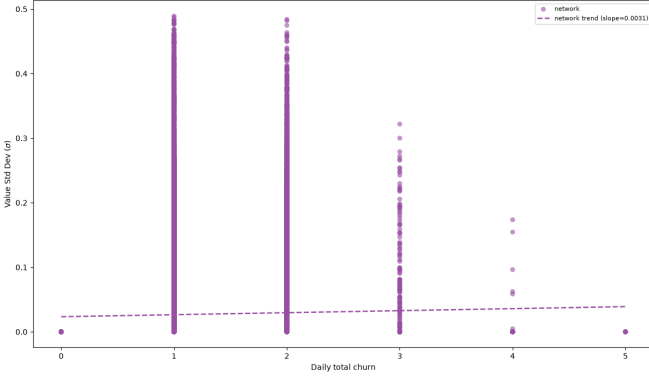


Fig. 15. Daily HSDir churn against daily ring-position dispersion, network-wide. The fitted trend line is essentially flat (slope  $\approx 0.003$ ), indicating no relationship between the two.

of the month’s distinct directories are active on any single day, therefore daily coverage is limited, yet the full month’s population is large enough to eventually touch nearly every bucket. The exact percentages are given in Table LXXIV.

TP is the exception, and it is the sharpest illustration so far of the small-population limitation that has recurred throughout this section. Alone among the populations, its month-wide HSDir-index coverage does not even reach 30% (28.98%), whereas every other population saturates near 100%. The responsible HSDir population that the probed onions touch is small enough that, even cumulatively across the entire month, it cannot approach full ring coverage. The remaining uncovered fraction is not merely a per-day effect but a whole-month one.

Onion-index coverage is far lower than HSDir-index coverage throughout, with the tracked context reaching 42.19% against the HSDir index’s 99.97%. This is an expected scale effect rather than a concern, there are always far fewer distinct onion identities than distinct responsible directories, so the upper bound on how much of the ring the onion index can touch is lower even when pooled across the whole month. Within that lower upper bound, TP’s onion-index coverage (4.20%) again sits well below EV’s and SC’s (18.66% and 18.98%), consistent with TP’s much smaller probed-onion population noted earlier.

#### F. Descriptor Hosting: Upload/Fetch Persistence

The final analysis moves in granularity, from the relay to the descriptor. It operates on the ring-event probe log at the level of the individual blinded public key, grouped by the composite descriptor key that pairs each blinded key with its onion address, and it exists only for the tracked population, since descriptor-level upload and fetch events are recorded only for the onions this study operates or probes. Three views, the per-descriptor upload and fetch counts, their distinct-hourly-bucket complement, and a silent-failure classification that flags descriptors published but never retrieved, or retrieved but never published. The per-descriptor totals, hourly-persistence figures and silent-failure percentages for the tracked population and each service type are collected in the appendix.

The distribution of upload counts per descriptor is strongly bimodal, consistent with prior reported measurements. Most descriptors are uploaded only a handful of times, resulting in a pronounced peak at the lower end of the histogram,

but a second substantial cluster occurs in the uppermost truncated interval, with roughly one-third of the tracked descriptors exhibiting high persistence rather than a gradual decline in upload frequency. Figure 16 shows this for the tracked population. This bimodal pattern echoes the related V3 measurement study, whose blinded-public-key upload and fetch distributions similarly distinguishes descriptors uploaded only a few times from a separate cluster that is persistently republished around the observed mean [9]. Although the measurement setups differ and the numerical values are therefore not directly comparable, both studies exhibit a similar distributional shape.

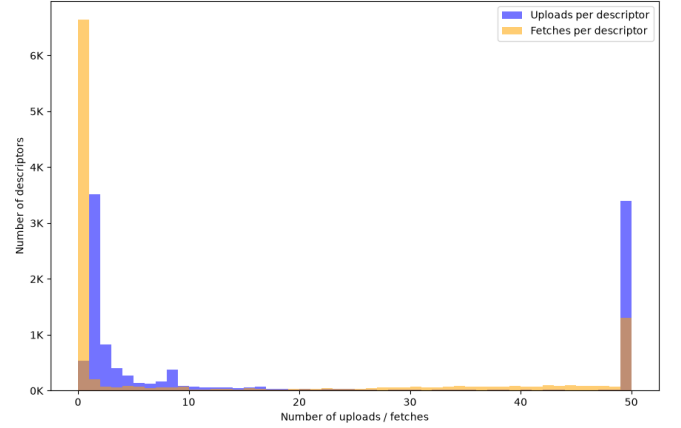


Fig. 16. Per-descriptor upload and fetch counts, tracked population. Uploads (in blue) show a strongly bimodal shape: a large spike at low counts and a second cluster at the clipped upper bin ( $\sim 50$ ). Fetches (in orange) concentrate near zero.

A secondary feature of these histograms distinguishes the two owned service types. In the SC population, fetches as well as uploads cluster in the same upper truncated interval, identifying descriptors that are both frequently uploaded and frequently fetched. In contrast, the EV population lacks a comparable secondary fetch cluster, with fetches remaining concentrated near zero. A cautious interpretation is that static, persistent onion services accumulate repeat client fetches over time, whereas daily-rotating ephemeral services are too short-lived to develop comparable traffic. The corresponding service-type histograms are provided in the appendix.

The classification of silent failures follows directly from the owned–probed distinction, which is intrinsic to its definition. Figure 17 shows the two silent-failure categories by service type. The two owned service types, EV and SC, occupy almost the entirety of the “uploaded, never fetched” category and are essentially absent from “fetched, never uploaded”, whereas TP is the mirror image, occupying “fetched, never uploaded” and registering no instances in “uploaded, never fetched.” This pattern follows structurally from the probing model. TP services are only fetched, never uploaded, because the infrastructure does not operate them. Consequently, TP records no failed or maximum uploads and no descriptors in the “uploaded, never fetched” category, while 95.88% fall into “fetched, never uploaded”, a definitional result rather than an anomaly.

The aggregate “fetched, never uploaded” rate of 3.70% is driven almost entirely by TP’s structurally distinct 413 probed descriptors, of which the 95.88% outlier fall into this category.

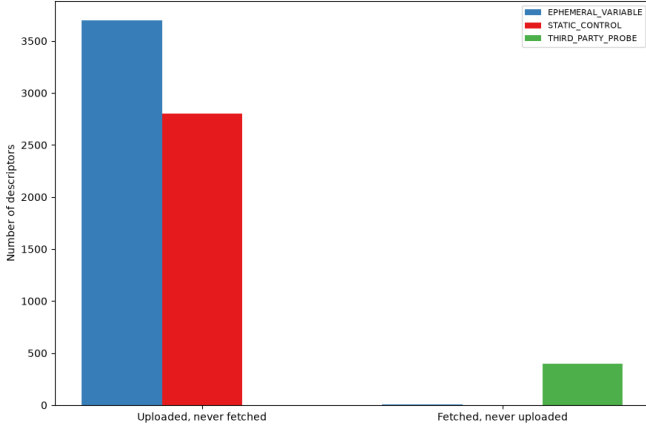


Fig. 17. Silent-failure descriptor categories by service type, tracked population. EV and SC occupy the “uploaded, never fetched” category almost exclusively; TP occupies “fetched, never uploaded” almost exclusively. Each population is essentially absent from the other category.

By contrast, EV and SC rates are negligible at 0.08% and 0.05%, respectively. The “uploaded, never fetched” rate is consistent across owned services, 59.78% for EV and 65.48% for SC, compared with 59.76% overall and therefore provides the more meaningful hosting signal.

Failed uploads are more concentrated per descriptor for the static service type. Across the tracked population, 8431 failed-upload events fall on 2478 distinct descriptors. Split by owned type, the totals are similar, 4222 events for EV and 4209 for SC, but they are distributed differently. SC’s events fall on fewer descriptors (1034, about 4.1 events each among those affected) than EV’s (1444, about 2.9 each). The static type concentrates its upload failures on a smaller set of repeatedly failing descriptors. The exact per-descriptor totals are given in the appendix.

Finally, a methodology note on the descriptor key. A modest share of raw identifiers span more than one service type or onion address in the upstream log, 1743 and 2608 such identifiers respectively. This is a known collision, and the composite descriptor key already used throughout resolves it by splitting each colliding identifier per onion, so the specific service-type totals reconcile cleanly against the tracked total.

## VI. CONCLUSIONS

This study observed a mixed population of static, ephemeral and third-party onion services over a continuous one-month window, together with the network-wide directory consensus, in order to characterise the reliability, operator structure and hosting behaviour of modern V3 hidden service directories and, on that basis, to address the central question of whether a directory failure reflects a single node going down or a whole operator.

A single structural cause explains the third-party population’s behaviour throughout. The TP’s services behaved distinctly in every analysis, with the shortest uptime sessions (Table III), the highest proportional churn rates (Tables V and VI), the clearest example of the coverage-population size effect, with an incompleteness ring coverage (Table LXXIV), and no upload activity whatsoever in either ring events or descriptor-hosting. These are not independent results,

since the third-party set covers a far smaller probed population, thirteen onions against fifty each for the two owned types, so every join-departure is a proportionally larger movement and every distribution is drawn from fewer points. And by definition, the infrastructure never controls the services, so its descriptor-hosting profile is one of fetches without uploads by construction. Read together, the third-party results form a small, structurally distinct control group whose differences indicate that the measurement behaves as designed rather than revealing anomalies in the directories themselves

Aggregate stability coexists with daily volatility across the measured population. A recurring pattern was observed independently in the uptime, churn and ring-coverage results, whereas the monthly measures indicated near-complete stability, the corresponding daily measures reflected substantially lower stability. Month-wide ring coverage nears completeness for the network and owned populations while daily coverage remains far lower (Table LXXIV). The rotated-directory population is highly stable in aggregate, at 97.25% active for the tracked set against 96.16% network-wide (Table IV), even as churn events cluster closely at the daily consensus boundary. This divergence is not a contradiction but a property of the population itself. Individual directories change daily, yet the population is large and persistent enough to remain collectively stable and to eventually cover nearly the entire hash ring. A single-day snapshot therefore underestimates the directory service’s long-term stability, an important distinction when assessing system availability.

The primary question has a direct answer, operator-scale presence is real and detectable, but does not translate into ring-position control. On one hand, operator-scale Sybil presence is evident and measurable through structural clustering, one declared family dominates every population holding roughly a tenth of the entire directory set and a larger share as the population narrows (Table X). And a small, recurring set of /24 subnets concentrates directories across every context while exact-address concentration stays negligible, consistent with an operator distributing nodes across multiple addresses within the same allocation. On the other hand, this concentration does not translate into disproportionate control of the ring. Position placement remains uniform even for populations with strong Sybil concentration (Figure 13), as the SHA3-256 derivation of ring indices predicts [4], [5]. The operator-level drop analysis clarifies the source of failures. The dominant family tops the *partial*-drop rankings in every population yet appears in no *total*-drop ranking, whereas total drops are dominated by small families. A failure confined to a few directories of the dominant family is therefore the common case and reflects single-node churn within a large operator, while a whole-operator outage of that family was never observed. Total drops, when they occur, belong to small operators for whom losing every node is a minor event. Family and subnet clustering make it possible to distinguish single-node failures from operator-level failures, with the largest operator’s Sybil presence appearing primarily as distributed partial churn rather than as concentrated control over ring positions or coordinated, population-wide failure.

All together, these findings indicate that the modern V3 directory population is, at network scale, reliable, uniformly distributed and resistant to any single operator dominating the ring, nonetheless the population exhibits a distinct large-

operator footprint whose failures appear as persistent partial churn. Combining consensus auditing with active descriptor probing, proved operator structure and silent hosting failures that would remain undetected through consensus analysis alone, representing the study’s principal contribution. The study is limited by its scope, a single one-month window, a probed set of tens rather than hundreds of onions, and an operator-attribution approach resting on declared families and network-prefix clustering, which a deliberately evasive operator could work to obscure. Future work should therefore extend the observation window, expand the probed population, and validate structural operator indicators against behavioural evidence.

#### REFERENCES

- [1] R. Dingledine, N. Mathewson, and P. F. Syverson, “Tor: The second-generation onion router,” in *Proceedings of the 13th USENIX Security Symposium*, San Diego, CA, USA: USENIX Association, Aug. 2004, pp. 303–320. Accessed: Jul. 19, 2026. [Online]. Available: <https://www.usenix.org/conference/13th-usenix-security-symposium/tor-second-generation-onion-router>
- [2] D. Goulet. “Onion service version 2 deprecation timeline,” The Tor Project, Accessed: May 21, 2026. [Online]. Available: <https://blog.torproject.org/v2-deprecation-timeline/>
- [3] A. Biryukov, I. Pustogarov, and R.-P. Weinmann, “Trawling for tor hidden services: Detection, measurement, deanonymization,” in *Proceedings of the 2013 IEEE Symposium on Security and Privacy*, San Francisco, CA, USA: IEEE, May 2013, pp. 80–94. doi: 10.1109/SP.2013.15 Accessed: Jul. 19, 2026. [Online]. Available: <https://ieeexplore.ieee.org/document/6547103>
- [4] The Tor Project. “Tor rendezvous specification - version 3,” Accessed: May 21, 2026. [Online]. Available: <https://spec.torproject.org/rend-spec/>
- [5] The Tor Project. “Tor directory protocol, version 3,” Accessed: May 21, 2026. [Online]. Available: <https://spec.torproject.org/dir-spec/>
- [6] The Tor Project. “How do onion services work?” Accessed: May 21, 2026. [Online]. Available: <https://community.torproject.org/onion-services/overview/>
- [7] The Tor Project. “Tor shared random subsystem specification,” Accessed: May 21, 2026. [Online]. Available: <https://spec.torproject.org/srv-spec/>
- [8] P. Winter, R. Ensafi, K. Loesing, and N. Feamster, “Identifying and characterizing sybils in the tor network,” in *Proceedings of the 25th USENIX Security Symposium*, Austin, TX, USA: USENIX Association, Aug. 2016, pp. 1169–1185. Accessed: Jul. 19, 2026. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/winter>
- [9] T. Höller, M. Roland, and R. Mayrhofer, “On the state of V3 onion services,” in *Proceedings of the ACM SIGCOMM 2021 Workshop on Free and Open Communications on the Internet (FOCI ’21)*, ACM, Aug. 2021, pp. 50–56. doi: 10.1145/3473604.3474565 Accessed: Jul. 19, 2026. [Online]. Available: <https://doi.org/10.1145/3473604.3474565>
- [10] T. Höller and R. Mayrhofer, “A case study on DDoS attacks against Tor relays,” in *Free and Open Communications on the Internet (FOCI)*, 2024, pp. 64–67. Accessed: Jul. 19, 2026. [Online]. Available: <https://www.petsymposium.org/foci/2024/foci-2024-0014.php>
- [11] Q. Zhang, Z. Teng, X. Wang, Y. Gao, Q. Liu, and J. Shi, “HSDirSniper: A new attack exploiting vulnerabilities in Tor’s hidden service directories,” in *Proceedings of the ACM Web Conference 2024 (WWW ’24)*, Singapore: ACM, 2024, pp. 1812–1823. doi: 10.1145/3589334.3645591 Accessed: Jul. 19, 2026. [Online]. Available: <https://doi.org/10.1145/3589334.3645591>
- [12] T. Höller, T. Raab, M. Roland, and R. Mayrhofer, “On the feasibility of short-lived dynamic onion services,” in *2021 IEEE Security and Privacy Workshops (SPW)*, San Francisco, CA, USA: IEEE, May 2021, pp. 25–30. doi: 10.1109/SPW53761.2021.00012 Accessed: Jul. 19, 2026. [Online]. Available: <https://doi.org/10.1109/SPW53761.2021.00012>
- [13] C. Sendner, D. Schreider, and A. Dmitrienko, “Impact analysis of sybil attacks in the Tor network,” in *ICT Systems Security and Privacy Protection (IFIP SEC 2025)*, ser. IFIP Advances in Information and Communication Technology, vol. 746, Springer, 2025. doi: 10.1007/978-3-031-92886-4\_13 Accessed: Jul. 19, 2026. [Online]. Available: [https://doi.org/10.1007/978-3-031-92886-4\\_13](https://doi.org/10.1007/978-3-031-92886-4_13)
- [14] The Tor Project. “TC: A tor control protocol (version 1),” Accessed: May 21, 2026. [Online]. Available: <https://spec.torproject.org/control-spec/>
- [15] The Tor Project. “Stem: Python controller library for tor.” GitHub repository; latest release v1.8.2, June 2023, Accessed: May 21, 2026. [Online]. Available: <https://github.com/torproject/stem>

## APPENDIX A RESULTS

### A. HSDir Reliability: Uptime and Stability

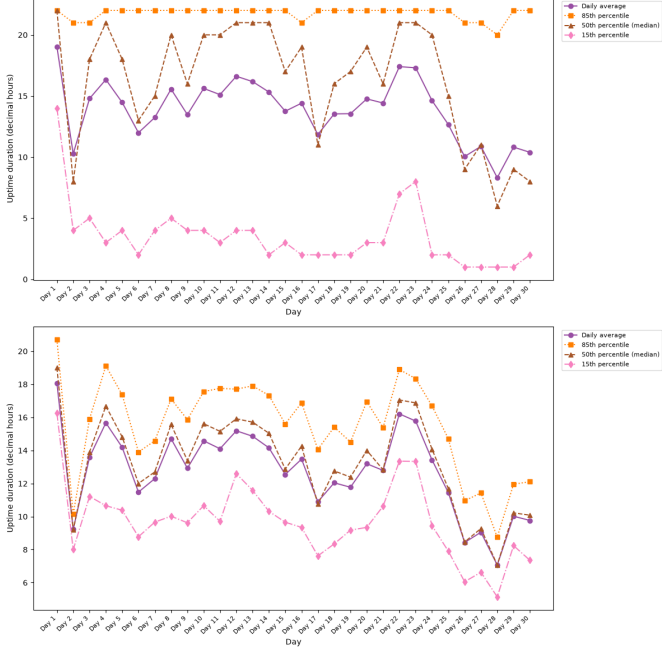


Fig. 21. Tracked per-day uptime durations: plain (top) and onion-averaged (bottom).

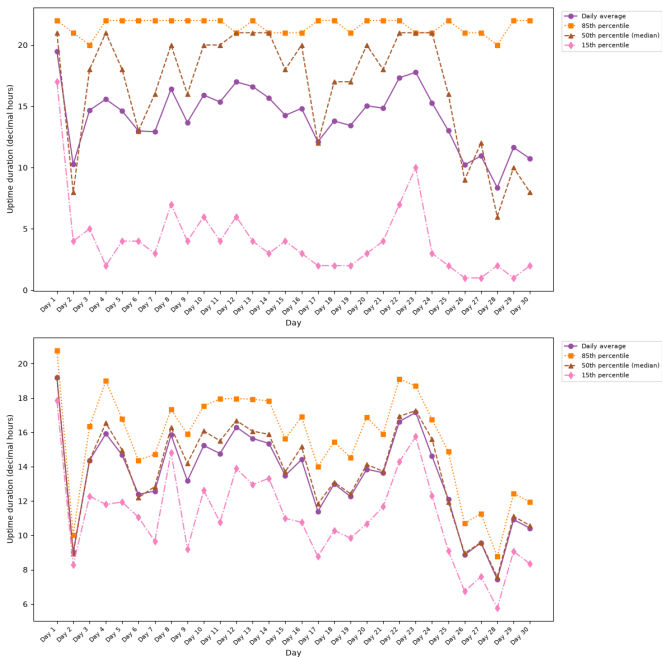


Fig. 22. EV per-day uptime durations: plain (top) and onion-averaged (bottom).

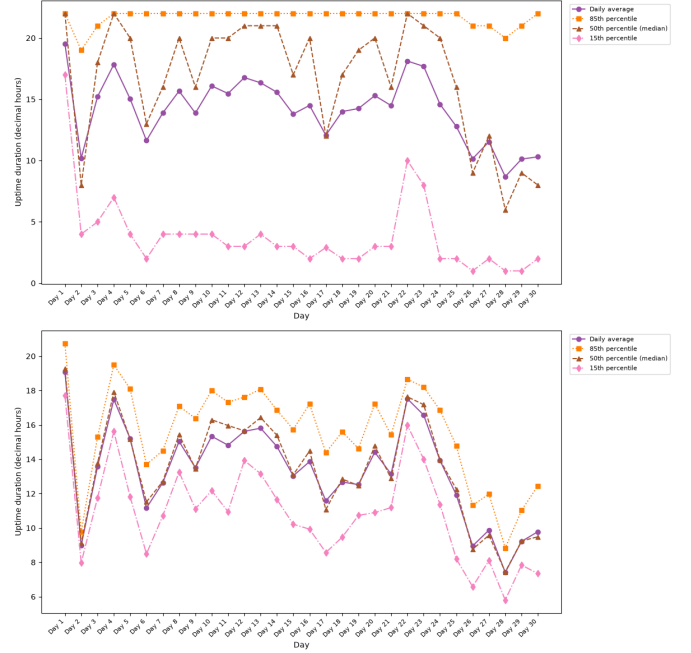


Fig. 23. SC per-day uptime durations: plain (top) and onion-averaged (bottom).



Fig. 24. TP per-day uptime durations: plain (top) and onion-averaged (bottom).

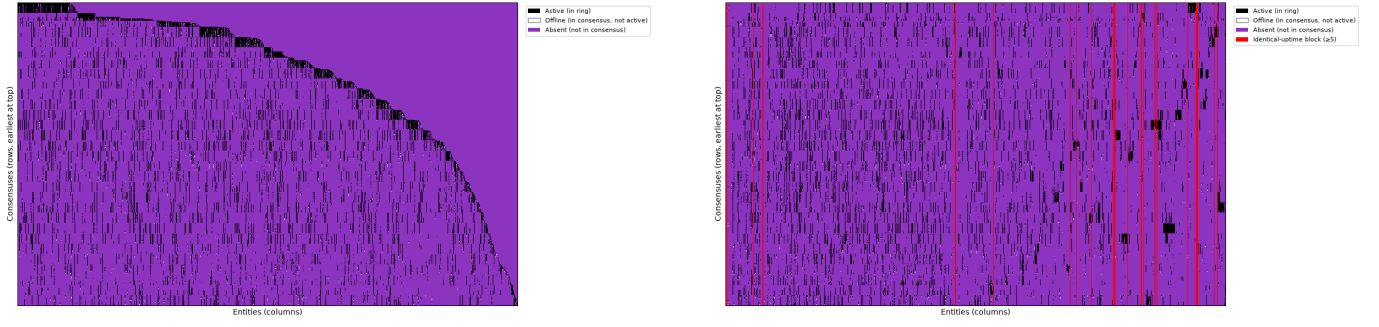


Fig. 18. EV uptime matrix: raw (left) and uptime-similarity clustered (right).

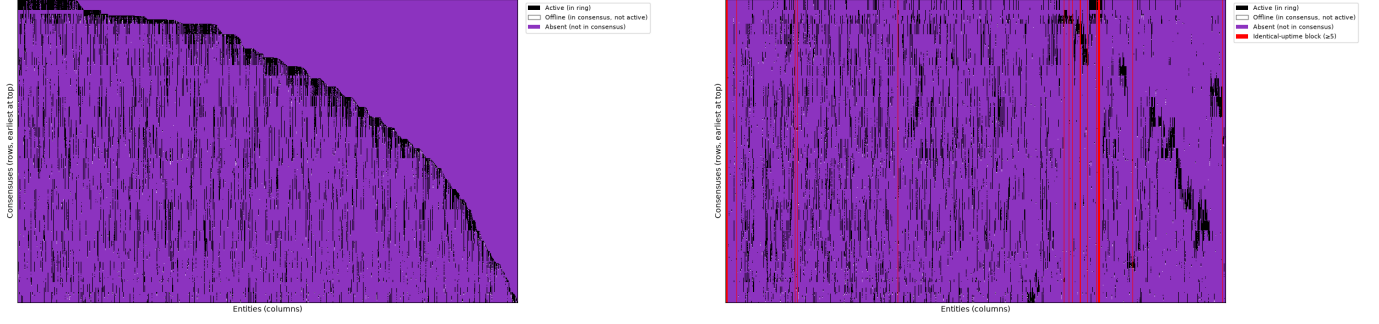


Fig. 19. SC uptime matrix: raw (left) and uptime-similarity clustered (right).

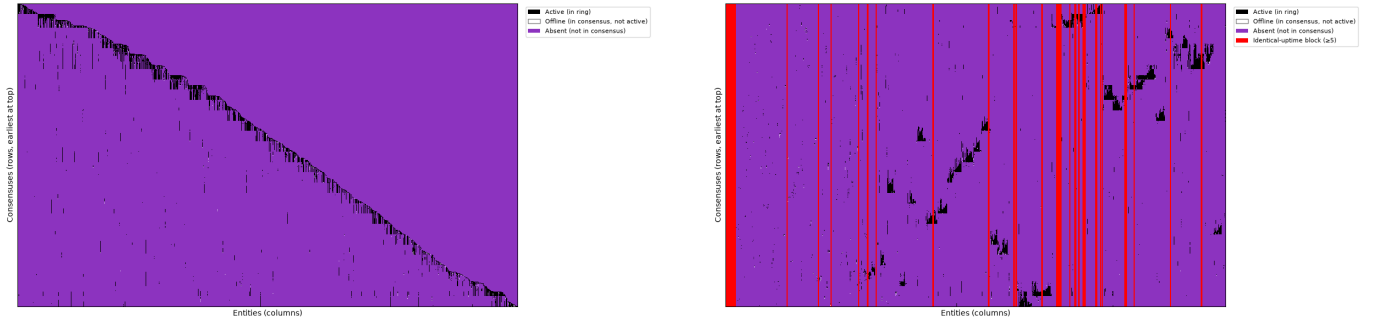


Fig. 20. TP uptime matrix: raw (left) and uptime-similarity clustered (right).

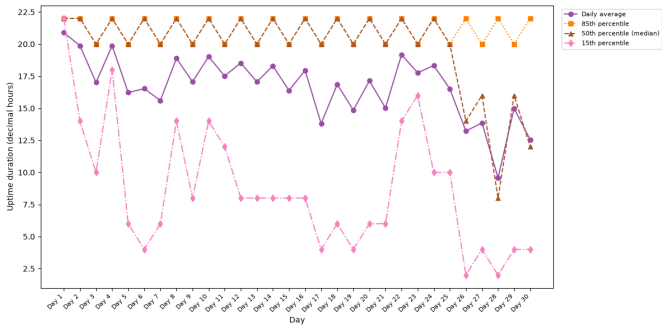


Fig. 25. Network-wide per-day uptime durations.

## B. Operator Identity and Sybil Concentration

| Rank | Family ID       | Nodes | Share (%) |
|------|-----------------|-------|-----------|
| 1    | ri96phwtVtkwug/ | 775   | 9.82      |
| 2    | IgJeqr1GmahHPLh | 320   | 4.05      |
| 3    | 5vHhiPVy3pZwFsR | 257   | 3.26      |
| 4    | cagGVCNHwlHnE50 | 252   | 3.19      |
| 5    | YTCZ7hE06qMkdfr | 123   | 1.56      |

Table X. Top declared-family groups — Network.

| Rank | Family ID       | Nodes | Share (%) |
|------|-----------------|-------|-----------|
| 1    | ri96phwtVtkwug/ | 775   | 10.44     |
| 2    | cagGVCNHwlHnE50 | 242   | 3.26      |
| 3    | 5vHhiPVy3pZwFsR | 240   | 3.23      |
| 4    | IgJeqr1GmahHPLh | 212   | 2.86      |
| 5    | YTCZ7hE06qMkdfr | 123   | 1.66      |

Table XI. Top declared-family groups — Tracked.

| Rank | Family ID       | Nodes | Share (%) |
|------|-----------------|-------|-----------|
| 1    | ri96phwtVtkwug/ | 772   | 11.01     |
| 2    | 5vHhiPVy3pZwFsR | 225   | 3.21      |
| 3    | cagGVCNHwlHnE50 | 221   | 3.15      |
| 4    | IgJeqr1GmahHPLh | 146   | 2.08      |
| 5    | YTCZ7hE06qMkdfr | 122   | 1.74      |

Table XII. Top declared-family groups — EV.

| Rank | Family ID       | Nodes | Share (%) |
|------|-----------------|-------|-----------|
| 1    | ri96phwtVtkwug/ | 760   | 11.16     |
| 2    | 5vHhiPVy3pZwFsR | 216   | 3.17      |
| 3    | cagGVCNHwlHnE50 | 209   | 3.07      |
| 4    | IgJeqr1GmahHPLh | 138   | 2.03      |
| 5    | YTCZ7hE06qMkdfr | 117   | 1.72      |

Table XIII. Top declared-family groups — SC.

| Rank | Family ID       | Nodes | Share (%) |
|------|-----------------|-------|-----------|
| 1    | ri96phwtVtkwug/ | 294   | 13.54     |
| 2    | cagGVCNHwlHnE50 | 48    | 2.21      |
| 3    | 5vHhiPVy3pZwFsR | 44    | 2.03      |
| 4    | i2Dfzk60I2eG99o | 32    | 1.47      |
| 5    | YTCZ7hE06qMkdfr | 29    | 1.34      |

Table XIV. Top declared-family groups — TP.

| Rank | IP address      | Nodes | Share (%) |
|------|-----------------|-------|-----------|
| 1    | 107.208.159.58  | 16    | 0.20      |
| 2    | 185.243.218.231 | 8     | 0.10      |
| 3    | 184.8.85.200    | 8     | 0.10      |
| 4    | 185.243.218.232 | 8     | 0.10      |
| 5    | 185.243.218.233 | 8     | 0.10      |

Table XV. Top exact-IP groups — Network.

| Rank | IP address     | Nodes | Share (%) |
|------|----------------|-------|-----------|
| 1    | 107.208.159.58 | 10    | 0.13      |
| 2    | 171.25.193.39  | 8     | 0.11      |
| 3    | 171.25.193.36  | 8     | 0.11      |
| 4    | 171.25.193.35  | 8     | 0.11      |
| 5    | 82.153.138.57  | 8     | 0.11      |

Table XVI. Top exact-IP groups — Tracked.

| Rank | IP address     | Nodes | Share (%) |
|------|----------------|-------|-----------|
| 1    | 154.53.58.161  | 8     | 0.11      |
| 2    | 171.25.193.39  | 8     | 0.11      |
| 3    | 95.214.53.96   | 8     | 0.11      |
| 4    | 94.142.241.153 | 8     | 0.11      |
| 5    | 171.25.193.40  | 8     | 0.11      |

Table XVII. Top exact-IP groups — EV.

| Rank | IP address     | Nodes | Share (%) |
|------|----------------|-------|-----------|
| 1    | 193.74.17.56   | 8     | 0.12      |
| 2    | 80.67.167.81   | 8     | 0.12      |
| 3    | 87.236.199.223 | 8     | 0.12      |
| 4    | 80.94.92.92    | 8     | 0.12      |
| 5    | 171.25.193.39  | 8     | 0.12      |

Table XVIII. Top exact-IP groups — SC.

| Rank | IP address      | Nodes | Share (%) |
|------|-----------------|-------|-----------|
| 1    | 94.142.241.153  | 6     | 0.28      |
| 2    | 67.163.153.240  | 6     | 0.28      |
| 3    | 116.255.1.163   | 5     | 0.23      |
| 4    | 192.3.140.104   | 5     | 0.23      |
| 5    | 185.243.218.231 | 5     | 0.23      |

Table XIX. Top exact-IP groups — TP.

| Rank | Subnet          | Nodes | Share (%) |
|------|-----------------|-------|-----------|
| 1    | 204.8.96.0/24   | 321   | 4.07      |
| 2    | 64.65.62.0/24   | 244   | 3.09      |
| 3    | 96.9.98.0/24    | 216   | 2.74      |
| 4    | 64.65.1.0/24    | 192   | 2.43      |
| 5    | 192.42.116.0/24 | 154   | 1.95      |

Table XX. Top /24-subnet groups — Network.

| Rank | Subnet          | Nodes | Share (%) |
|------|-----------------|-------|-----------|
| 1    | 64.65.62.0/24   | 244   | 3.29      |
| 2    | 96.9.98.0/24    | 216   | 2.91      |
| 3    | 204.8.96.0/24   | 212   | 2.86      |
| 4    | 64.65.1.0/24    | 192   | 2.59      |
| 5    | 192.42.116.0/24 | 154   | 2.07      |

Table XXI. Top /24-subnet groups — Tracked.

| Rank | Subnet          | Nodes | Share (%) |
|------|-----------------|-------|-----------|
| 1    | 64.65.62.0/24   | 243   | 3.47      |
| 2    | 96.9.98.0/24    | 216   | 3.08      |
| 3    | 64.65.1.0/24    | 190   | 2.71      |
| 4    | 192.42.116.0/24 | 152   | 2.17      |
| 5    | 204.8.96.0/24   | 146   | 2.08      |

Table XXII. Top /24-subnet groups — EV.

| Rank | Subnet          | Nodes | Share (%) |
|------|-----------------|-------|-----------|
| 1    | 64.65.62.0/24   | 241   | 3.54      |
| 2    | 96.9.98.0/24    | 211   | 3.10      |
| 3    | 64.65.1.0/24    | 190   | 2.79      |
| 4    | 192.42.116.0/24 | 152   | 2.23      |
| 5    | 204.8.96.0/24   | 138   | 2.03      |

Table XXIII. Top /24-subnet groups — SC.

| Rank | Subnet          | Nodes | Share (%) |
|------|-----------------|-------|-----------|
| 1    | 64.65.62.0/24   | 98    | 4.51      |
| 2    | 64.65.1.0/24    | 82    | 3.78      |
| 3    | 96.9.98.0/24    | 77    | 3.55      |
| 4    | 192.42.116.0/24 | 46    | 2.12      |
| 5    | 23.129.64.0/24  | 32    | 1.47      |

Table XXIV. Top /24-subnet groups  
— TP.

| Rank | Family ID       | Snapshots |
|------|-----------------|-----------|
| 1    | ri96phwtVtkwug/ | 196       |
| 2    | 5vHhiPVy3pZwFsR | 153       |
| 3    | cagGVCNHw1HnE50 | 118       |
| 4    | F93+I20y7I9Y2jq | 84        |
| 5    | 46J5htZq56gwAdA | 71        |

Table XXV. Family-ID entity drops:  
partial-drop snapshot  
count — Network.

| Rank | Family ID       | Snapshots |
|------|-----------------|-----------|
| 1    | A1c+7nA7nr7C1UT | 12        |
| 2    | 90LM8nvZE0IQle/ | 8         |
| 3    | diHFmyrGgjlFxrP | 8         |
| 4    | LBTaj5ouM26C2Hm | 7         |
| 5    | SzJPStiCcWG4/7S | 7         |

Table XXVI. Family-ID entity drops:  
total-drop snapshot  
count — Network.

| Rank | Family ID       | Snapshots |
|------|-----------------|-----------|
| 1    | ri96phwtVtkwug/ | 260       |
| 2    | 5vHhiPVy3pZwFsR | 186       |
| 3    | cagGVCNHw1HnE50 | 158       |
| 4    | F93+I20y7I9Y2jq | 79        |
| 5    | 10sJh1ANJ1/JkKF | 59        |

Table XXVII. Family-ID entity  
drops: partial-drop  
snapshot count —  
Tracked.

| Rank | Family ID       | Snapshots |
|------|-----------------|-----------|
| 1    | YTCZ7hE06qMkdfr | 19        |
| 2    | DFRImPtN+OgGNWR | 19        |
| 3    | haw9ZBtD2D5f9ed | 17        |
| 4    | i2Dfzk60I2eG99o | 15        |
| 5    | SzJPStiCcWG4/7S | 12        |

Table XXVIII. Family-ID entity  
drops: total-drop  
snapshot count —  
Tracked.

| Rank | Family ID       | Snapshots |
|------|-----------------|-----------|
| 1    | ri96phwtVtkwug/ | 164       |
| 2    | 5vHhiPVy3pZwFsR | 112       |
| 3    | cagGVCNHw1HnE50 | 100       |
| 4    | F93+I20y7I9Y2jq | 51        |
| 5    | IgJeqr1GmahHPLh | 34        |

Table XXIX. Family-ID entity  
drops: partial-drop  
snapshot count — EV.

| Rank | Family ID       | Snapshots |
|------|-----------------|-----------|
| 1    | YTCZ7hE06qMkdfr | 18        |
| 2    | DFRImPtN+OgGNWR | 16        |
| 3    | haw9ZBtD2D5f9ed | 16        |
| 4    | i2Dfzk60I2eG99o | 12        |
| 5    | Or+tKhdy84NCOAz | 11        |

Table XXX. Family-ID entity drops:  
total-drop snapshot  
count — EV.

| Rank | Family ID       | Snapshots |
|------|-----------------|-----------|
| 1    | ri96phwtVtkwug/ | 199       |
| 2    | 5vHhiPVy3pZwFsR | 126       |
| 3    | cagGVCNHw1HnE50 | 92        |
| 4    | F93+I20y7I9Y2jq | 46        |
| 5    | 10sJh1ANJ1/JkKF | 34        |

Table XXXI. Family-ID entity  
drops: partial-drop  
snapshot count — SC.

| Rank | Family ID       | Snapshots |
|------|-----------------|-----------|
| 1    | YTCZ7hE06qMkdfr | 15        |
| 2    | DFRImPtN+OgGNWR | 15        |
| 3    | i2Dfzk60I2eG99o | 14        |
| 4    | KlIcMnrJQEDPgTg | 11        |
| 5    | Or+tKhdy84NCOAz | 11        |

Table XXXII. Family-ID entity  
drops: total-drop  
snapshot count — SC.

| Rank | Family ID       | Snapshots |
|------|-----------------|-----------|
| 1    | ri96phwtVtkwug/ | 36        |
| 2    | cagGVCNHw1HnE50 | 9         |
| 3    | 5vHhiPVy3pZwFsR | 4         |
| 4    | 46J5htZq56gwAdA | 3         |
| 5    | F93+I20y7I9Y2jq | 1         |

Table XXXIII. Family-ID entity  
drops: partial-drop  
snapshot count — TP.

| Rank | Family ID       | Snapshots |
|------|-----------------|-----------|
| 1    | IgJeqr1GmahHPLh | 15        |
| 2    | YTCZ7hE06qMkdfr | 10        |
| 3    | DFRImPtN+OgGNWR | 6         |
| 4    | RkrBlpP1WMUElRB | 4         |
| 5    | /2h1rENAM4TSSv4 | 3         |

Table XXXIV. Family-ID entity drops: total-drop snapshot count — TP.

| Rank | IP address      | Snapshots |
|------|-----------------|-----------|
| 1    | 185.246.188.74  | 23        |
| 2    | 67.163.153.240  | 21        |
| 3    | 193.200.229.34  | 21        |
| 4    | 188.165.136.211 | 21        |
| 5    | 94.131.113.212  | 20        |

Table XXXV. Exact-IP entity drops: partial-drop snapshot count — Network.

| Rank | IP address      | Snapshots |
|------|-----------------|-----------|
| 1    | 94.131.113.212  | 21        |
| 2    | 74.82.47.194    | 19        |
| 3    | 185.220.100.250 | 16        |
| 4    | 5.181.80.181    | 14        |
| 5    | 193.201.126.176 | 14        |

Table XXXVI. Exact-IP entity drops: total-drop snapshot count — Network.

| Rank | IP address     | Snapshots |
|------|----------------|-----------|
| 1    | 144.217.32.158 | 24        |
| 2    | 193.200.229.34 | 22        |
| 3    | 94.142.241.153 | 16        |
| 4    | 51.89.2.63     | 16        |
| 5    | 87.236.199.223 | 16        |

Table XXXVII. Exact-IP entity drops: partial-drop snapshot count — Tracked.

| Rank | IP address     | Snapshots |
|------|----------------|-----------|
| 1    | 94.131.113.212 | 21        |
| 2    | 171.25.193.36  | 18        |
| 3    | 51.38.110.234  | 18        |
| 4    | 93.123.12.112  | 16        |
| 5    | 87.98.242.239  | 16        |

Table XXXVIII. Exact-IP entity drops: total-drop snapshot count — Tracked.

| Rank | IP address     | Snapshots |
|------|----------------|-----------|
| 1    | 171.25.193.131 | 9         |
| 2    | 193.200.229.34 | 9         |
| 3    | 144.217.32.158 | 7         |
| 4    | 171.25.193.39  | 7         |
| 5    | 185.246.188.74 | 6         |

Table XXXIX. Exact-IP entity drops: partial-drop snapshot count — EV.

| Rank | IP address      | Snapshots |
|------|-----------------|-----------|
| 1    | 87.98.242.239   | 16        |
| 2    | 171.25.193.36   | 16        |
| 3    | 93.123.12.112   | 13        |
| 4    | 172.232.188.206 | 12        |
| 5    | 51.38.110.234   | 11        |

Table XL. Exact-IP entity drops: total-drop snapshot count — EV.

| Rank | IP address      | Snapshots |
|------|-----------------|-----------|
| 1    | 203.55.81.2     | 9         |
| 2    | 96.44.154.224   | 8         |
| 3    | 192.227.183.149 | 8         |
| 4    | 193.200.229.34  | 8         |
| 5    | 137.226.34.45   | 7         |

Table XLI. Exact-IP entity drops: partial-drop snapshot count — SC.

| Rank | IP address     | Snapshots |
|------|----------------|-----------|
| 1    | 94.142.241.153 | 19        |
| 2    | 51.89.2.63     | 17        |
| 3    | 213.32.104.213 | 16        |
| 4    | 171.25.193.39  | 14        |
| 5    | 96.9.98.174    | 14        |

Table XLII. Exact-IP entity drops: total-drop snapshot count — SC.

| Rank | IP address     | Snapshots |
|------|----------------|-----------|
| 1    | 147.90.235.20  | 1         |
| 2    | 185.246.188.74 | 1         |

Table XLIII. Exact-IP entity drops: partial-drop snapshot count — TP.

| Rank | IP address    | Snapshots |
|------|---------------|-----------|
| 1    | 23.191.200.72 | 6         |
| 2    | 51.77.85.63   | 5         |
| 3    | 23.191.200.31 | 5         |
| 4    | 45.147.7.156  | 5         |
| 5    | 171.25.193.36 | 4         |

Table XLIV. Exact-IP entity drops: total-drop snapshot count — TP.

| Rank | Subnet          | Snapshots |
|------|-----------------|-----------|
| 1    | 96.9.98.0/24    | 142       |
| 2    | 64.65.62.0/24   | 126       |
| 3    | 64.65.1.0/24    | 107       |
| 4    | 140.78.100.0/24 | 106       |
| 5    | 192.42.116.0/24 | 104       |

Table XLV. /24-subnet entity drops:  
partial-drop snapshot  
count — Network.

| Rank | Subnet           | Snapshots |
|------|------------------|-----------|
| 1    | 94.131.113.0/24  | 21        |
| 2    | 74.82.47.0/24    | 19        |
| 3    | 193.201.126.0/24 | 14        |
| 4    | 50.233.233.0/24  | 13        |
| 5    | 94.23.149.0/24   | 12        |

Table XLVI. /24-subnet entity  
drops: total-drop  
snapshot count —  
Network.

| Rank | Subnet          | Snapshots |
|------|-----------------|-----------|
| 1    | 96.9.98.0/24    | 140       |
| 2    | 64.65.62.0/24   | 132       |
| 3    | 64.65.1.0/24    | 124       |
| 4    | 192.42.116.0/24 | 106       |
| 5    | 140.78.100.0/24 | 102       |

Table XLVII. /24-subnet entity  
drops: partial-drop  
snapshot count —  
Tracked.

| Rank | Subnet          | Snapshots |
|------|-----------------|-----------|
| 1    | 94.131.113.0/24 | 21        |
| 2    | 171.25.193.0/24 | 19        |
| 3    | 23.191.200.0/24 | 19        |
| 4    | 51.38.110.0/24  | 18        |
| 5    | 93.123.12.0/24  | 16        |

Table XLVIII. /24-subnet entity  
drops: total-drop  
snapshot count —  
Tracked.

| Rank | Subnet          | Snapshots |
|------|-----------------|-----------|
| 1    | 64.65.62.0/24   | 81        |
| 2    | 96.9.98.0/24    | 79        |
| 3    | 64.65.1.0/24    | 76        |
| 4    | 192.42.116.0/24 | 60        |
| 5    | 140.78.100.0/24 | 51        |

Table XLIX. /24-subnet entity  
drops: partial-drop  
snapshot count — EV.

| Rank | Subnet           | Snapshots |
|------|------------------|-----------|
| 1    | 23.191.200.0/24  | 18        |
| 2    | 171.25.193.0/24  | 16        |
| 3    | 87.98.242.0/24   | 16        |
| 4    | 93.123.12.0/24   | 13        |
| 5    | 190.211.254.0/24 | 13        |

Table L. /24-subnet entity drops:  
total-drop snapshot count  
— EV.

| Rank | Subnet          | Snapshots |
|------|-----------------|-----------|
| 1    | 96.9.98.0/24    | 103       |
| 2    | 64.65.62.0/24   | 94        |
| 3    | 64.65.1.0/24    | 81        |
| 4    | 192.42.116.0/24 | 71        |
| 5    | 140.78.100.0/24 | 57        |

Table LI. /24-subnet entity drops:  
partial-drop snapshot count  
— SC.

| Rank | Subnet          | Snapshots |
|------|-----------------|-----------|
| 1    | 94.142.241.0/24 | 19        |
| 2    | 51.89.2.0/24    | 17        |
| 3    | 213.32.104.0/24 | 16        |
| 4    | 171.25.193.0/24 | 15        |
| 5    | 23.191.200.0/24 | 15        |

Table LII. /24-subnet entity drops:  
total-drop snapshot count  
— SC.

| Rank | Subnet           | Snapshots |
|------|------------------|-----------|
| 1    | 64.65.62.0/24    | 11        |
| 2    | 96.9.98.0/24     | 10        |
| 3    | 64.65.1.0/24     | 6         |
| 4    | 192.42.116.0/24  | 5         |
| 5    | 185.220.101.0/24 | 1         |

Table LIII. /24-subnet entity drops:  
partial-drop snapshot  
count — TP.

| Rank | Subnet          | Snapshots |
|------|-----------------|-----------|
| 1    | 204.8.96.0/24   | 15        |
| 2    | 140.78.100.0/24 | 12        |
| 3    | 23.191.200.0/24 | 10        |
| 4    | 171.25.193.0/24 | 6         |
| 5    | 51.77.85.0/24   | 5         |

Table LIV. /24-subnet entity drops:  
total-drop snapshot count  
— TP.

| Rank | Nickname     | Distinct fps |
|------|--------------|--------------|
| 1    | Quetzalcoat1 | 257          |
| 2    | prsv         | 252          |
| 3    | theTorDoor   | 100          |
| 4    | zgato        | 96           |
| 5    | bauruine     | 60           |

Table LV. Nickname fingerprint  
mutation — Network.

| Rank | Nickname     | Distinct fps |
|------|--------------|--------------|
| 1    | prsv         | 241          |
| 2    | Quetzalcoat1 | 240          |
| 3    | theTorDoor   | 99           |
| 4    | zgato        | 85           |
| 5    | bauruine     | 60           |

Table LVI. Nickname fingerprint mutation — Tracked.

| Rank | Nickname     | Distinct fps |
|------|--------------|--------------|
| 1    | Quetzalcoat1 | 225          |
| 2    | prsv         | 221          |
| 3    | theTorDoor   | 97           |
| 4    | zgato        | 77           |
| 5    | bauruine     | 58           |

Table LVII. Nickname fingerprint mutation — EV.

| Rank | Nickname     | Distinct fps |
|------|--------------|--------------|
| 1    | Quetzalcoat1 | 215          |
| 2    | prsv         | 208          |
| 3    | theTorDoor   | 93           |
| 4    | zgato        | 75           |
| 5    | bauruine     | 59           |

Table LVIII. Nickname fingerprint mutation — SC.

| Rank | Nickname     | Distinct fps |
|------|--------------|--------------|
| 1    | prsv         | 47           |
| 2    | Quetzalcoat1 | 44           |
| 3    | theTorDoor   | 26           |
| 4    | bauruine     | 22           |
| 5    | zgato        | 21           |

Table LIX. Nickname fingerprint mutation — TP.

| Rank | Nickname      | Distinct IPs |
|------|---------------|--------------|
| 1    | theTorDoor    | 100          |
| 2    | Quetzalcoat1  | 68           |
| 3    | zgato         | 59           |
| 4    | prsv          | 56           |
| 5    | ForPrivacyNET | 24           |

Table LX. Nickname IP-address mutation — Network.

| Rank | Nickname      | Distinct IPs |
|------|---------------|--------------|
| 1    | theTorDoor    | 99           |
| 2    | Quetzalcoat1  | 68           |
| 3    | prsv          | 56           |
| 4    | zgato         | 55           |
| 5    | ForPrivacyNET | 24           |

Table LXI. Nickname IP-address mutation — Tracked.

| Rank | Nickname      | Distinct IPs |
|------|---------------|--------------|
| 1    | theTorDoor    | 97           |
| 2    | Quetzalcoat1  | 67           |
| 3    | prsv          | 56           |
| 4    | zgato         | 52           |
| 5    | ForPrivacyNET | 24           |

Table LXII. Nickname IP-address mutation — EV.

| Rank | Nickname      | Distinct IPs |
|------|---------------|--------------|
| 1    | theTorDoor    | 93           |
| 2    | Quetzalcoat1  | 67           |
| 3    | prsv          | 54           |
| 4    | zgato         | 51           |
| 5    | ForPrivacyNET | 24           |

Table LXIII. Nickname IP-address mutation — SC.

| Rank | Nickname      | Distinct IPs |
|------|---------------|--------------|
| 1    | Quetzalcoat1  | 30           |
| 2    | prsv          | 27           |
| 3    | theTorDoor    | 26           |
| 4    | zgato         | 19           |
| 5    | ForPrivacyNET | 12           |

Table LXIV. Nickname IP-address mutation — TP.

| Rank | Nickname     | Distinct /24s |
|------|--------------|---------------|
| 1    | zgato        | 59            |
| 2    | Quetzalcoat1 | 57            |
| 3    | prsv         | 41            |
| 4    | Killix       | 19            |
| 5    | RelayUli     | 13            |

Table LXV. Nickname subnet mutation — Network.

| Rank | Nickname      | Distinct /24s |
|------|---------------|---------------|
| 1    | Quetzalcoat1  | 57            |
| 2    | zgato         | 55            |
| 3    | prsv          | 41            |
| 4    | Killix        | 19            |
| 5    | opencommunity | 13            |

Table LXVI. Nickname subnet mutation — Tracked.

| Rank | Nickname      | Distinct /24s |
|------|---------------|---------------|
| 1    | Quetzalcoat1  | 57            |
| 2    | zgato         | 52            |
| 3    | prsv          | 41            |
| 4    | Killix        | 18            |
| 5    | opencommunity | 13            |

Table LXVII. Nickname subnet mutation — EV.

| Rank | Nickname      | Distinct /24s |
|------|---------------|---------------|
| 1    | Quetzalcoat1  | 57            |
| 2    | zgato         | 51            |
| 3    | prsv          | 39            |
| 4    | Killix        | 16            |
| 5    | opencommunity | 12            |

Table LXVIII. Nickname subnet mutation — SC.

| Rank | Nickname      | Distinct /24s |
|------|---------------|---------------|
| 1    | Quetzalcoat1  | 29            |
| 2    | prsv          | 27            |
| 3    | zgato         | 19            |
| 4    | bauruine      | 8             |
| 5    | opencommunity | 8             |

Table LXIX. Nickname subnet mutation — TP.

| Population | Mixed | Only-with-ID | Only-without-ID |
|------------|-------|--------------|-----------------|
| Network    | 13.33 | 1.50         | 811.57          |
| Tracked    | 5.50  | 0.47         | 418.13          |
| EV         | 2.43  | 0.13         | 187.77          |
| SC         | 2.67  | 0.30         | 209.70          |
| TP         | 0.40  | 0.03         | 20.67           |

Table LXX. Undeclared-family per-day averages, no-ID-focused (nodes/day).

| Population | Mixed | Only-with-ID | Only-without-ID |
|------------|-------|--------------|-----------------|
| Network    | 0.00  | 0.13         | 8.47            |
| Tracked    | 0.00  | 0.00         | 2.30            |
| EV         | 0.00  | 0.00         | 1.10            |
| SC         | 0.00  | 0.00         | 1.13            |
| TP         | 0.00  | 0.00         | 0.07            |

Table LXXI. Undeclared-family per-day averages, with-ID-focused (nodes/day).

| Population | Mixed | Only-with-ID | Only-without-ID |
|------------|-------|--------------|-----------------|
| Network    | 197   | 43           | 1745            |
| Tracked    | 96    | 13           | 3542            |
| EV         | 41    | 5            | 1547            |
| SC         | 43    | 7            | 1512            |
| TP         | 12    | 1            | 483             |

Table LXXII. Undeclared-family month-wide totals, no-ID-focused (all consensus).

| Population | Mixed | Only-with-ID | Only-without-ID |
|------------|-------|--------------|-----------------|
| Network    | 0     | 0            | 2000            |
| Tracked    | 0     | 5            | 170             |
| EV         | 0     | 2            | 71              |
| SC         | 0     | 2            | 74              |
| TP         | 0     | 1            | 25              |

Table LXXIII. Undeclared-family month-wide totals, with-ID-focused (all consensus).

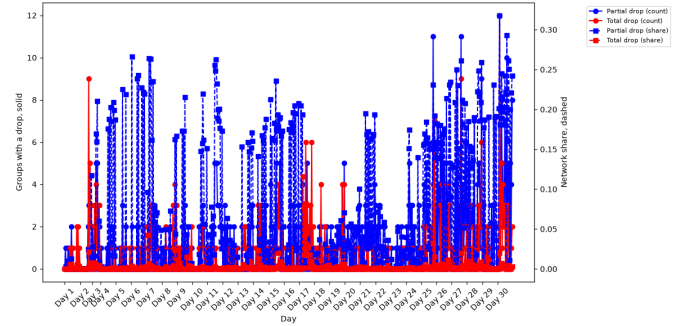


Fig. 26. Family-ID entity drops — EV.

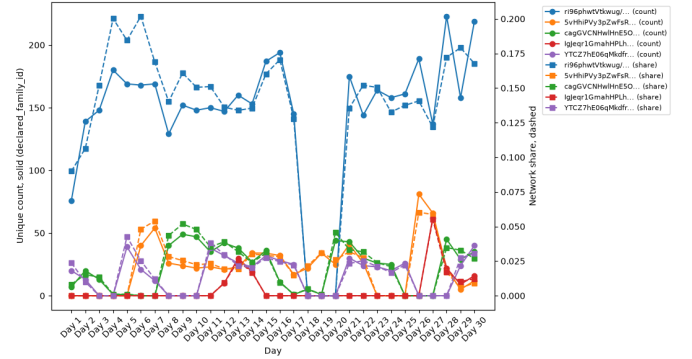


Fig. 27. Declared-family concentration — EV.

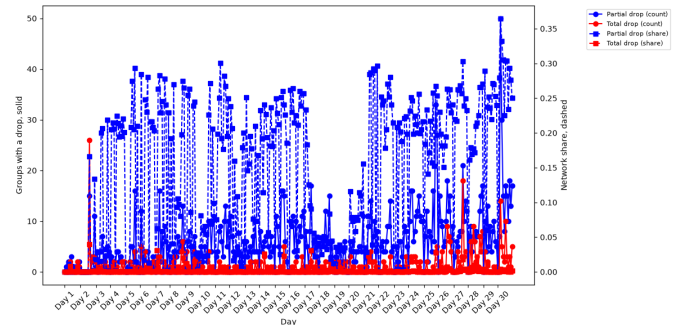


Fig. 28. Family-ID entity drops — network-wide.

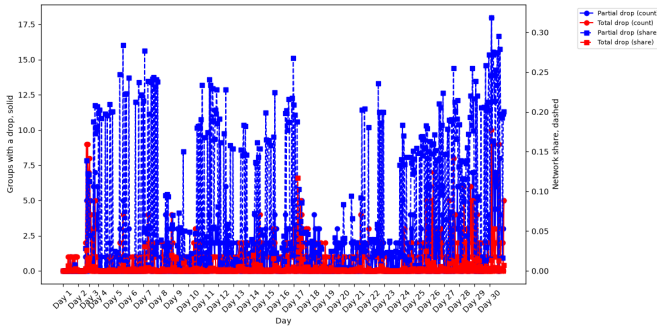


Fig. 29. Family-ID entity drops — SC.

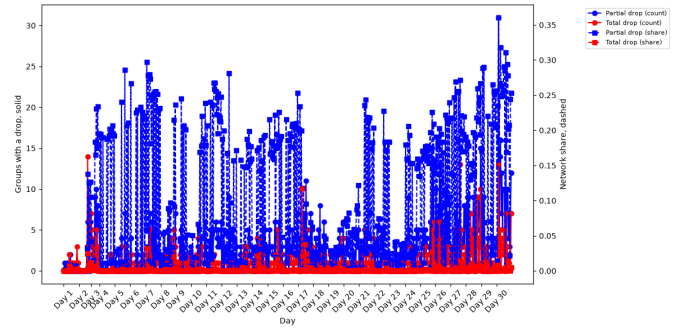


Fig. 33. Family-ID entity drops — tracked.

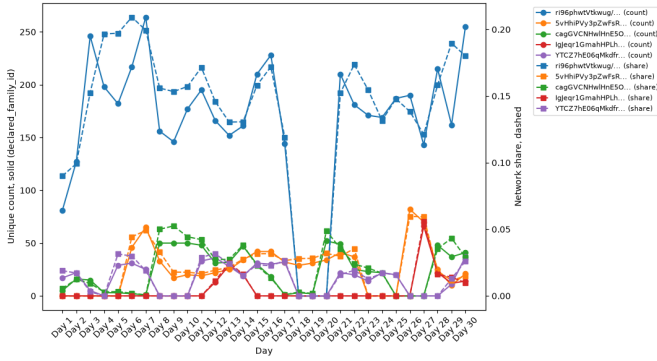


Fig. 30. Declared-family concentration — SC.

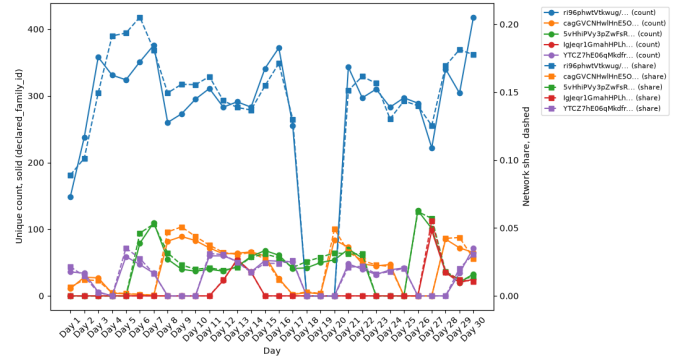


Fig. 34. Declared-family concentration — tracked.

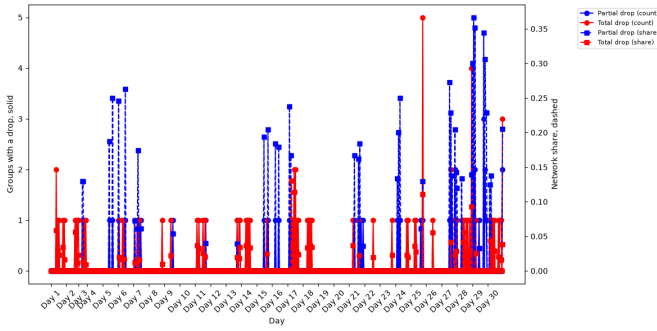


Fig. 31. Family-ID entity drops — TP.

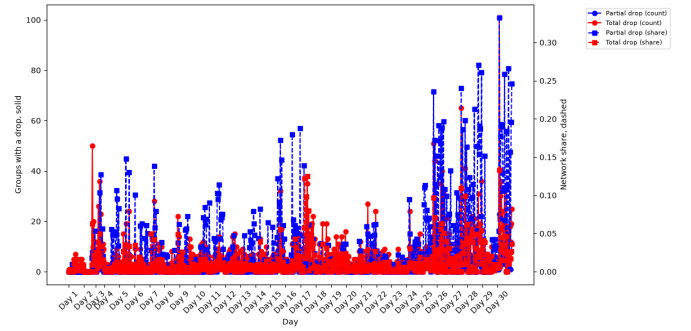


Fig. 35. /24-subnet entity drops — EV.

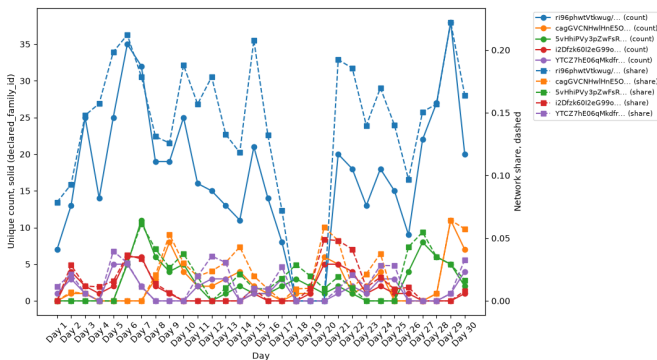


Fig. 32. Declared-family concentration — TP.

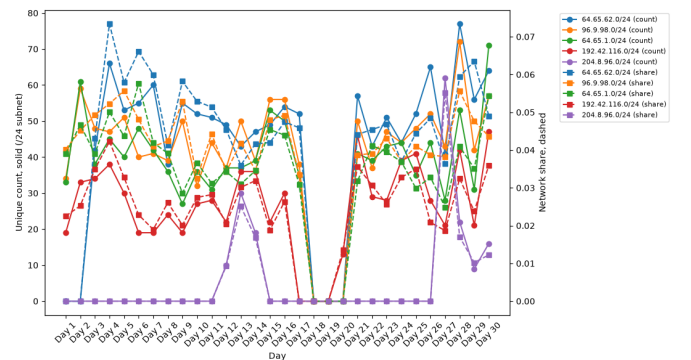


Fig. 36. /24-subnet concentration — EV.

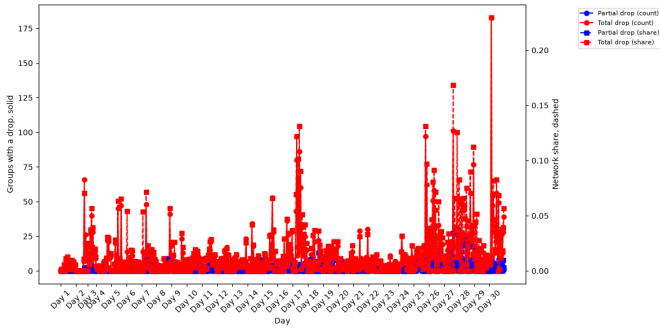


Fig. 37. Exact-IP entity drops — EV.

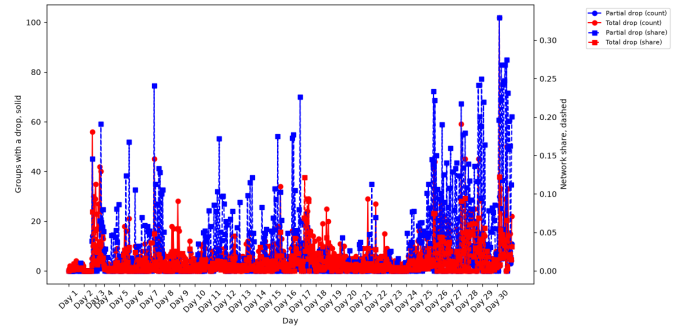


Fig. 41. /24-subnet entity drops — SC.

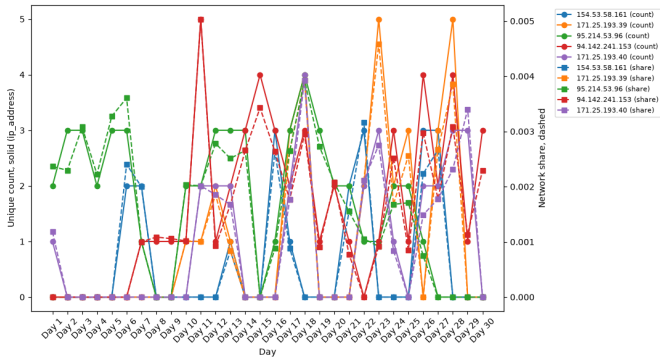


Fig. 38. Exact-IP concentration — EV.

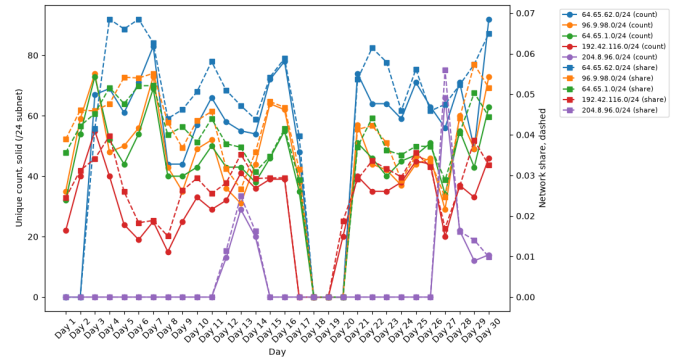


Fig. 42. /24-subnet concentration — SC.

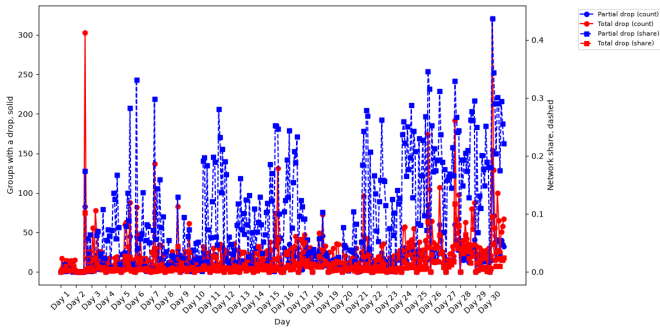


Fig. 39. /24-subnet entity drops — network-wide.

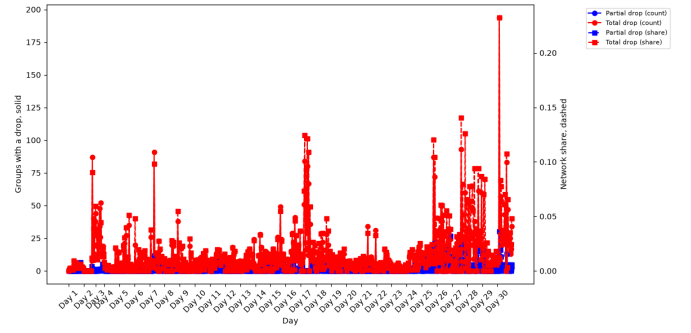


Fig. 43. Exact-IP entity drops — SC.

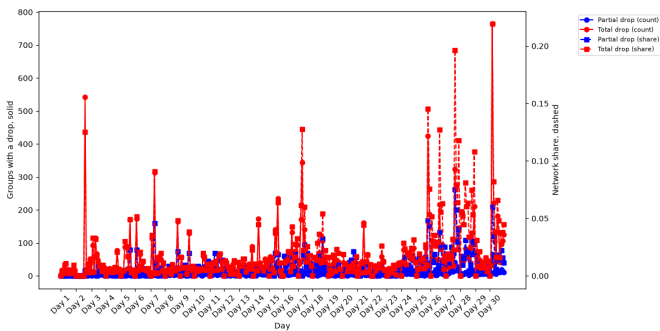


Fig. 40. Exact-IP entity drops — network-wide.

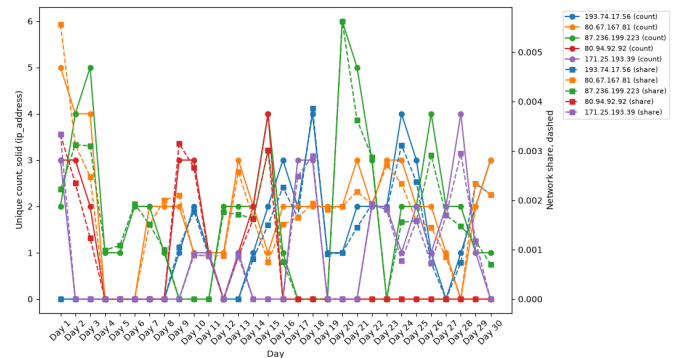


Fig. 44. Exact-IP concentration — SC.

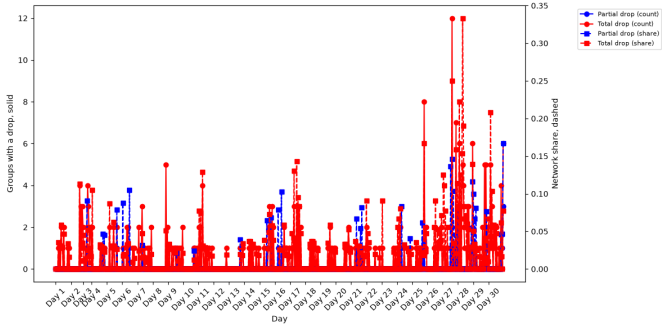


Fig. 45. /24-subnet entity drops — TP.

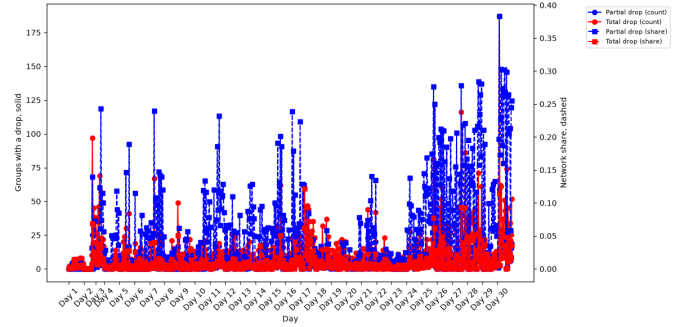


Fig. 49. /24-subnet entity drops — tracked.

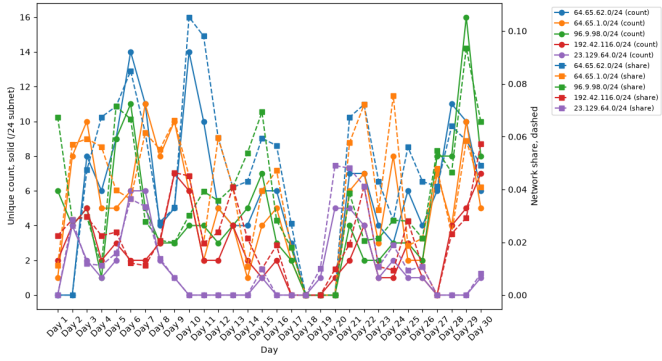


Fig. 46. /24-subnet concentration — TP.

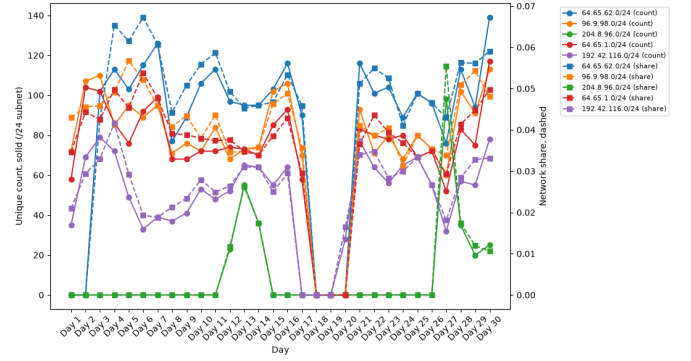


Fig. 50. /24-subnet concentration — tracked.

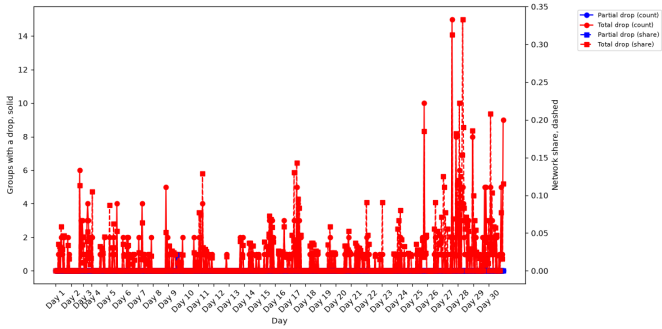


Fig. 47. Exact-IP entity drops — TP.

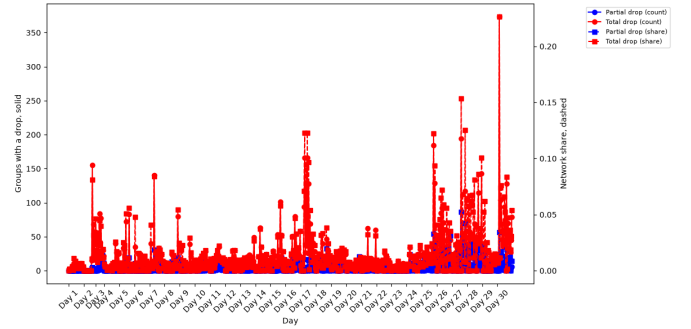


Fig. 51. Exact-IP entity drops — tracked.

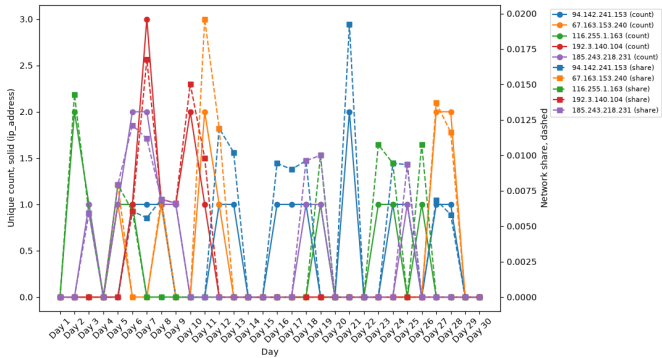


Fig. 48. Exact-IP concentration — TP.

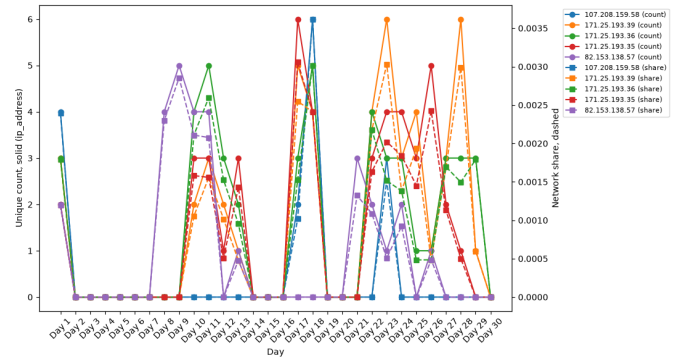


Fig. 52. Exact-IP concentration — tracked.

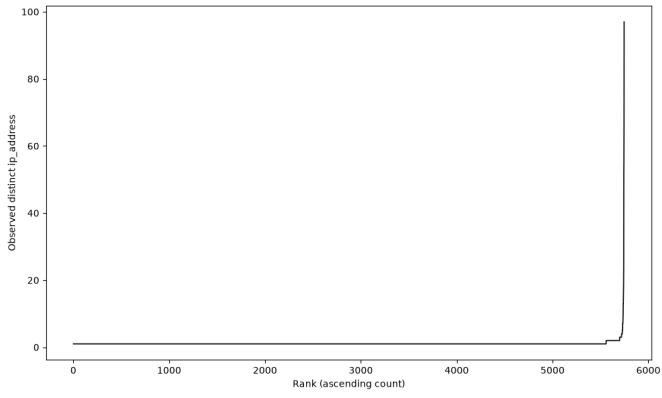


Fig. 53. Nickname IP-address mutation — EV.

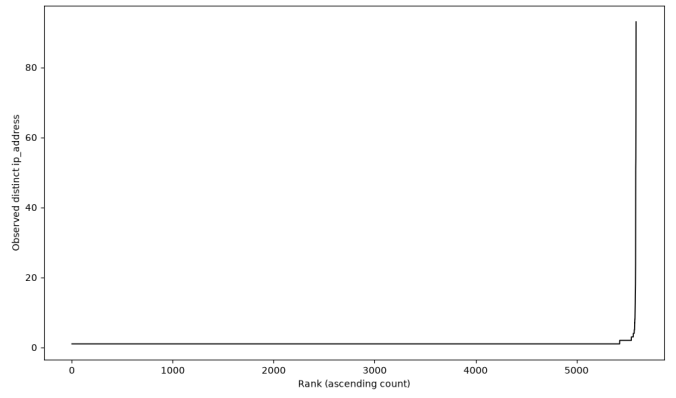


Fig. 57. Nickname IP-address mutation — SC.

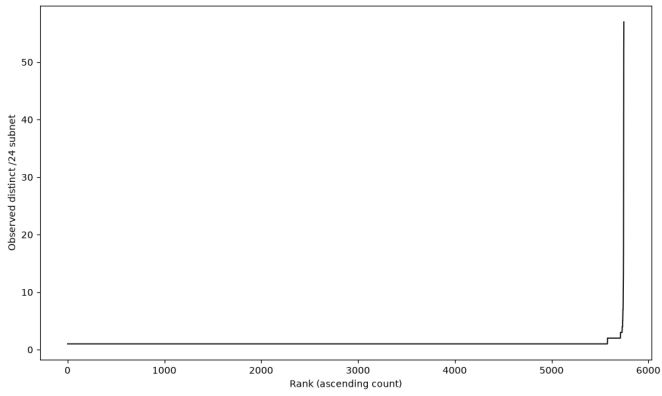


Fig. 54. Nickname /24-subnet mutation — EV.

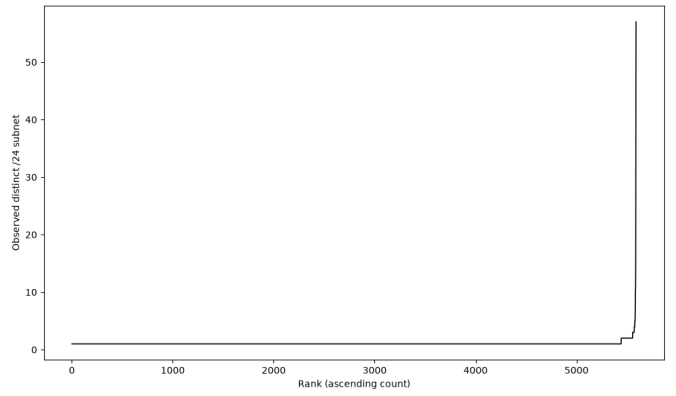


Fig. 58. Nickname /24-subnet mutation — SC.

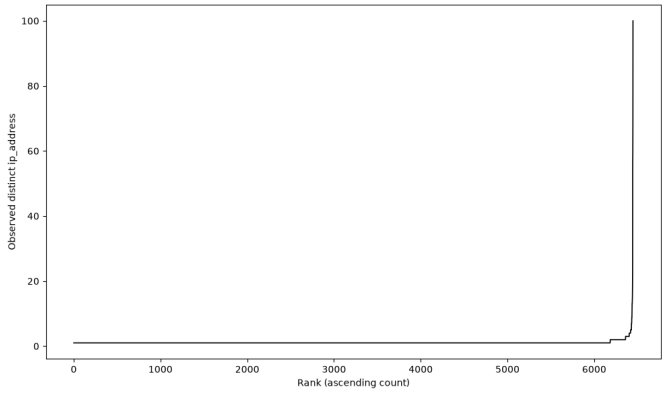


Fig. 55. Nickname IP-address mutation — network-wide.

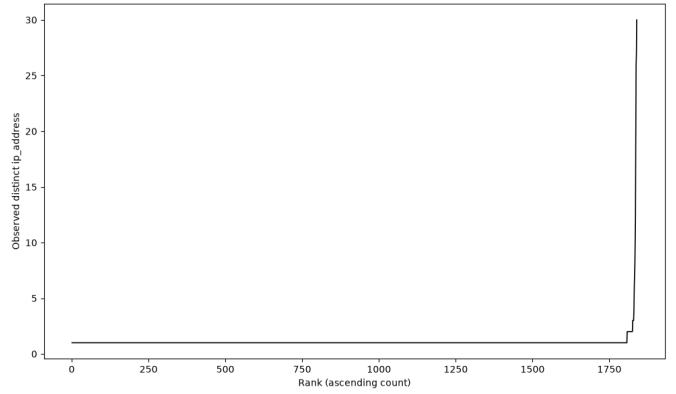


Fig. 59. Nickname IP-address mutation — TP.

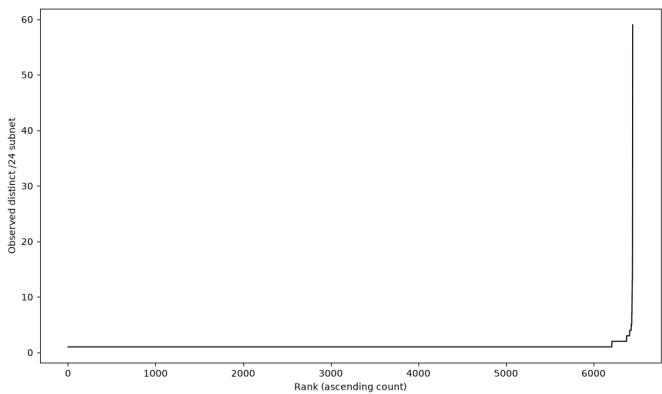


Fig. 56. Nickname /24-subnet mutation — network-wide.

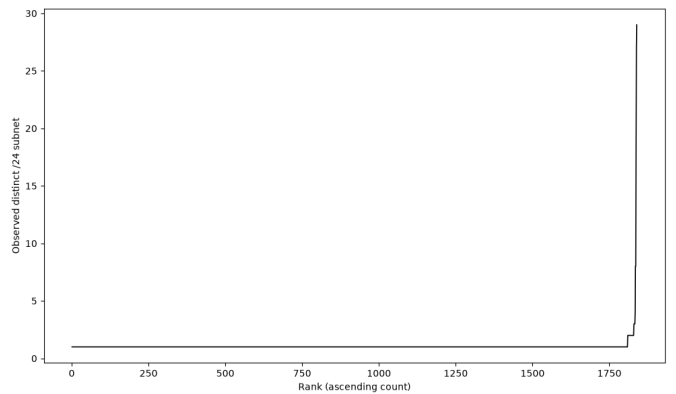


Fig. 60. Nickname /24-subnet mutation — TP.

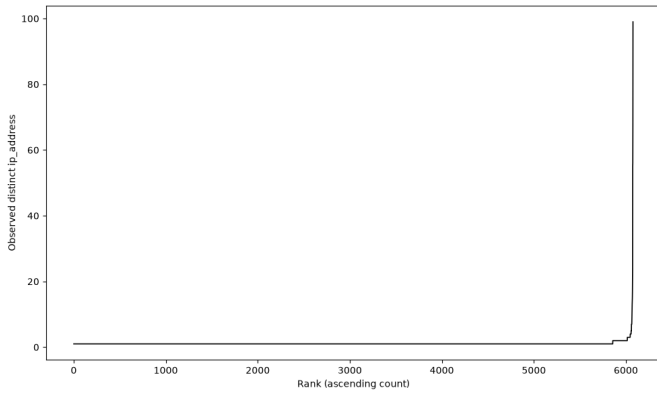


Fig. 61. Nickname IP-address mutation — tracked.

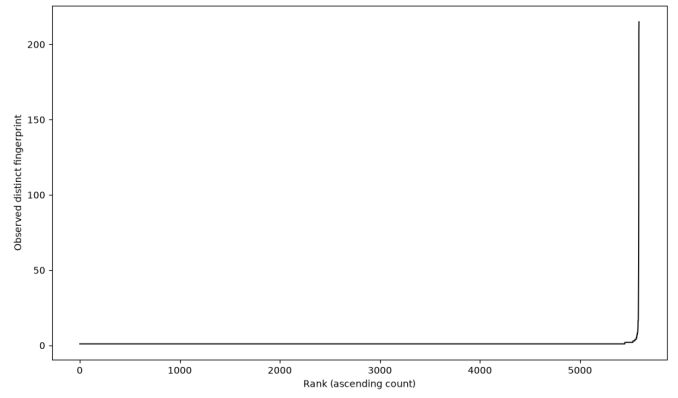


Fig. 65. Nickname fingerprint mutation — SC.

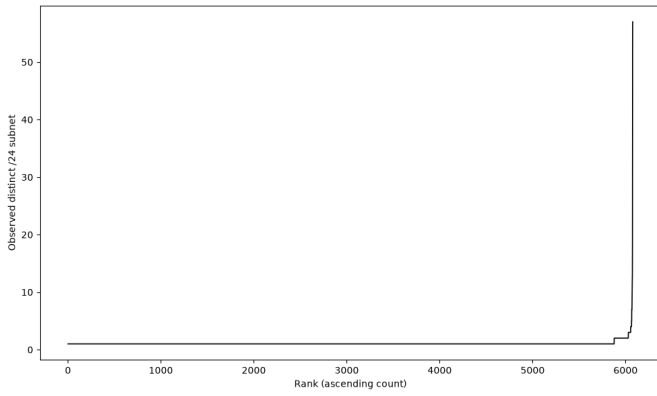


Fig. 62. Nickname /24-subnet mutation — tracked.

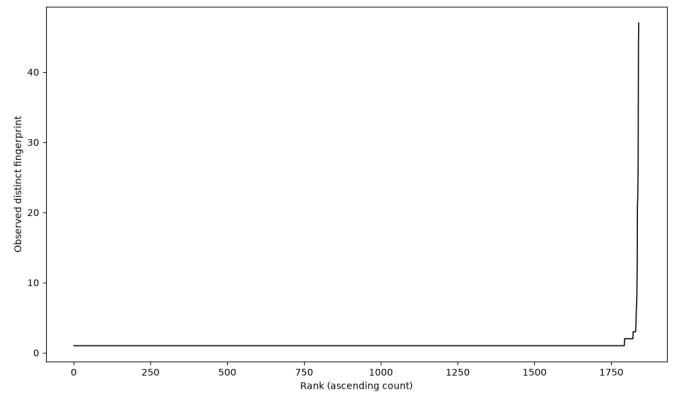


Fig. 66. Nickname fingerprint mutation — TP.

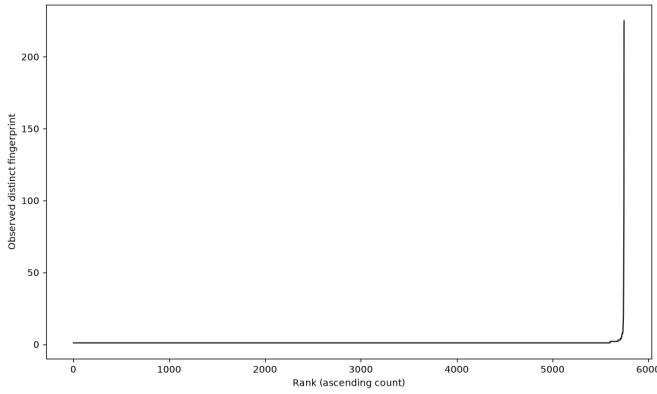


Fig. 63. Nickname fingerprint mutation — EV.

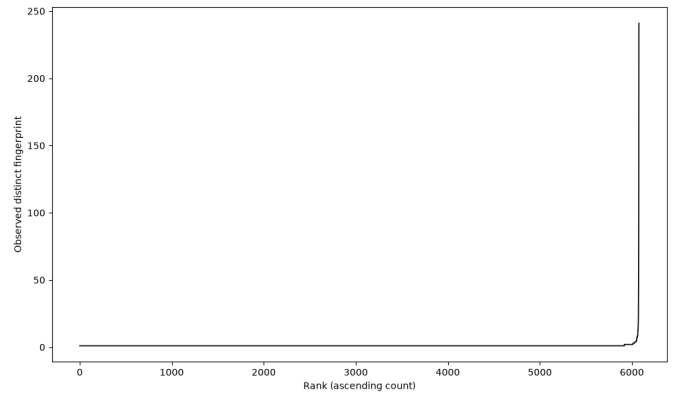


Fig. 67. Nickname fingerprint mutation — tracked.

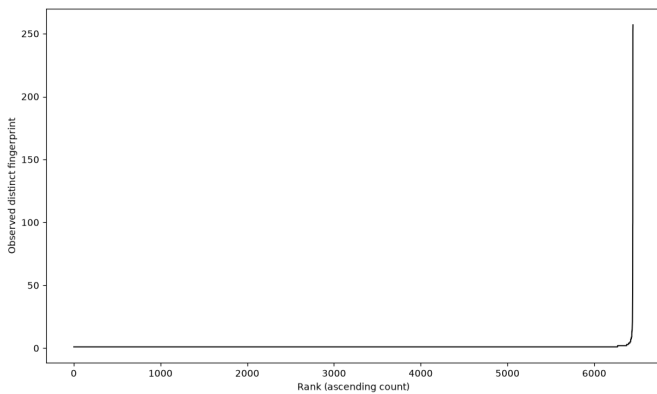


Fig. 64. Nickname fingerprint mutation — network-wide.

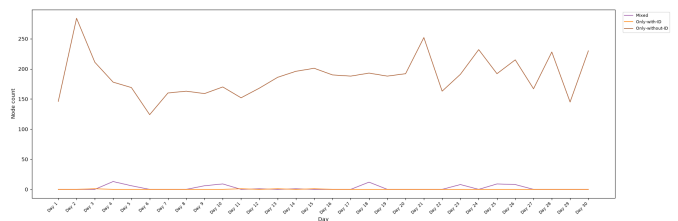


Fig. 68. ephemeral variable undeclared families no ID.

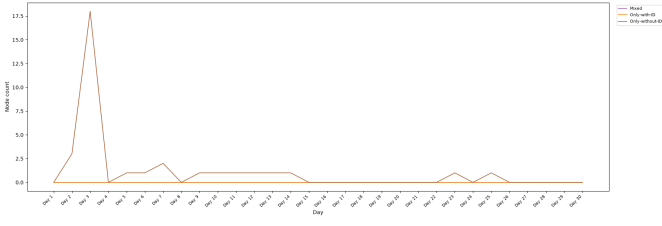


Fig. 69. ephemeral variable undeclared families with ID.

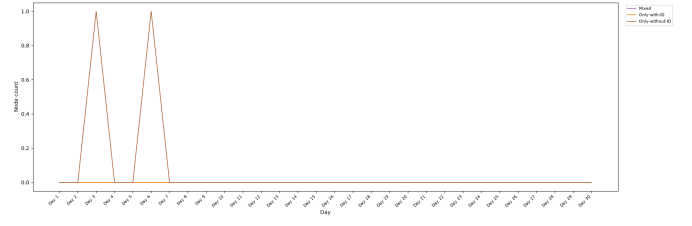


Fig. 75. third party probe undeclared families with ID.

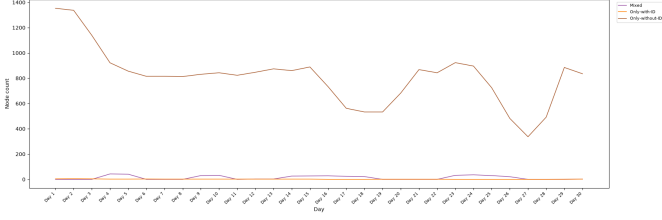


Fig. 70. network undeclared families no ID.

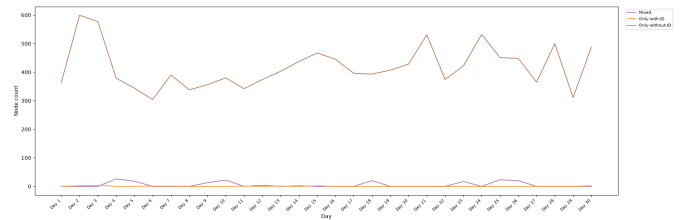


Fig. 76. tracked undeclared families no ID.

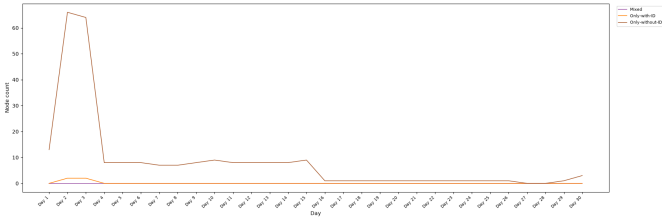


Fig. 71. network undeclared families with ID.

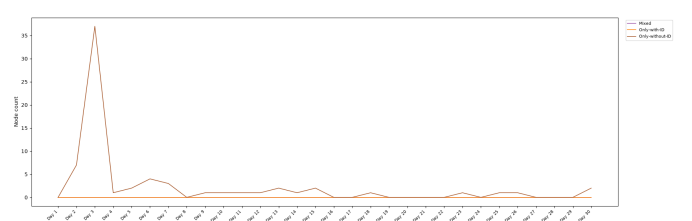


Fig. 77. tracked undeclared families with ID.

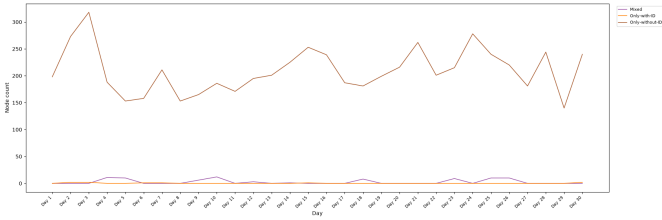


Fig. 72. static control undeclared families no ID.

### C. Hash-Ring Position

|            | HSDir index |       | Onion index |       |
|------------|-------------|-------|-------------|-------|
| Population | Pooled      | Daily | Pooled      | Daily |
| Network    | 100.00      | 36.82 | —           | —     |
| Tracked    | 99.97       | 29.87 | 42.19       | 2.60  |
| EV         | 93.42       | 9.27  | 18.66       | 0.69  |
| SC         | 98.77       | 18.04 | 18.98       | 1.37  |
| TP         | 28.98       | 1.45  | 4.20        | 0.26  |

Table LXXIV. Ring-coverage percentages at 10,000-bucket resolution.

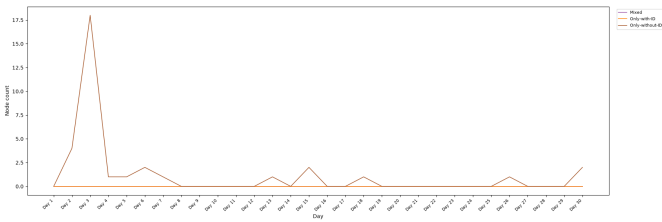


Fig. 73. static control undeclared families with ID.

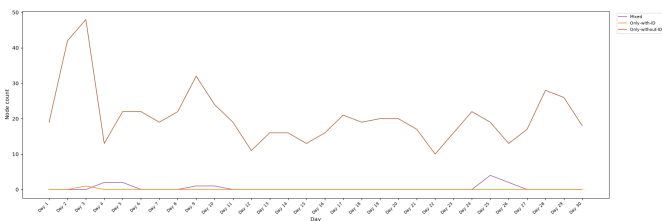


Fig. 74. third party probe undeclared families no ID.

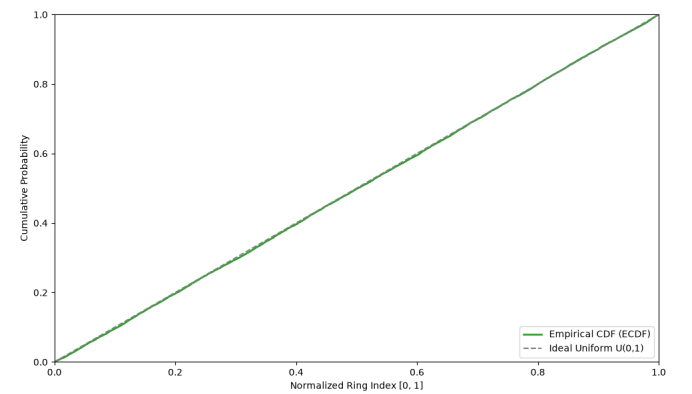


Fig. 78. HSDir index: uniformity ECDF — tracked.

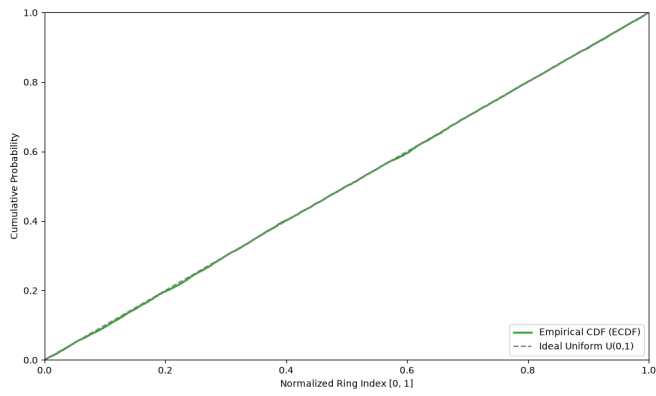


Fig. 79. HSDir index: uniformity ECDF — EV.

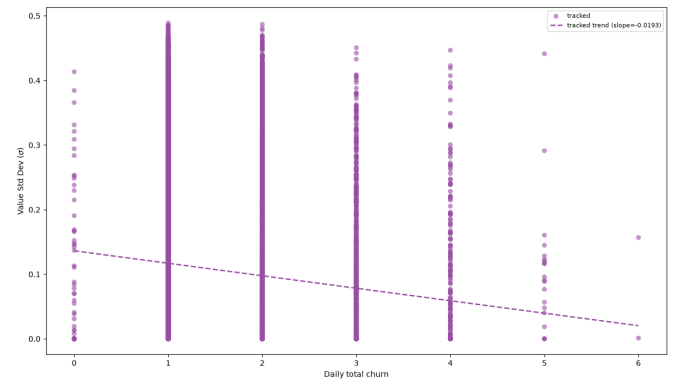


Fig. 82. HSDir index: churn vs. dispersion — tracked.

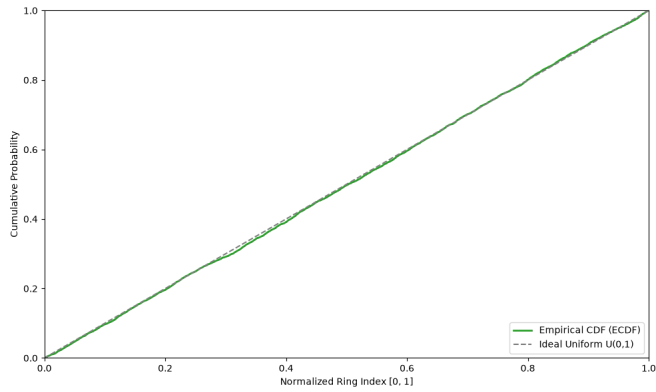


Fig. 80. HSDir index: uniformity ECDF — SC.

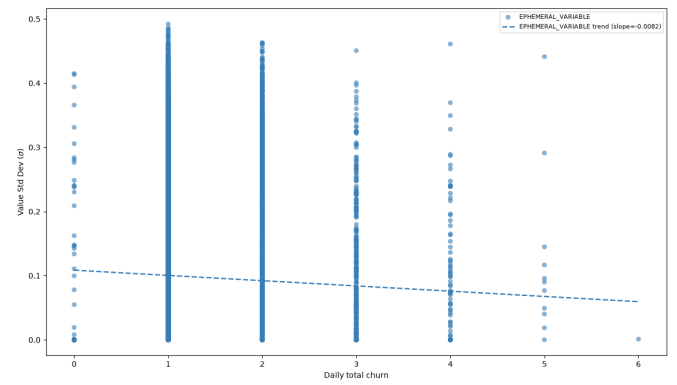


Fig. 83. HSDir index: churn vs. dispersion — EV.

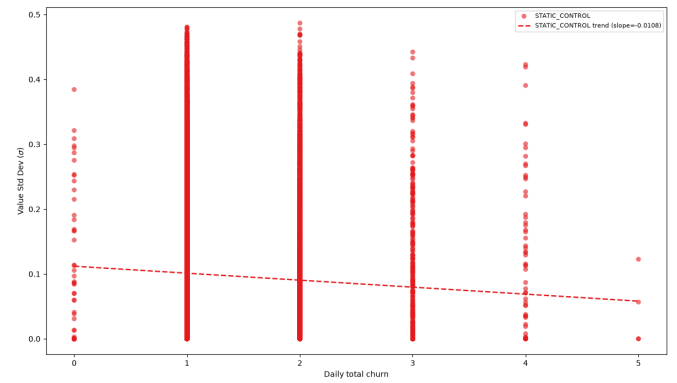


Fig. 84. HSDir index: churn vs. dispersion — SC.

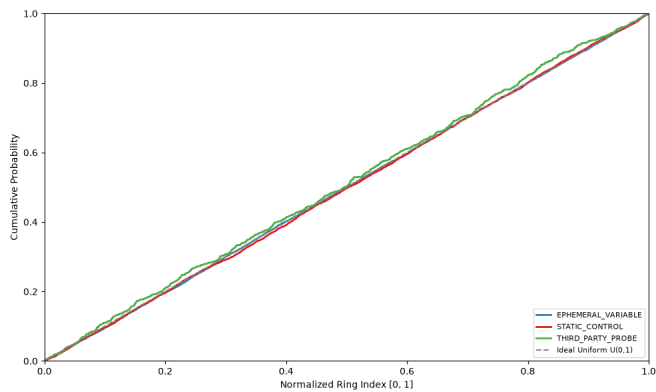


Fig. 81. HSDir index: uniformity ECDF — service-type comparison.

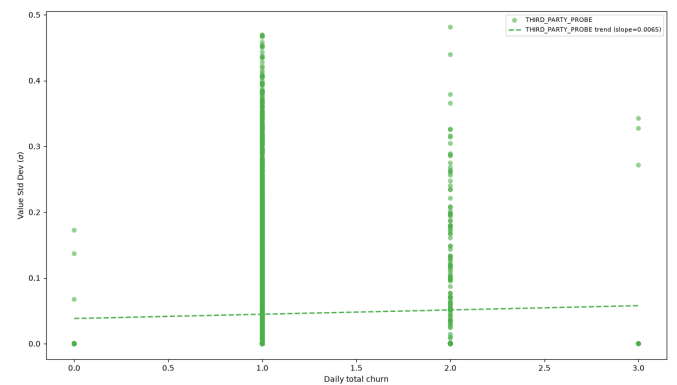


Fig. 85. HSDir index: churn vs. dispersion — TP.

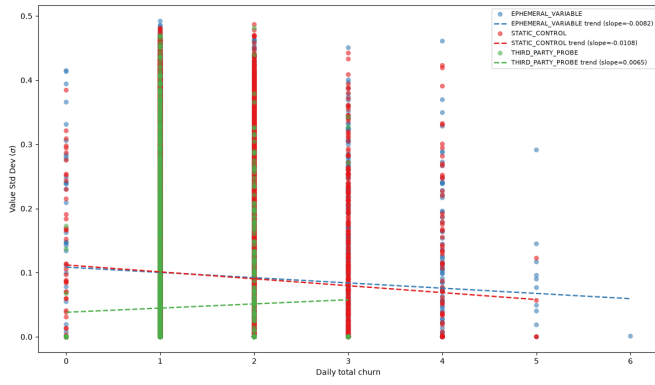


Fig. 86. HSDir index: churn vs. dispersion — service types.

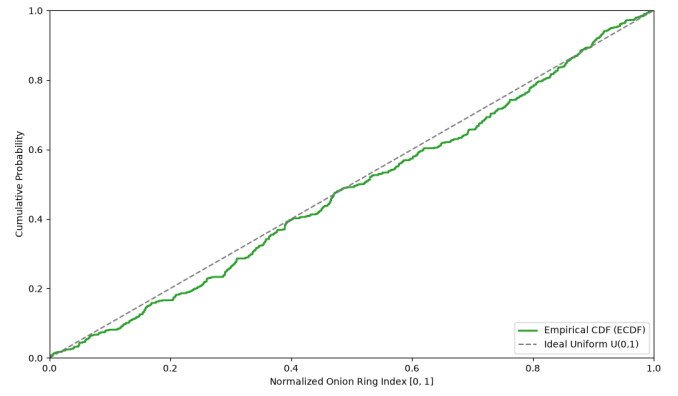


Fig. 90. Onion index: uniformity ECDF — TP.

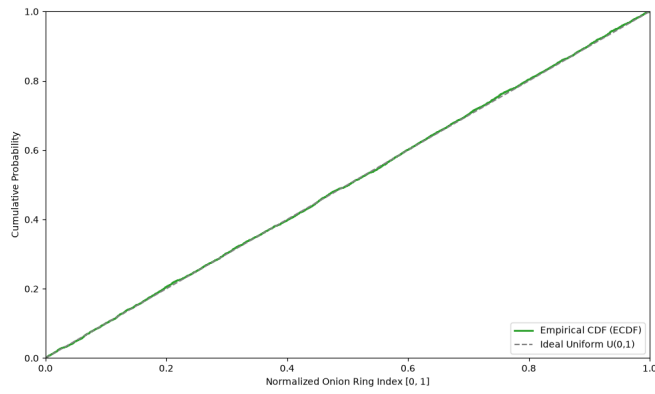


Fig. 87. Onion index: uniformity ECDF — tracked.

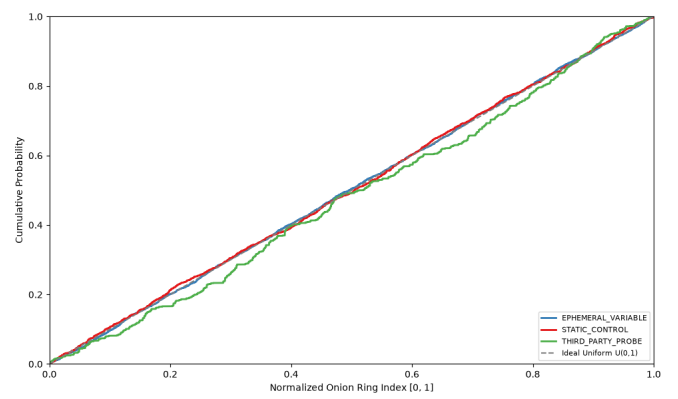


Fig. 91. Onion index: uniformity ECDF — service-type comparison.

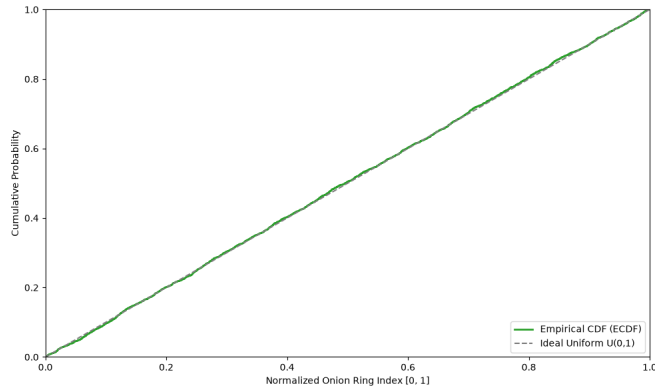


Fig. 88. Onion index: uniformity ECDF — EV.

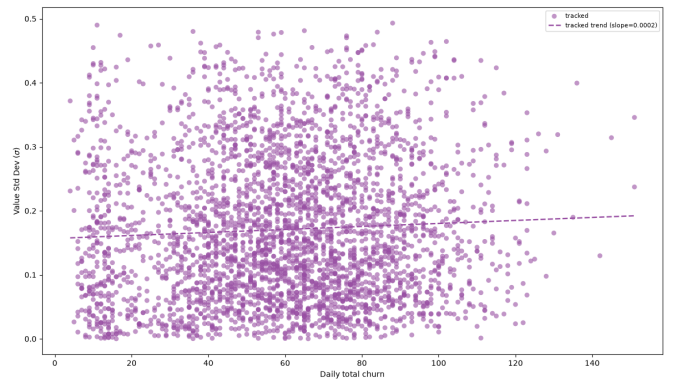


Fig. 92. Onion index: churn vs. dispersion — tracked.

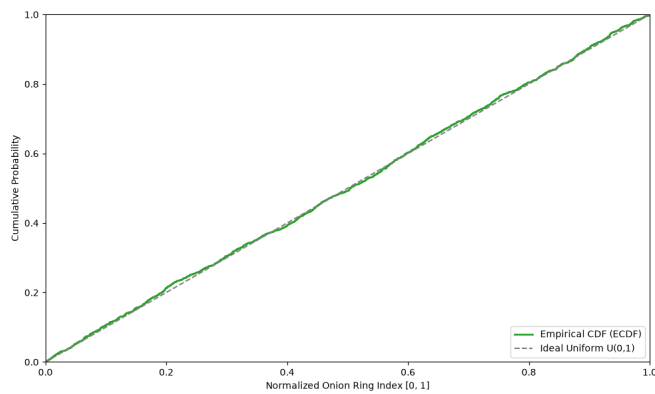


Fig. 89. Onion index: uniformity ECDF — SC.

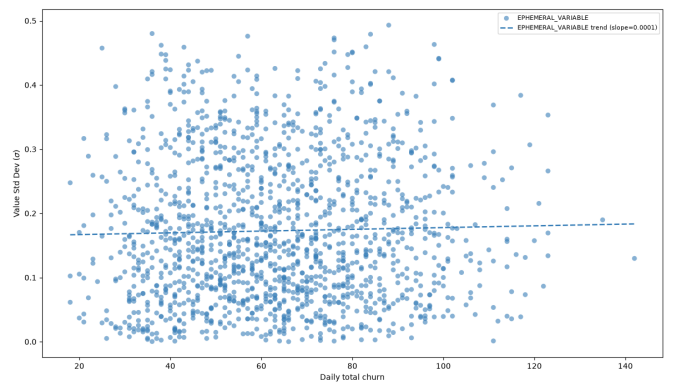


Fig. 93. Onion index: churn vs. dispersion — EV.

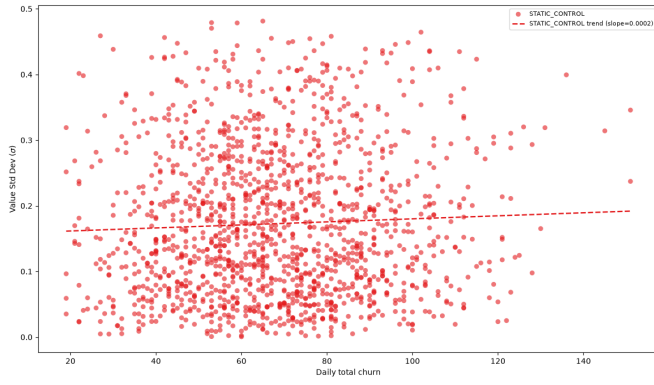


Fig. 94. Onion index: churn vs. dispersion — SC.

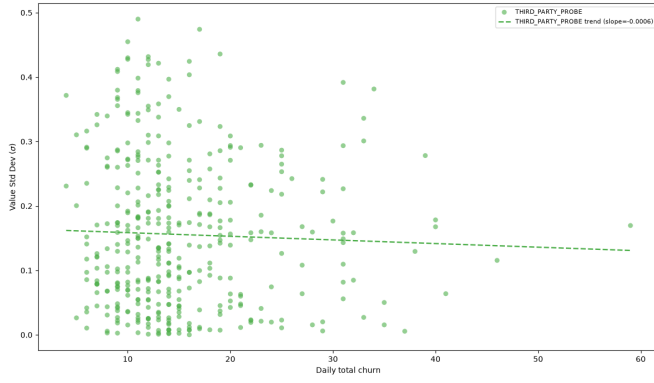


Fig. 95. Onion index: churn vs. dispersion — TP.

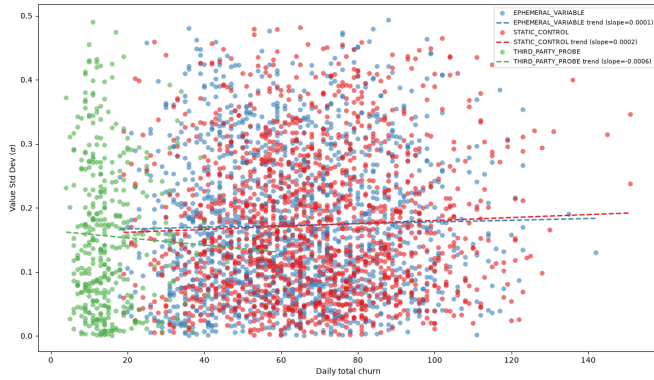


Fig. 96. Onion index: churn vs. dispersion — service types.

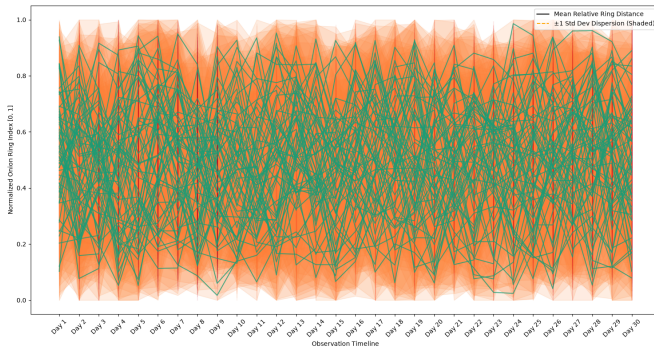


Fig. 97. Onion index: per-onion distance dispersion — tracked.

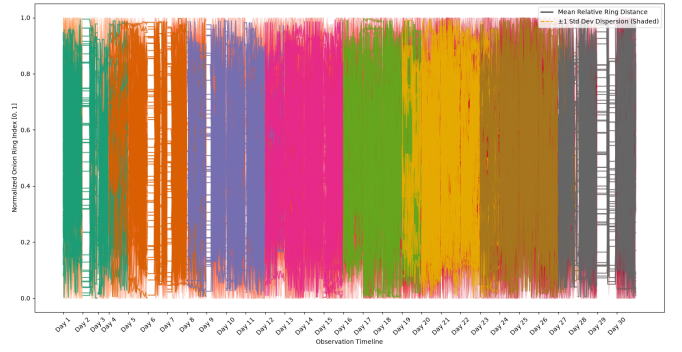


Fig. 98. Onion index: per-onion distance dispersion — EV.

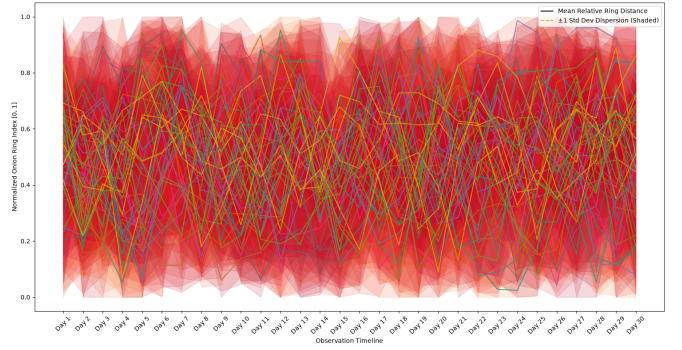


Fig. 99. Onion index: per-onion distance dispersion — SC.

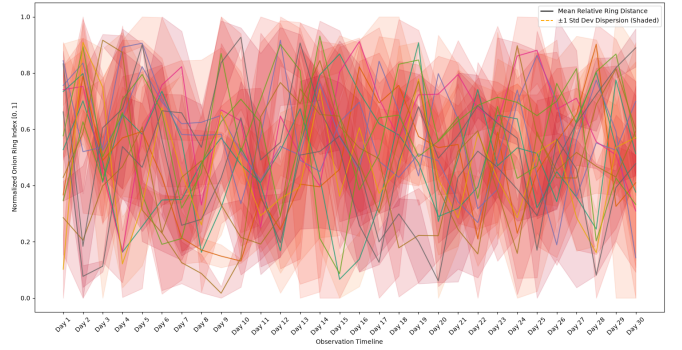


Fig. 100. Onion index: per-onion distance dispersion — TP.

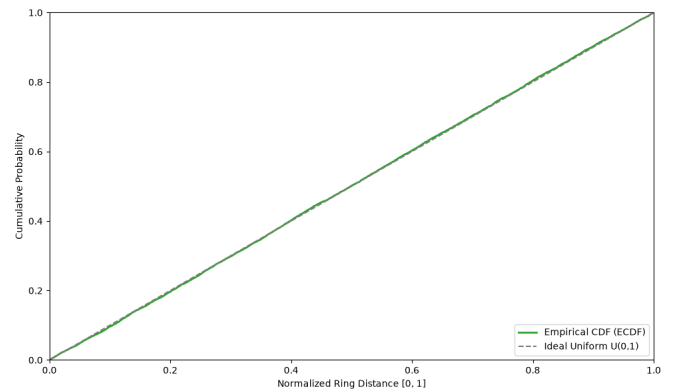


Fig. 101. Onion-to-HSDir distance: uniformity ECDF — tracked.

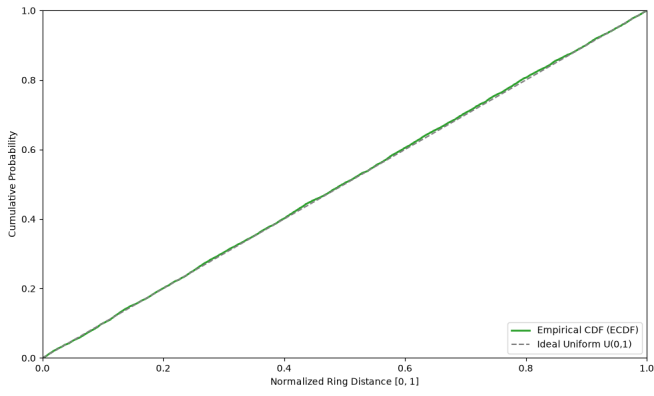


Fig. 102. Onion-to-HSDir distance: uniformity ECDF — EV.

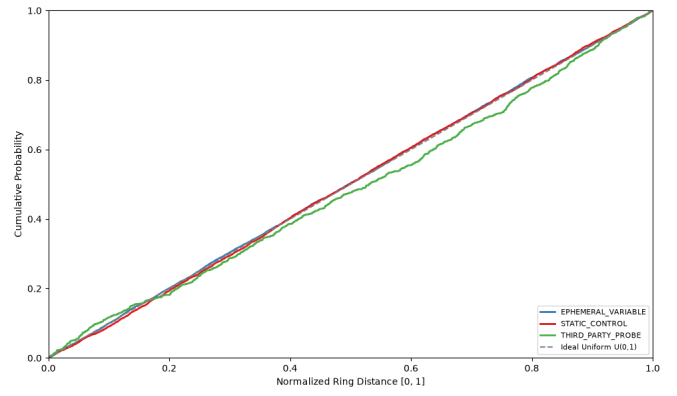


Fig. 105. Onion-to-HSDir distance: uniformity ECDF — service types.

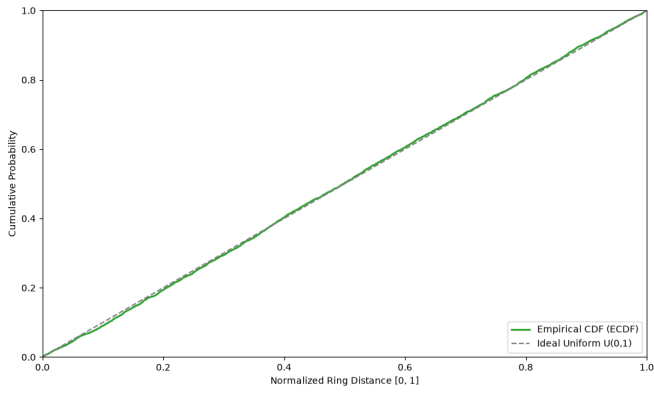


Fig. 103. Onion-to-HSDir distance: uniformity ECDF — SC.

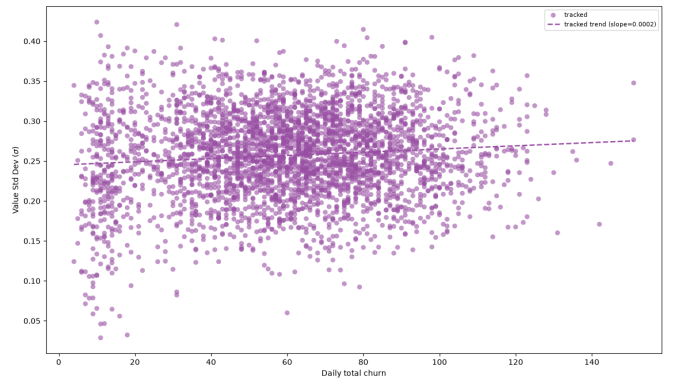


Fig. 106. Onion-to-HSDir distance: churn vs. dispersion — tracked.

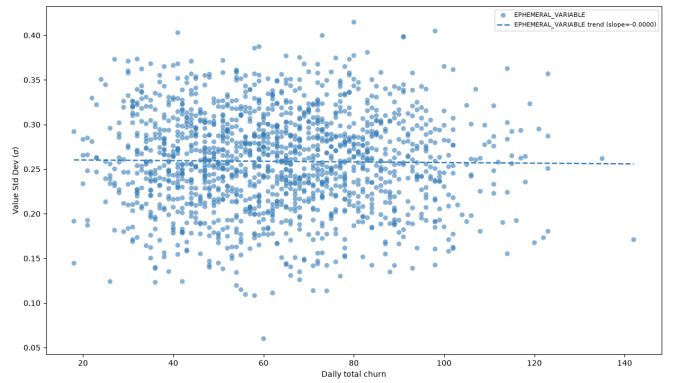


Fig. 107. Onion-to-HSDir distance: churn vs. dispersion — EV.

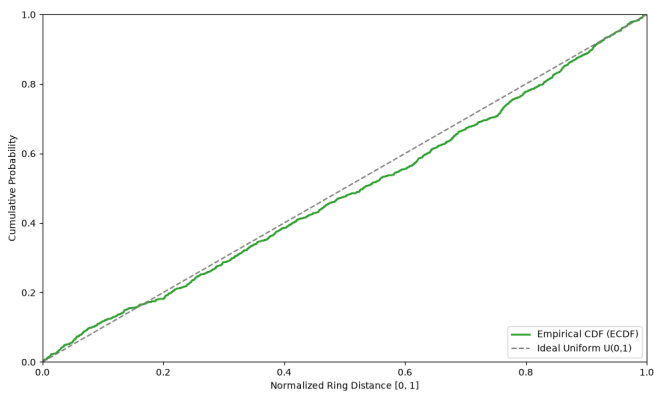


Fig. 104. Onion-to-HSDir distance: uniformity ECDF — TP.

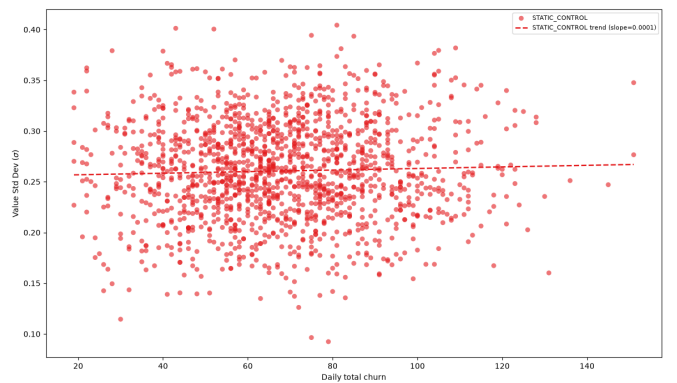


Fig. 108. Onion-to-HSDir distance: churn vs. dispersion — SC.

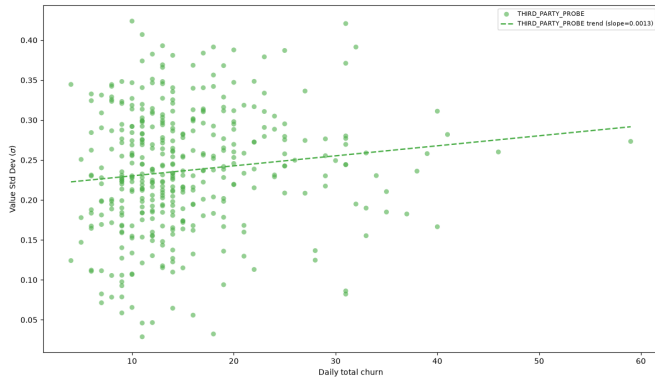


Fig. 109. Onion-to-HSDir distance: churn vs. dispersion — TP.

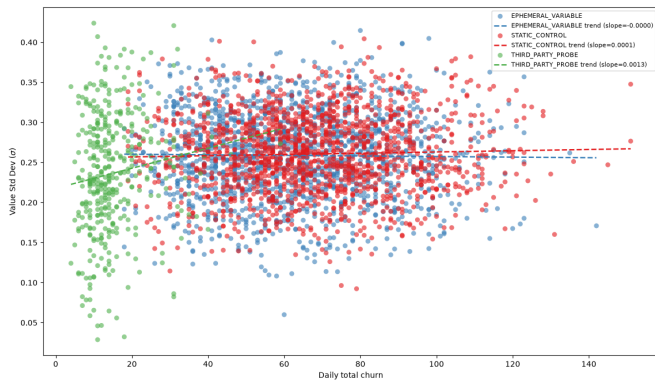


Fig. 110. Onion-to-HSDir distance: churn vs. dispersion — service types.

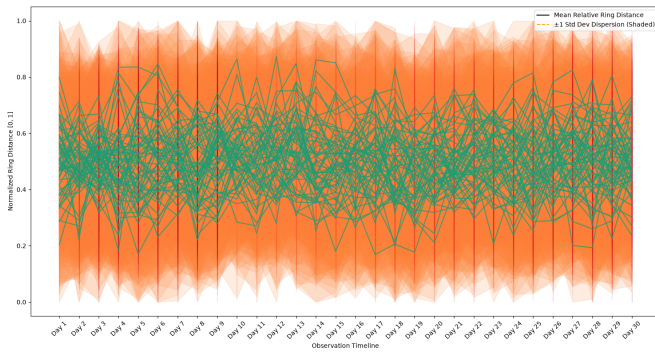


Fig. 111. Onion-to-HSDir distance: per-onion distance dispersion — tracked.

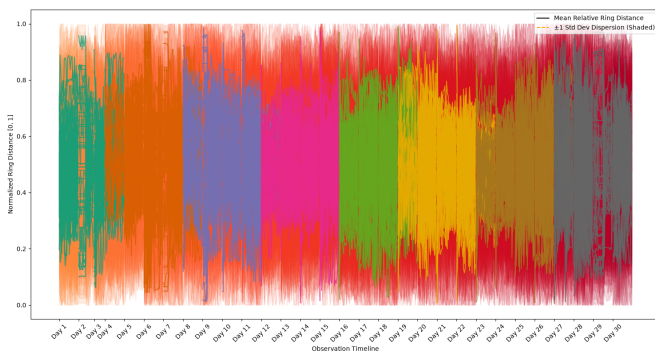


Fig. 112. Onion-to-HSDir distance: per-onion distance dispersion — EV.

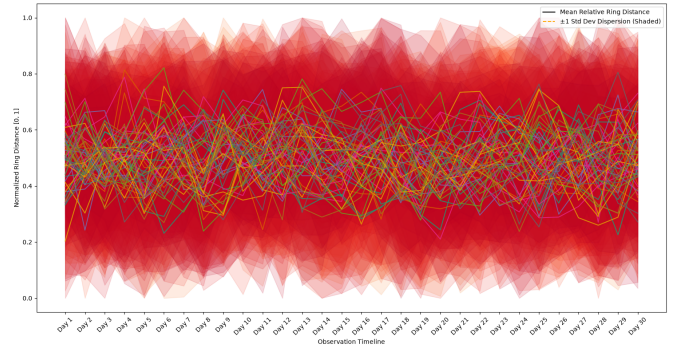


Fig. 113. Onion-to-HSDir distance: per-onion distance dispersion — SC.

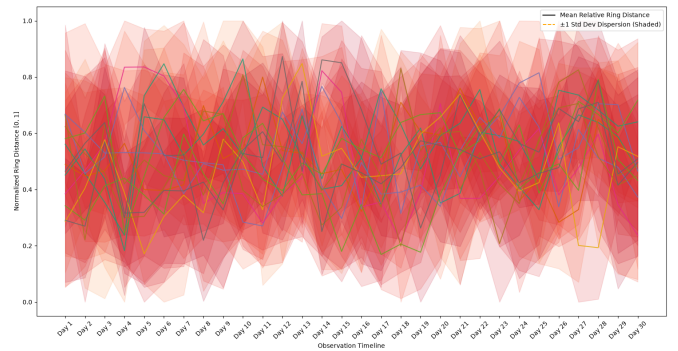


Fig. 114. Onion-to-HSDir distance: per-onion distance dispersion — TP.

#### D. Descriptor Hosting: Upload/Fetch Persistence

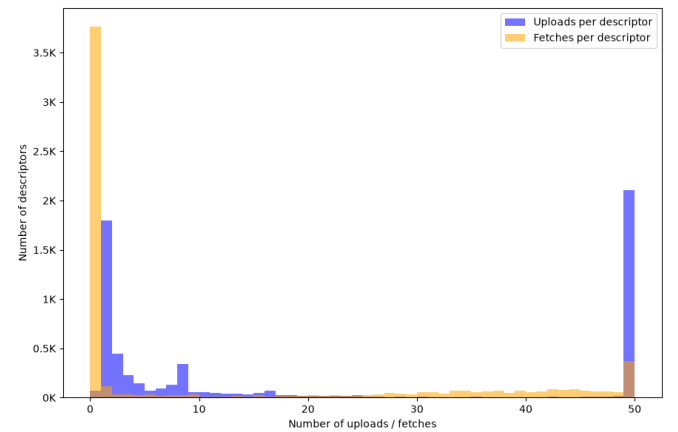


Fig. 115. Per-descriptor upload and fetch counts — EV.

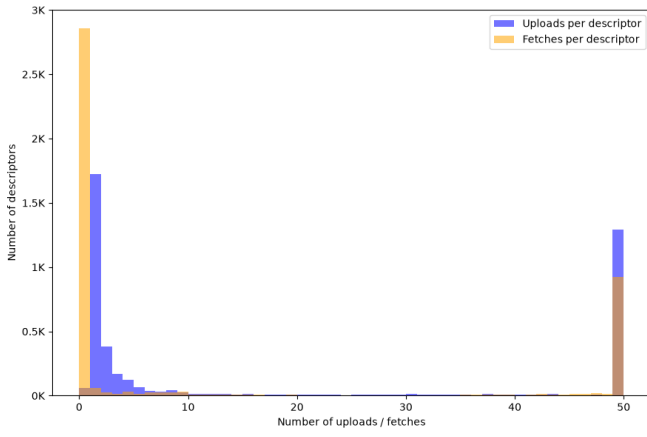


Fig. 116. Per-descriptor upload and fetch counts — SC.

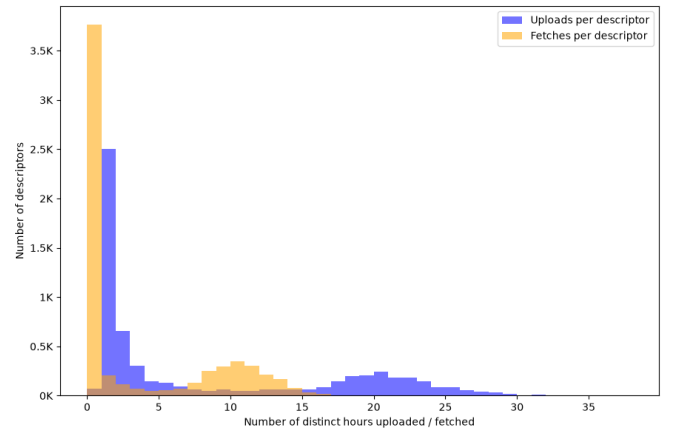


Fig. 119. Distinct-hourly-bucket upload and fetch persistence — EV.

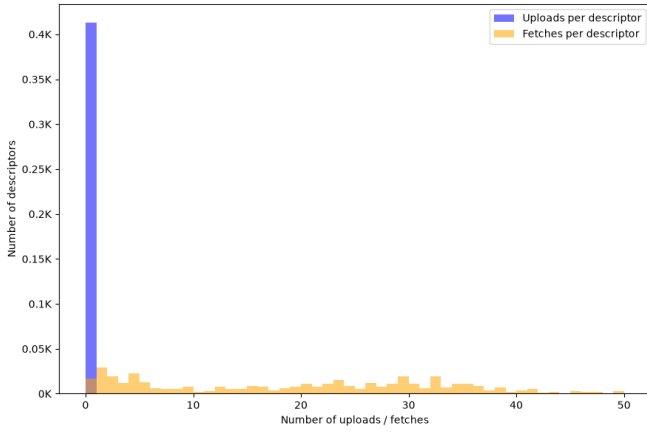


Fig. 117. Per-descriptor upload and fetch counts — TP.

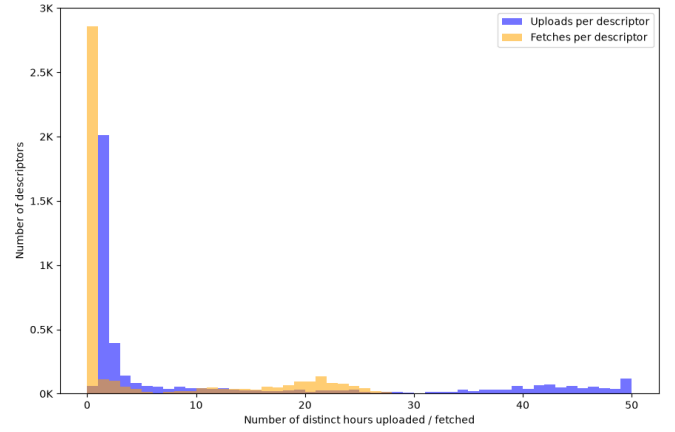


Fig. 120. Distinct-hourly-bucket upload and fetch persistence — SC.

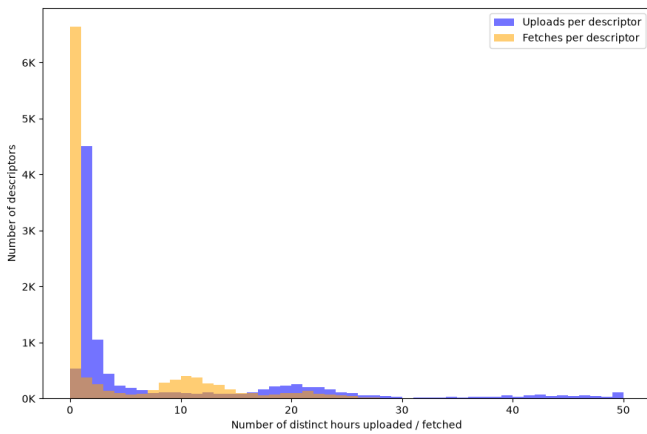


Fig. 118. Distinct-hourly-bucket upload and fetch persistence — tracked.

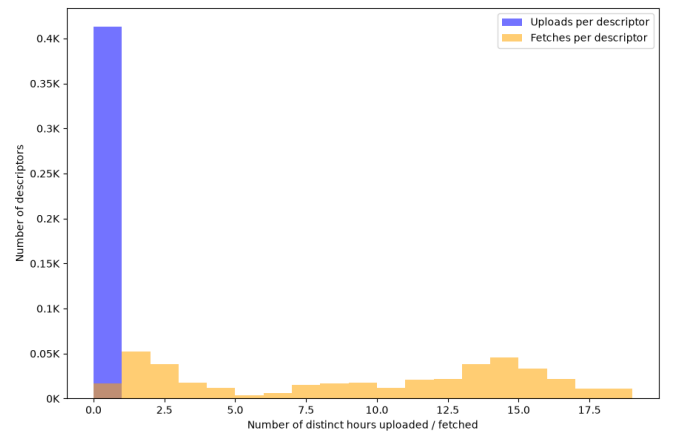


Fig. 121. Distinct-hourly-bucket upload and fetch persistence — TP.

| Population | Desc. | Fail ev. | Fail desc. | Max up. | Max fet. |
|------------|-------|----------|------------|---------|----------|
| Tracked    | 10881 | 8431     | 2478       | 378     | 123      |
| EV         | 6186  | 4222     | 1444       | 199     | 72       |
| SC         | 4282  | 4209     | 1034       | 378     | 123      |
| TP         | 413   | 0        | 0          | 0       | 55       |

Table LXXV. Per-descriptor  
upload/fetch totals —  
tracked and by service  
type.

| Population | Max up.-hours | Max fet.-hours |
|------------|---------------|----------------|
| Tracked    | 66            | 32             |
| EV         | 38            | 18             |
| SC         | 66            | 32             |
| TP         | 0             | 19             |

Table LXXVI. Distinct-hourly-  
bucket upload/fetch  
persistence — tracked  
and by service type.

|            | Up., never fet. |       | Fet., never up. |       |
|------------|-----------------|-------|-----------------|-------|
| Population | Count           | %     | Count           | %     |
| Tracked    | 6502            | 59.76 | 403             | 3.70  |
| EV         | 3698            | 59.78 | 5               | 0.08  |
| SC         | 2804            | 65.48 | 2               | 0.05  |
| TP         | 0               | 0.00  | 396             | 95.88 |

Table LXXVII. Silent-failure  
descriptor  
classification —  
tracked and by  
service type.

## APPENDIX B

### DECLARATION OF USE OF GENERATIVE ARTIFICIAL INTELLIGENCE (AI)

## DECLARATION OF USE OF GENERATIVE ARTIFICIAL INTELLIGENCE (AI) IN MASTER FINAL PROJECT (TFM)

I have used Generative AI in my TFM

Check all that apply:

**YES** ☐ **NO** ☐

If you have ticked YES, please complete the following 3 parts of this document:

### Part 1: Reflection on legal, ethical and responsible behavior

Please be aware that the use of AI carries some risks and may lead to a series of serious academic consequences: the TFM will not be evaluated by the University if the use of AI involves the use of confidential data, copyrighted materials, or personal data, used without complying with the conditions required in each case (authorization of the interested parties, authorization of the owners, following the instructions of the University).

| Question                                                                                                                                                                                                                                                                                                                                                                          |                                                  |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| <p>1. In my interactions with AI tools, I have provided <b>confidential data</b>, always with the appropriate authorization of the data subjects. Confidentiality encompasses any information that a person or organization wishes to protect for legal, commercial, privacy, or strategic reasons (such as patents or trade secrets).</p>                                        |                                                  |
| <p>YES, I have used this data with permission</p>                                                                                                                                                                                                                                                                                                                                 | <p>NO, I have not used confidential data</p>     |
| <p>2. In my interaction with Generative AI tools, I have submitted <b>copyrighted materials</b> with the permission of those concerned.</p>                                                                                                                                                                                                                                       |                                                  |
| <p>YES, I have used these materials with permission from the copyright holders, or without permission because they fall within one of the exceptions or limitations permitted by law:</p> <ul style="list-style-type: none"> <li>• Public domain work</li> <li>• Licensed work (Creative Commons licenses)</li> <li>• Use of excerpts for research purposes (fair use)</li> </ul> | <p>NO, I have not used copyrighted materials</p> |

|                                                                                                                                                                                                                                                                                                                                                                                       |                                   |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| 3. In my interaction with Generative AI tools, I have submitted <b>personal data</b> with the consent of the data subjects.                                                                                                                                                                                                                                                           |                                   |
| YES, I have used this data with the authorization of the interested parties and in accordance with the instructions contained in the <a href="#">guide approved by the University</a> .                                                                                                                                                                                               | NO, I have not used personal data |
| 4. My use of the Generative AI tool has <b>respected its terms of use</b> , as well as the essential ethical principles, not being maliciously oriented to obtain an inappropriate result for the work presented, that is to say, one that produces an impression or knowledge contrary to the reality of the results obtained, that supplants my own work or that could harm people. |                                   |
| YES                                                                                                                                                                                                                                                                                                                                                                                   | NO                                |

## Part 2: Declaration of technical use

### Documentation and drafting:

- *Supporting reflection in relation to the development of the work: iterative process of analysing alternatives and approaches using AI*

I declare that I have made use of the Generative AI system **Gemini Flash 3.6** for **documentation and drafting**, specifically to help me understand, on several occasions, the content of the official technical sources underpinning this work, where that content was ambiguous or difficult to process, so that I could address those ambiguities appropriately in my own writing, using the prompt: *"I am reading the official Tor specification on the shared random value and on onion-service descriptor rotation. It says the shared random value is produced once per 1440-minute period and that descriptors are re-uploaded at time-period boundaries. Can you explain, in plain terms, how the shared random value's daily refresh and the descriptor's time-period rotation relate to each other, and whether they happen on the same schedule?"*, having as interaction the clarification of the relationship between the 24-hour shared-random-value refresh and the descriptor time-period rotation.

- *Revision or rewriting of previously drafted paragraphs*

I declare that I have made use of the Generative AI system **Duck AI, with model GPT-5.6 Luna**, for the **revision or rewriting of previously drafted paragraphs**, to help adapt paragraphs I had already written into the formal register expected of an academic research paper. I requested complete rewrites of passages whose information was particularly hard to adapt to that register, and I also used it to obtain a suitable synonym for a given word by supplying a complete sentence as context and asking for a synonym that fit that context, using the prompt: *"Rewrite the following paragraph so that it reads in the formal, precise style of a security research paper, keeping all technical facts unchanged: [paragraph I had already drafted]"*, having as interaction the adoption of a revised wording of that paragraph, which I reviewed and edited before incorporating it and, separately, using the prompt: *"In the sentence '[complete sentence]', suggest a more precise synonym for the word '[word]' that fits this context"*, having as interaction the selection of the proposed synonym where it genuinely fit the sentence.

- *Search for information or answers to specific questions*

I declare that I have made use of the Generative AI system **Gemini Flash 3.6** for the **search for information or answers to specific questions**, to answer specific questions I had about the official information on Tor when that information was ambiguous or difficult to comprehend, using the prompt: *"According to Tor's official specifications, at what UTC time is the new shared random value published each day, and how long before that boundary should a measurement tool stop trusting the current period's descriptor positions to avoid mixing two time periods?"*, having as interaction the answer regarding the 00:00 UTC period boundary and the practical implication of an exclusion window around it.

- *Bibliography search*

I declare that I have made use of the Generative AI system **Claude Opus 4.8 (High effort)** for **bibliography search**, to locate specific, verifiable, peer-reviewed studies dealing precisely with Hidden Service Directories (HSDirs). This was necessary because research addressing exactly what this work measures is scarce: most available literature concerns onion services in general rather than the directory relays themselves, which made finding sufficiently specific related work difficult, using the prompt: *"Find related-work studies on Tor V3 HSDir relay nodes for a thesis bibliography. This is a measurement study of the HSDir relays themselves, not of the onion services (the onion services are only the probe used to discover which HSDirs are active and where they sit in the hash ring). Find 3–4 real, verifiable, peer-reviewed studies for each of these objectives: uptime and stability of v3 HSDir nodes, churn and sybil patterns of v3 HSDirs, anomalous HSDir behaviour (nodes appearing, disappearing, or changing, and whether a failure is node-level or operator-level), related areas such as hash-ring position observation, sybil detection via IP ranges/family, and operator fingerprinting. Only reputable venues or the Tor Project's own research. Every entry needs a working link or DOI. Never invent a reference"*, having as interaction the identification of candidate studies, each with a verifiable DOI or link and a justification of its relevance, which I then reviewed, verified independently.

- *Summary of bibliography consulted - N/A*
- *Translation of texts consulted - N/A*

### ***Develop specific content***

*Generative AI has been used as a support tool for the development of the specific content of the TFM, including:*

- *Assistance in the development of lines of code (programming)*

I declare that I have made use of the Generative AI systems **Gemini Flash 3.6**, **Claude Sonnet 5 (Medium effort)**, and, on occasions where the logic of the analyses was of particularly high complexity, **Claude Opus 4.8 (High effort)**, for **assistance in the development of lines of code (programming)**. The programming logic and design were my own, the AI tools implemented that logic under my direction and supervision. The data-collection tool which interacts with Tor's ControlPort over a concurrent, five-thread architecture (a network-wide consensus auditor, a tracked-HSDir consensus monitor, an active descriptor prober issuing lookups, an asynchronous event listener, and a scaled ephemeral-onion manager), with locks to prevent race conditions during continuous logging was developed with the assistance of **Gemini Flash 3.6**. The analysis program, which comprises a large set of analysis functions (uptime and stability, churn and flag dynamics, sybil concentration via family-identifier and IP/subnet clustering, hash-ring position and coverage, and descriptor upload/fetch hosting), was developed with the assistance of **Claude Sonnet 5 (Medium effort)**, escalating to **Claude Opus 4.8 (High effort)** for the higher-complexity analysis logic.

- *Generation of diagrams, images, audios or videos - N/A*
- *Optimisation processes*

I declare that I have made use of the Generative AI systems **Gemini Flash 3.6** and **Claude Sonnet 5 (Medium effort)** for **optimisation processes**. I used **Gemini Flash 3.6** to optimise the data-collection processes of the fetcher, removing inconsistencies and unnecessary steps from the collection pipeline. I used **Claude Sonnet 5 (Medium effort)** to optimise several of the analysis processes, given the considerable computational effort required to produce the results presented in this work for example, replacing slower row-by-row processing with vectorised operations so that a month of hourly data could be analysed efficiently.

- *Data processing: collection, analysis, cross-checking of data...*

I declare that I have made use of the Generative AI systems **Gemini Flash 3.6**, **Claude Sonnet 5 (Medium effort)**, and, where the logic of the analyses was of particularly high complexity, **Claude Opus 4.8 (High effort)**, for **data processing (collection, analysis, and cross-checking of data)**. I used **Gemini Flash 3.6** to help build the information fetcher, and **Claude Sonnet 5 (Medium effort)** and **Claude Opus 4.8 (High effort)** to help build the analysis program. I also used these tools to cross-check the processing against a small training/example dataset with known characteristics, to confirm that the computed results and the generated visualisations were correct and internally consistent, and to help resolve any inconsistencies found in that check, using the prompt: "Add a complete Python function to `hmdir-analysis.py` that measures sybil behaviour among HSDir relays. Given a dataframe and a context label, for each hourly snapshot group the relays by their family identifier (`declared_family_id`), and for each family count active relays (status `ACTIVE_IN_RING`) versus inactive relays. Classify the operator as a partial drop if some of its relays are inactive while others remain active, and a total drop if all of its relays are inactive; also compute the network-share of relays dropped per snapshot. Plot per-day absolute partial/total drop counts and their network-shares, save the figure under `analysis-results/sybil/family-id/`, and print the top-5 families by each drop type. If the context label is 'tracked', additionally repeat the whole analysis per service type. Exclude the 'None' (no-family) bucket by default. Use only non-deprecated pandas/matplotlib APIs", having as interaction the incorporation of the resulting function into the analysis program, which I then reviewed, ran against the sample data, and verified before use.

- *Inspiration of ideas in the creative process - N/A*
- *Other uses linked to the generation of specific points of the specific development of the paper*

I declare that I have made use of the Generative AI systems **Claude Sonnet 5 (Medium effort)** and **Claude Opus 4.8 (High effort)** for **other uses linked to the generation of specific points of the specific development of the paper**, as a support to the analysis of the results rather than as a substitute for it. After producing the results and their visualisations, I directed a structured re-examination of them. I posed specific questions and specified particular relationships to verify across the different analyses, and the tools carried out those checks and reported back what they found. This directed cross-checking helped in two ways. First, it surfaced connections between results across different analyses for instance, that the third-party-probe population behaved distinctly on several separate metrics for one shared underlying reason, which were consistent in the data but which I had not immediately identified on a first pass. Second, it helped confirm results that were laborious to derive by hand and to flag apparent inconsistencies between an analysis and its visualisation for me to investigate and resolve. In every case the interpretation, the decision on what to verify, and the final conclusions were my own. The tools executed the rechecks I specified and returned their findings for my assessment.

### Part 3: Reflection on utility

Please provide a personal assessment (free format) of the strengths and weaknesses you have identified in the use of Generative AI tools in the development of your work. Mention if it has helped you in the learning process, or in the development or drawing conclusions from your work.

The use of Generative AI tools was, on balance, a clear net benefit to this work, primarily as a comprehension aid and as an implementation accelerator under my direction, rather than as a replacement for my own work.

The most valuable contribution was to my own understanding. Tor's V3 onion-service protocol is documented in dense, highly technical official specifications, and being able to pose targeted questions about ambiguous or hard-to-process passages, which let me arrive at a precise and unambiguous understanding of the material, of the scope of the research, and of what was actually feasible given that material, much faster than working from the raw specifications alone. This clearer comprehension led directly into a better-scoped project and more accurate writing.

The tools were also a substantial help in building the project's technical infrastructure. The research required a considerable amount of engineering before any analysis could begin, a custom, continuously running data collector built on a five-thread concurrent architecture over Tor's ControlPort, a full month of uninterrupted hourly data collection, and then a broad analysis program spanning uptime and stability, churn and flag dynamics, sybil concentration via family and subnet clustering, hash-ring position and coverage, and descriptor upload/fetch hosting. The logic and design of these tools were my own contribution, the AI implemented that logic under my supervision, which let me move faster through a large implementation surface, obtain useful feedback, and resolve issues and inconsistencies more quickly than I could have alone. Given the fixed one-month collection window, time saved on implementation was time that could go toward analysing, correlating, and interpreting the results. Finally, using AI to cross-check results and visualisations against a known example, and to help discover connections across analyses, improved my confidence in the correctness and consistency of the findings.

Nonetheless the tools were only as useful as the direction I gave them. They required precise, well-specified instructions, and their output always needed my review, verification, and correction before it could be trusted or used. Outputs were not reliable enough to accept unchecked, particularly for the higher-complexity analysis logic and for any factual claim about the protocol, which I verified against the official sources. Model and version behaviour also varied, and more capable models were needed for the harder logic. Crucially, the tools did not do the work for me, they did not choose the research question, design the measurement methodology, decide what to measure or why, or draw the conclusions. Those remained my own responsibility. Used with that understanding, Generative AI allowed me to deliver the full scope and infrastructure of this research to a higher standard and in less time, and it genuinely supported my learning throughout.